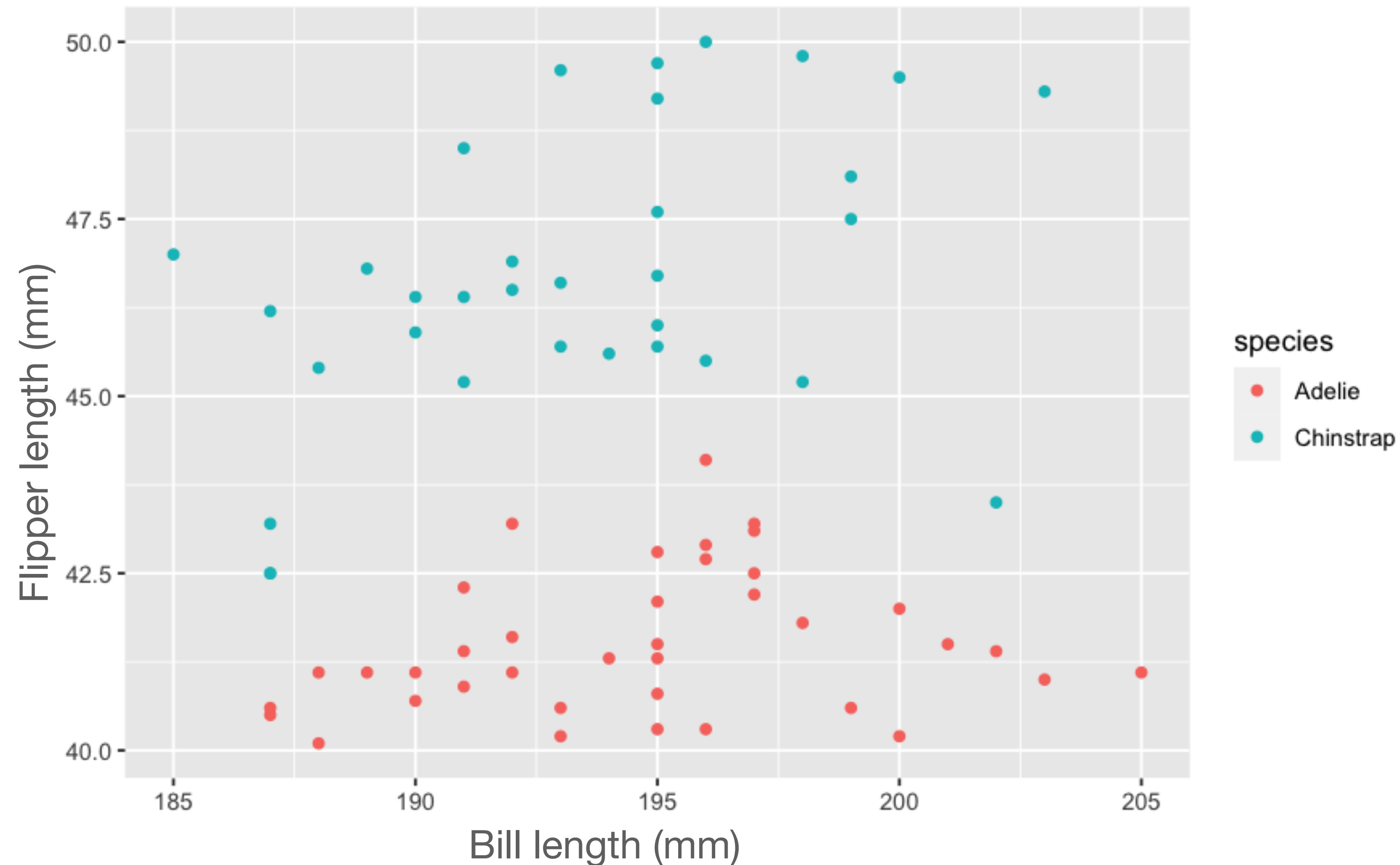


Support Vector Machines

DATA 5322

Dr. Mendible

Support Vector Classifier

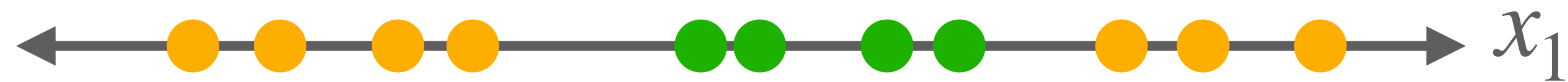


How can we handle data that is not linearly separable?

- ignore some points
- **don't use a straight line**

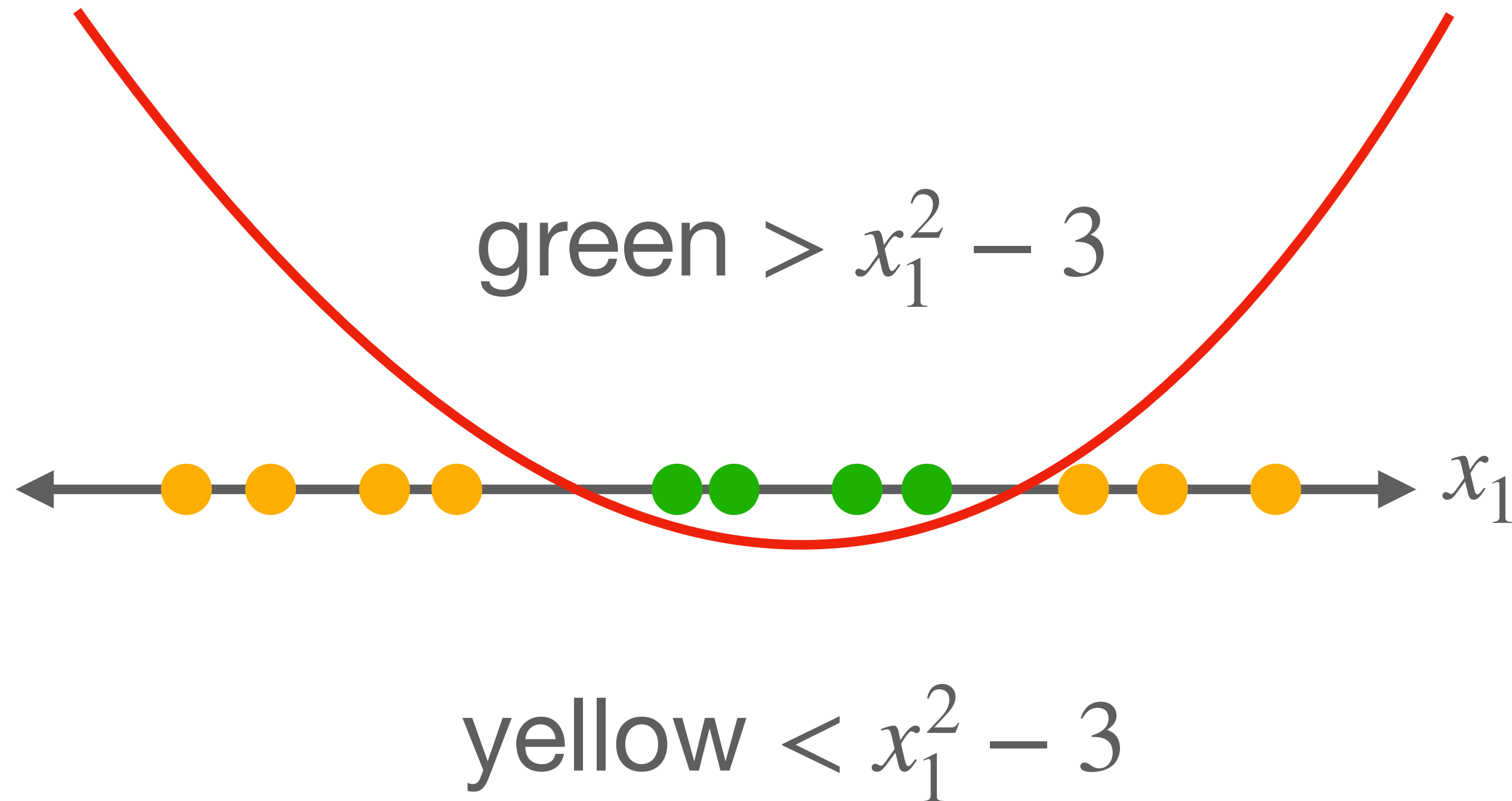
Support Vector Machines

Draw a function (does not need to be linear) to separate the two classes



Support Vector Machines

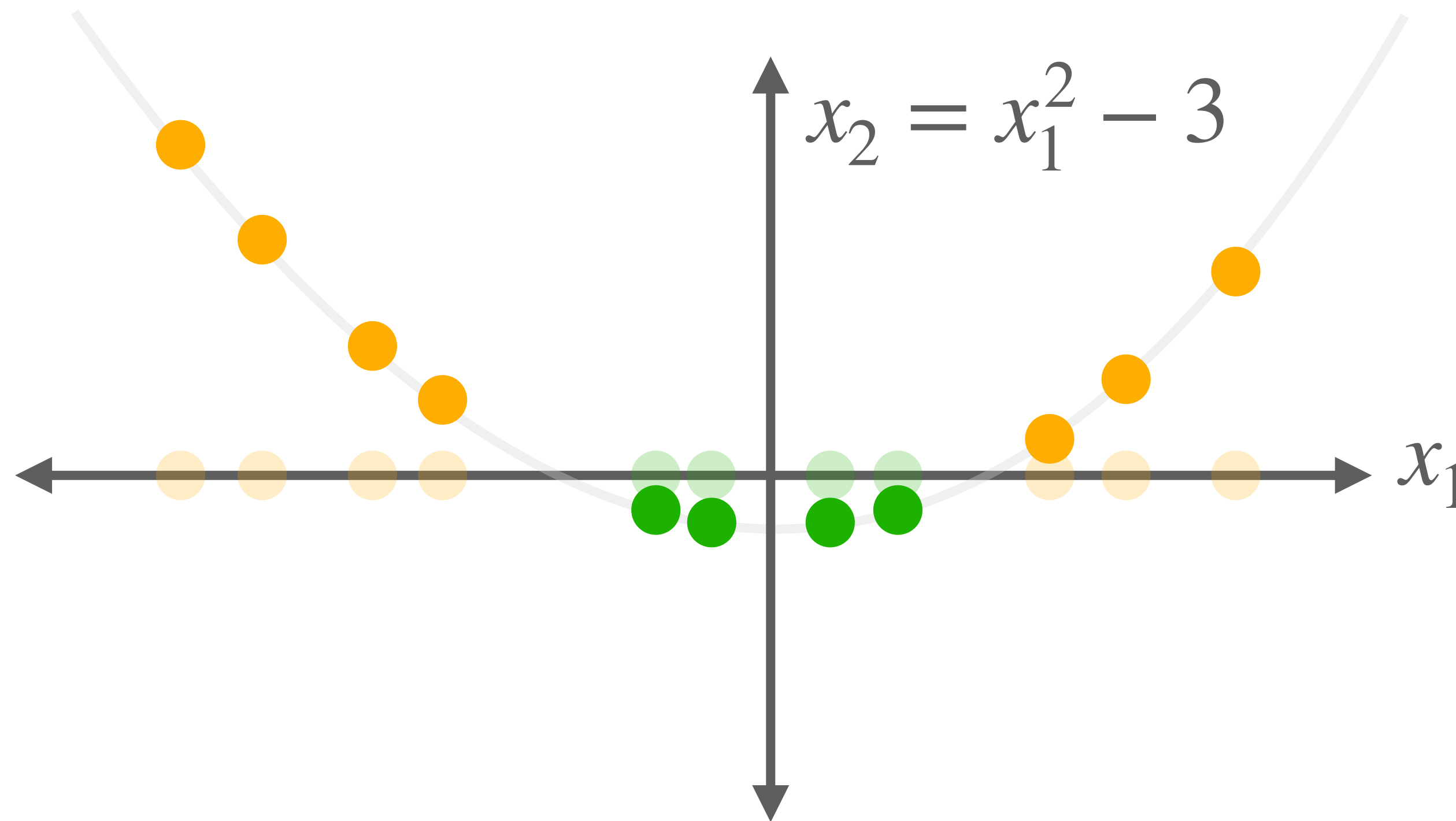
Draw a function (does not need to be linear) to separate the two classes



Support Vector Machines

Draw a function (does not need to be linear) to separate the two classes

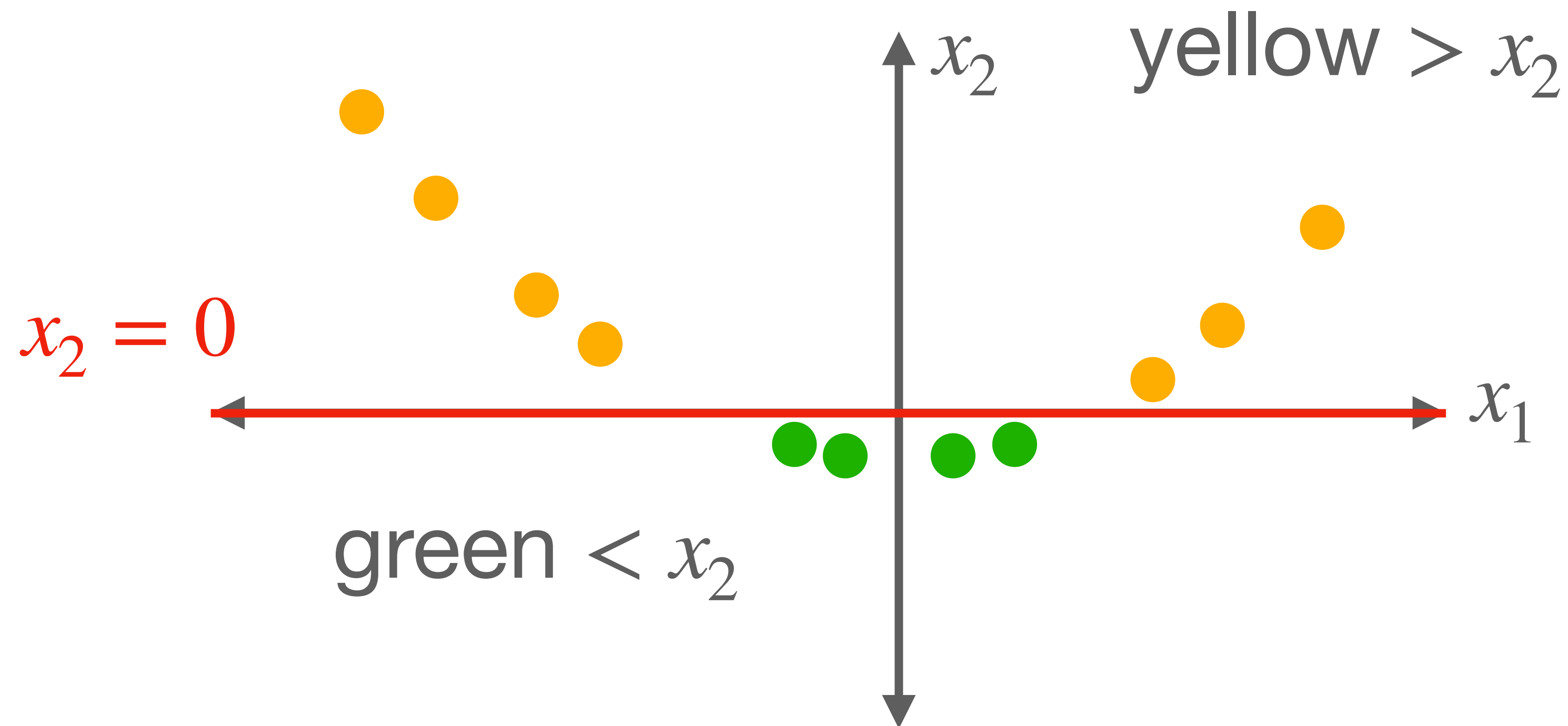
→ or, augment the original data by enlarging the feature space



Support Vector Machines

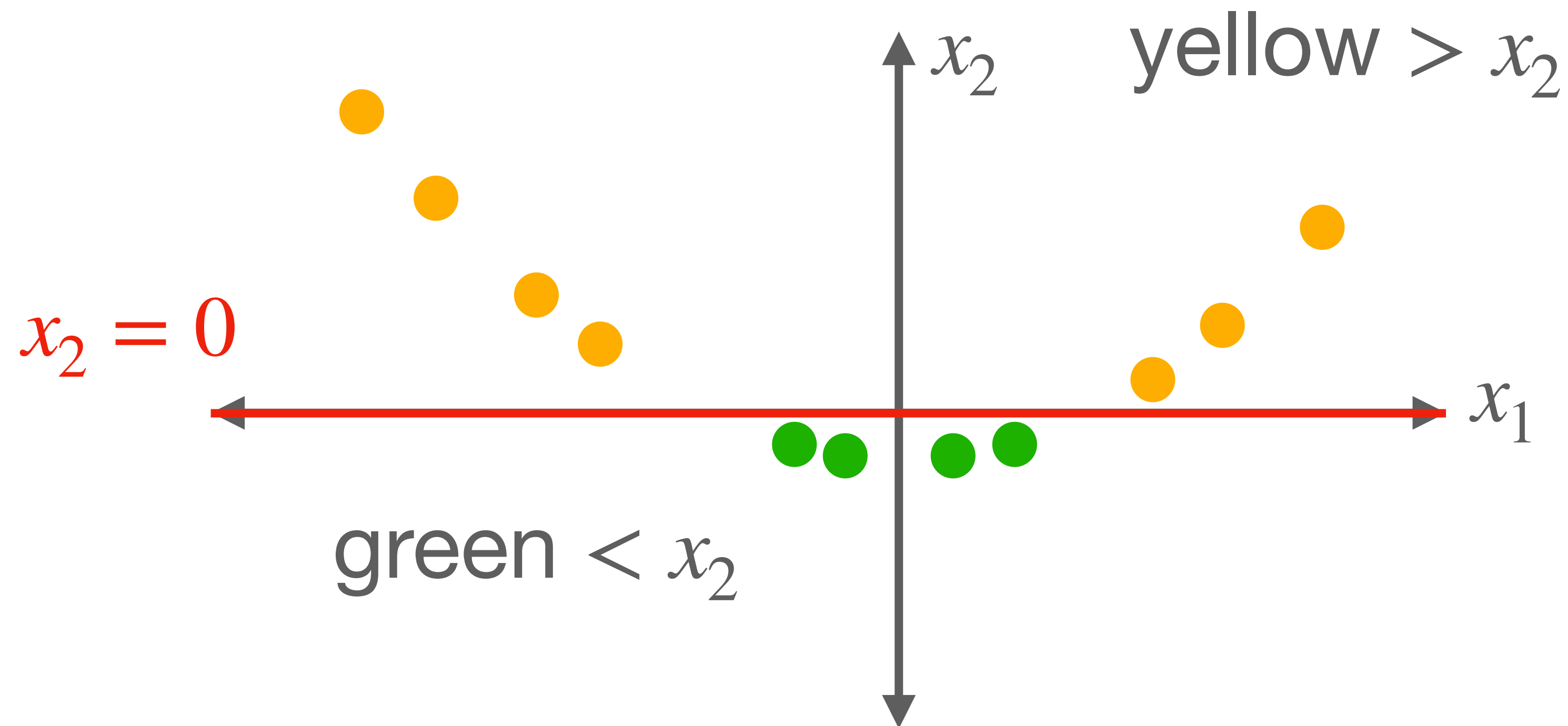
Draw a function (does not need to be linear) to separate the two classes

→ or, augment the original data by enlarging the feature space



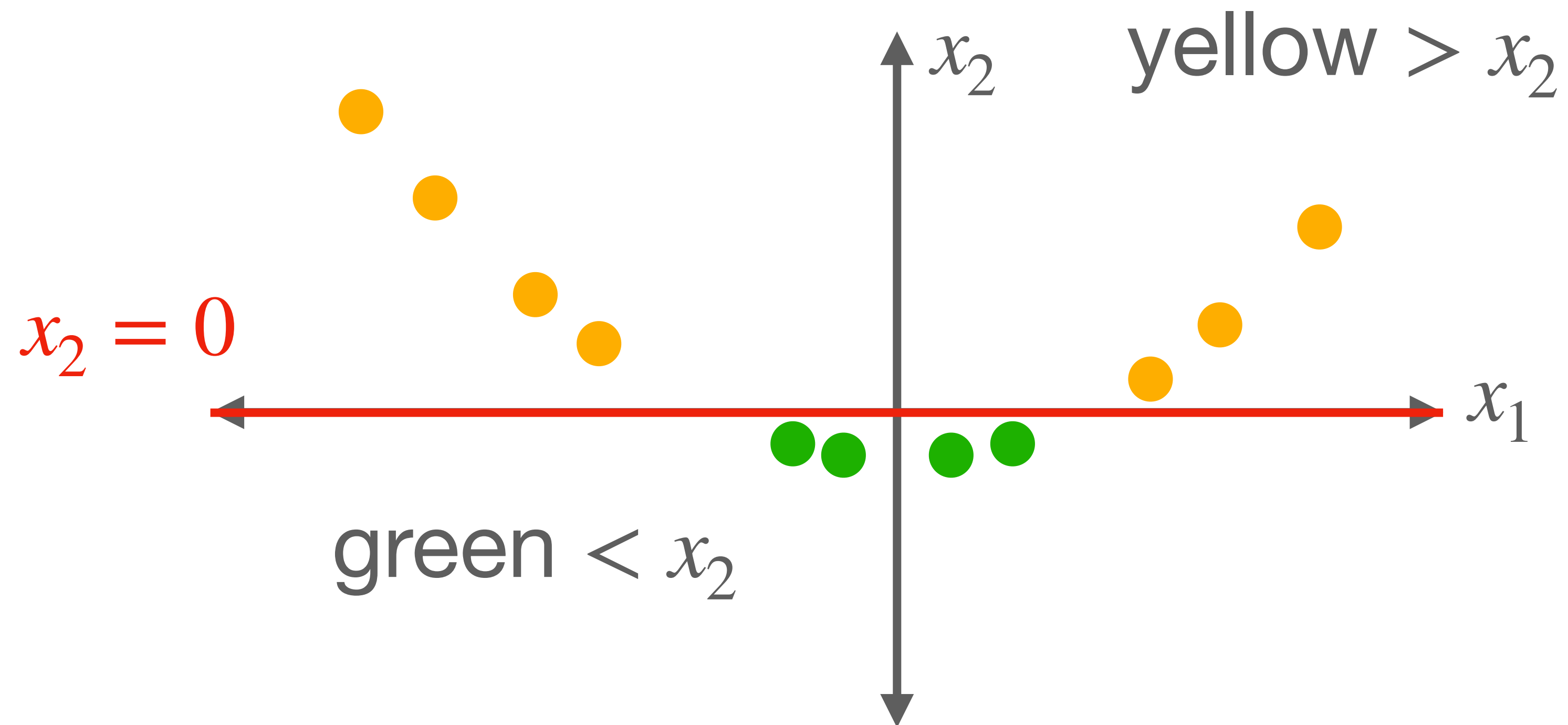
Support Vector Machines

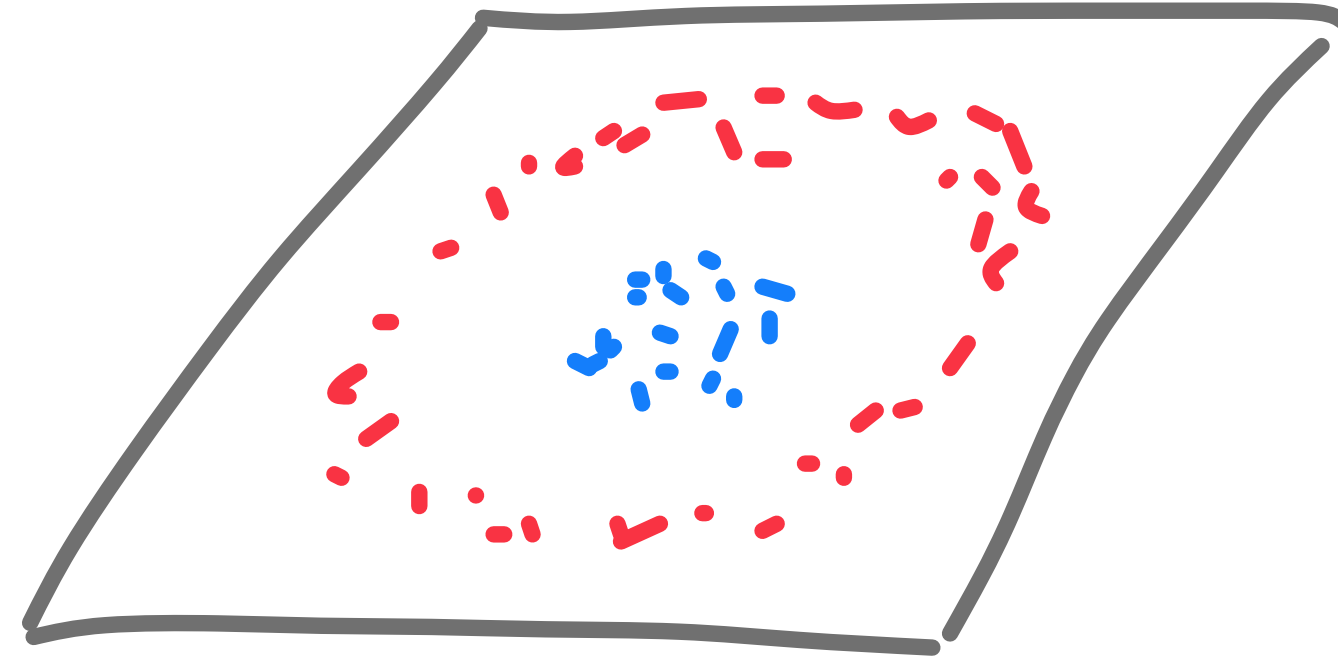
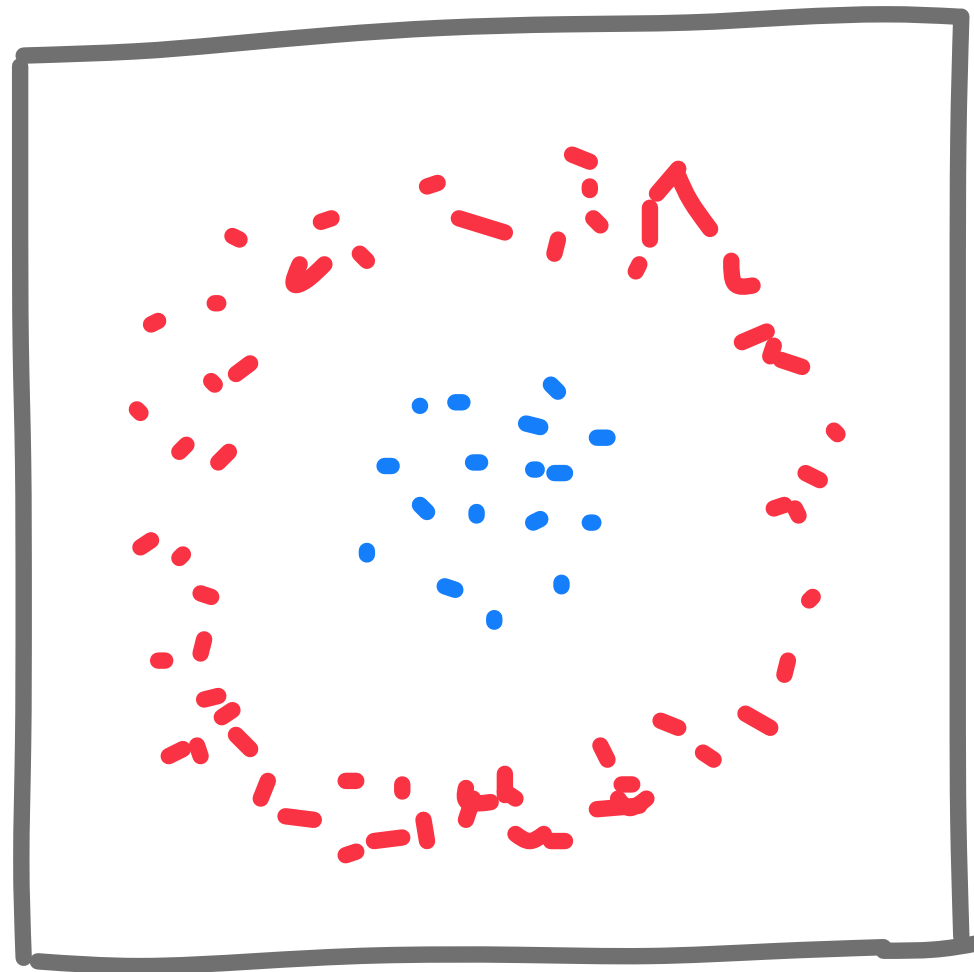
Non-linear cut in current dimension $\rightarrow$ linear cut in higher dimension



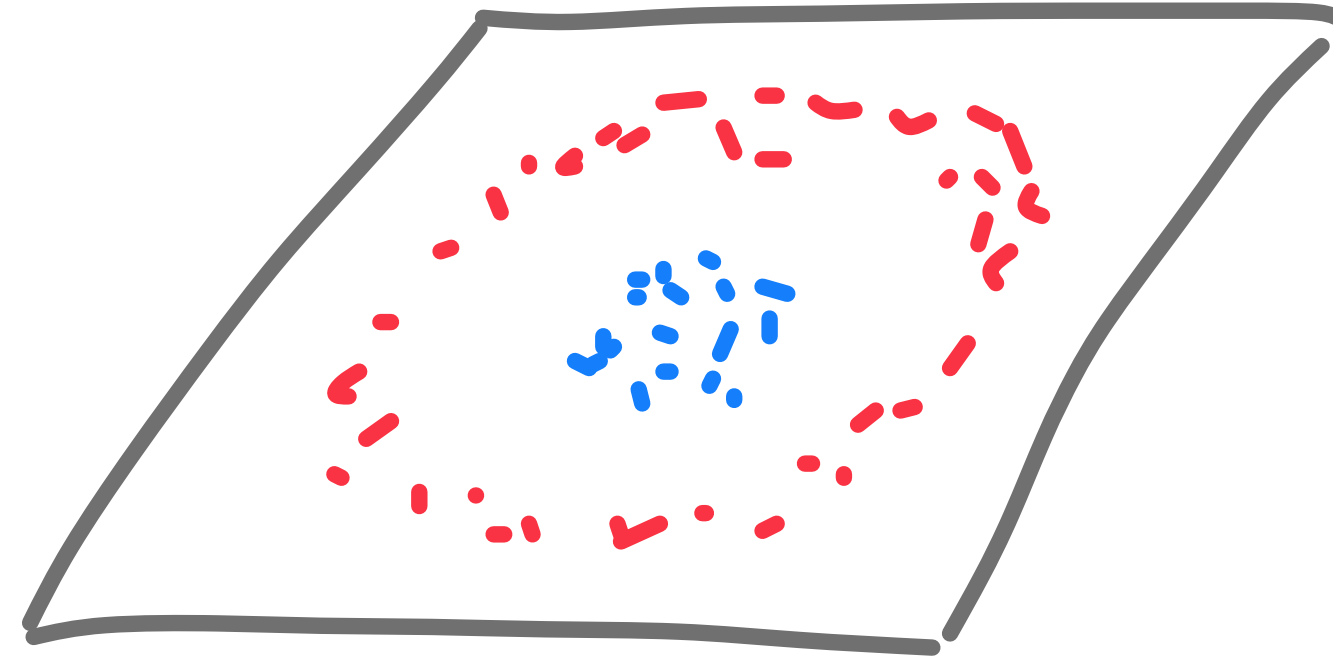
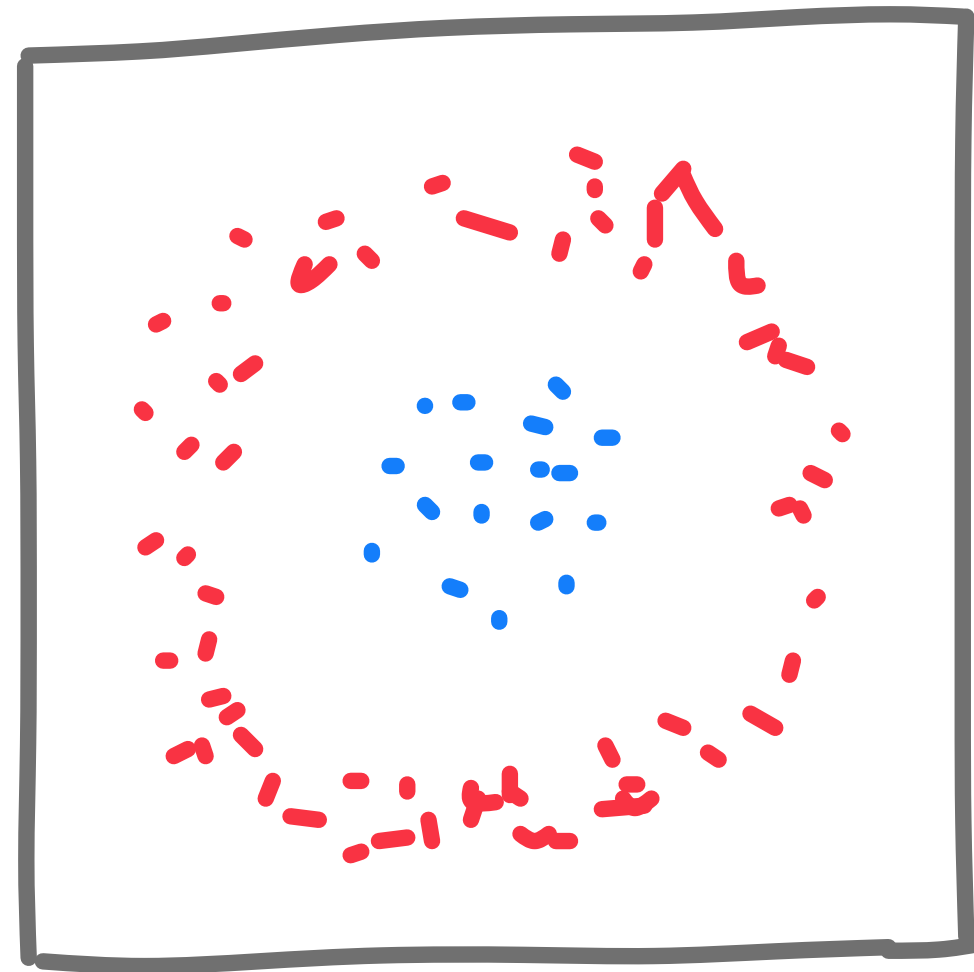
Stop and think...

What are the advantages of transforming the data?

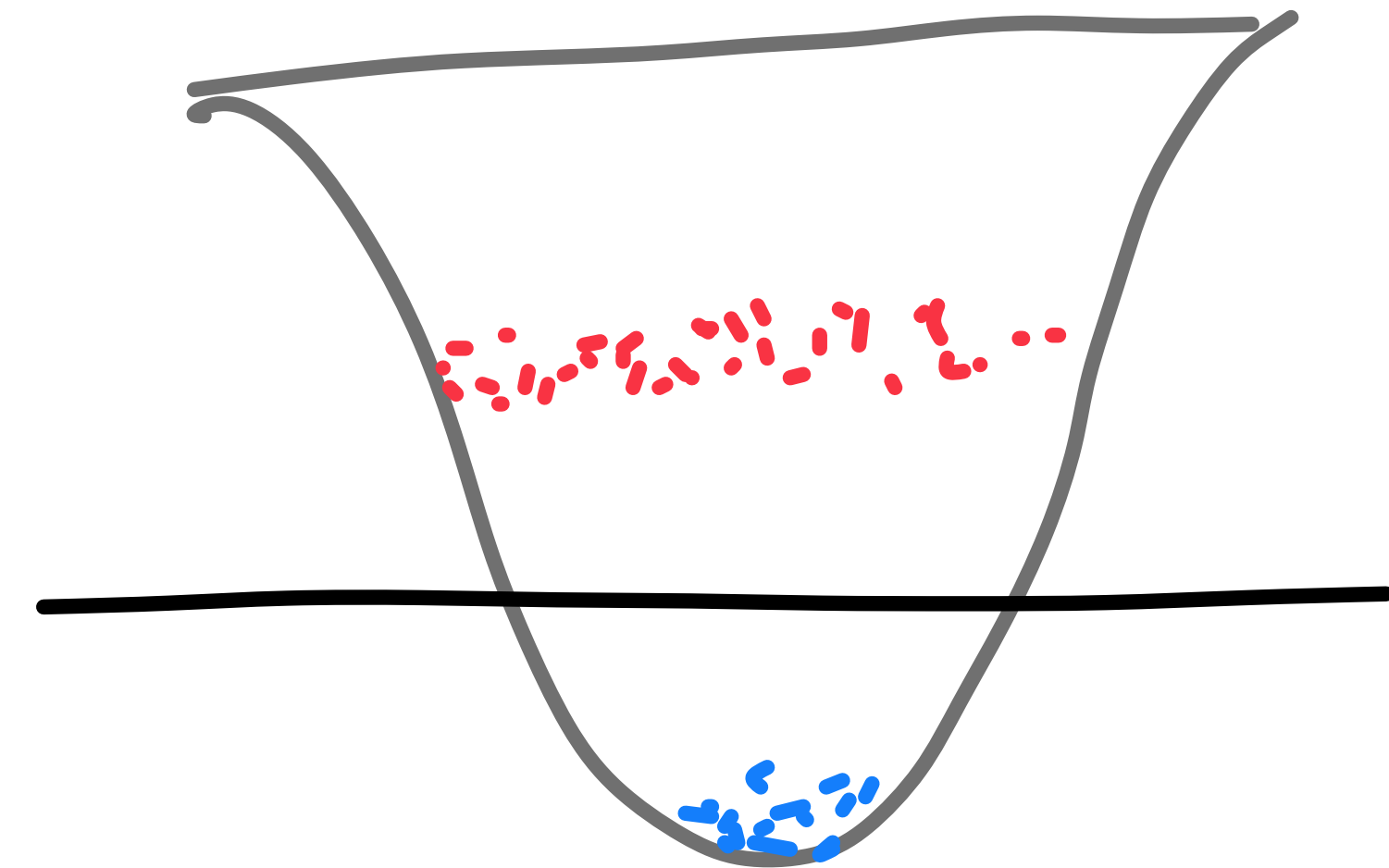
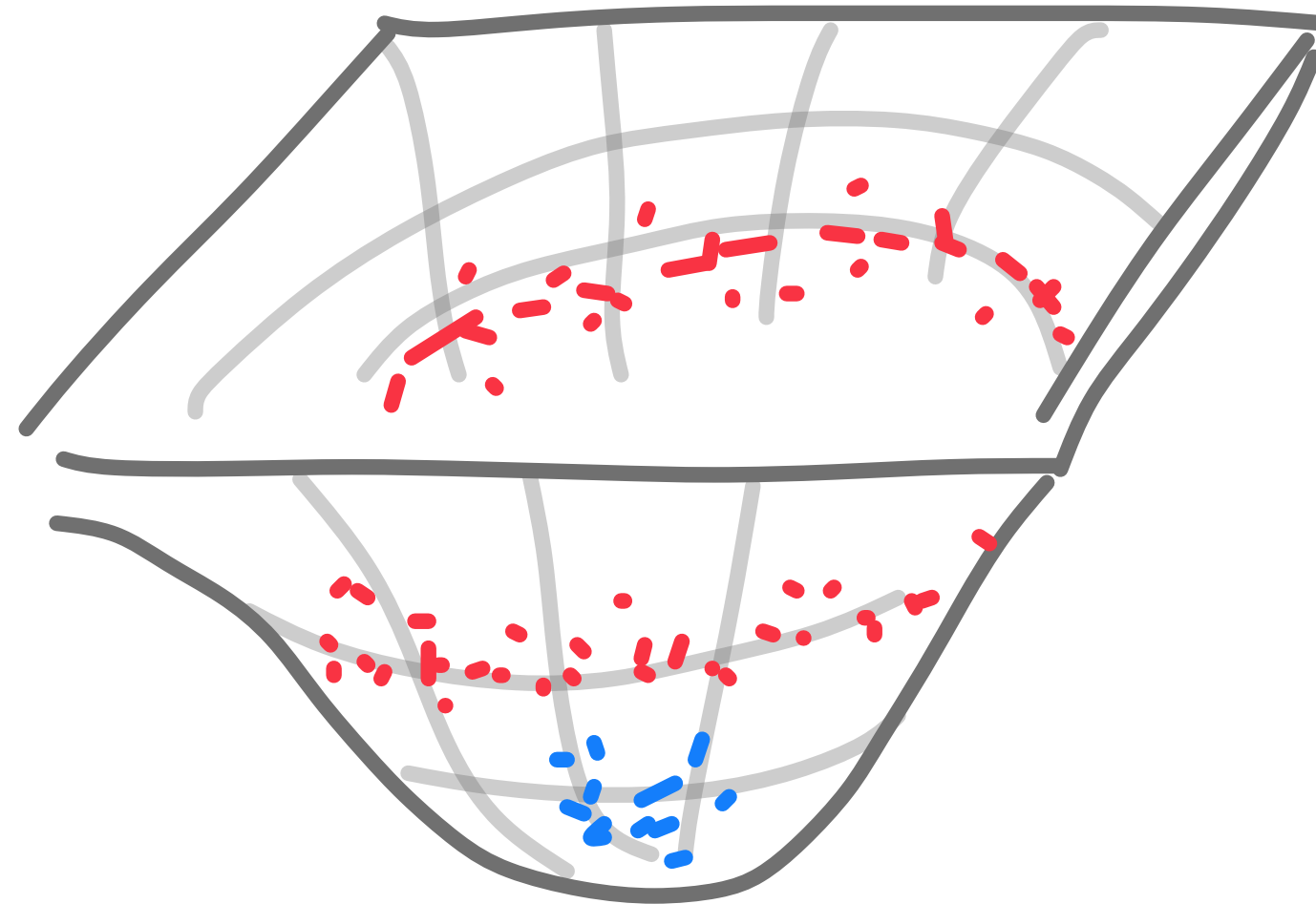




Flat plane in 2 dimensions, no hyperplane (straight line) will separate the two classes



Flat plane in 2 dimensions, no hyperplane (straight line) will separate the two classes



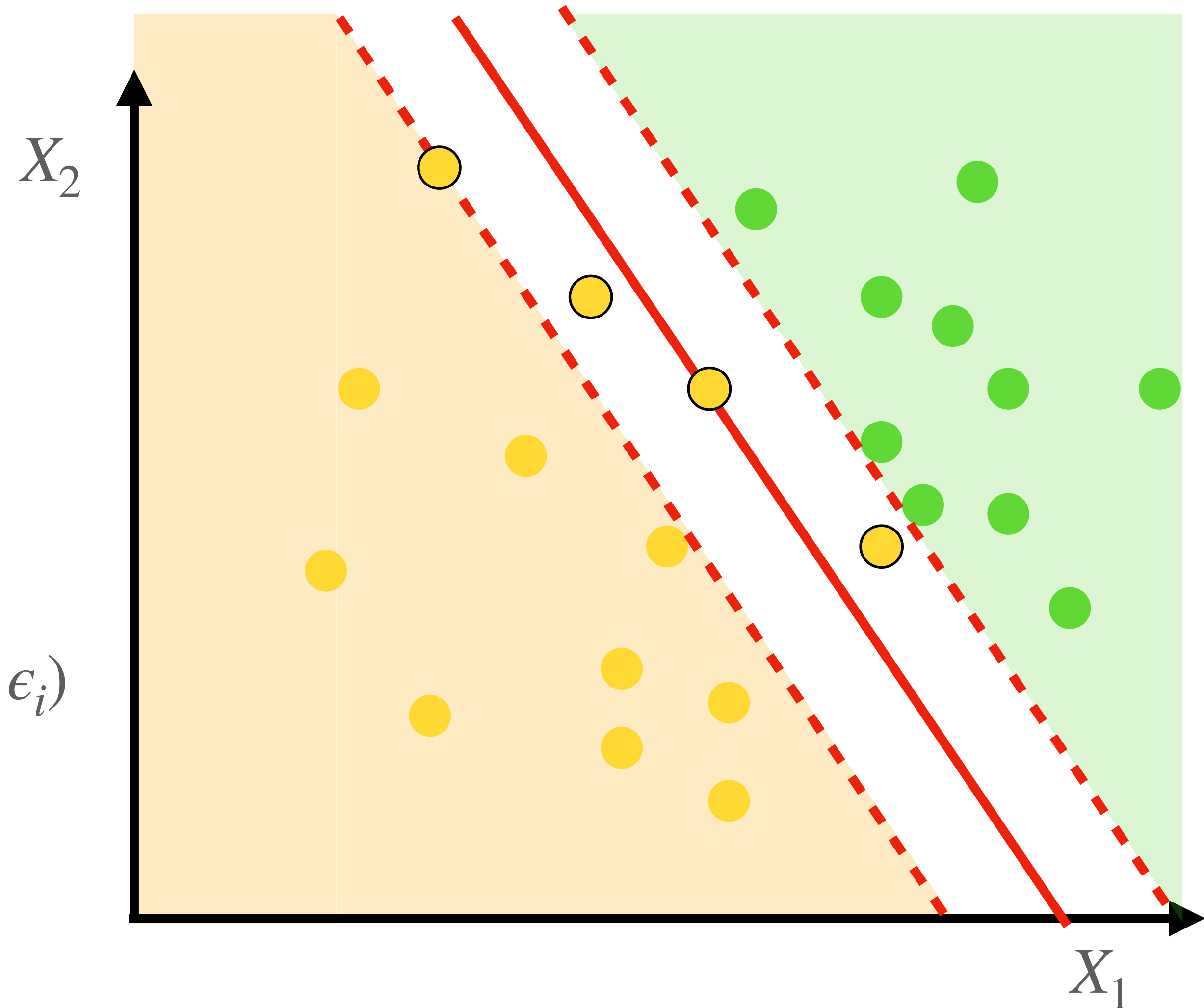
Adding a third dimension to the hyperplane allows a linear plane to separate the two classes entirely in the new dimension

Support Vector Classifier

Predictors $X = [X_1, X_2]$

Hyperplane $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$

Constraint $y_i (\beta_0 + \beta_1 X_1 + \beta_2 X_2) \geq M(1 - \epsilon_i)$



Support Vector Machine

Predictors $X = [X_1, X_2, X_1^2, X_2^2]$

Hyperplane $\beta_0 + \beta_{11}X_1 + \beta_{21}X_2 + \beta_{12}X_1^2 + \beta_{22}X_2^2 = 0$

Constraint $y_i (\beta_0 + \beta_{11}X_1 + \beta_{21}X_2 + \beta_{12}X_1^2 + \beta_{22}X_2^2) \geq M(1 - \epsilon_i)$

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i)$$

Stop and think...

What are the potential disadvantages of transforming the data?

Predictors $X = [X_1, X_2, X_1^2, X_2^2]$

Hyperplane $\beta_0 + \beta_{11}X_1 + \beta_{21}X_2 + \beta_{12}X_1^2 + \beta_{22}X_2^2 = 0$

Constraint $y_i (\beta_0 + \beta_{11}X_1 + \beta_{21}X_2 + \beta_{12}X_1^2 + \beta_{22}X_2^2) \geq M(1 - \epsilon_i)$

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i)$$

Stop and think...

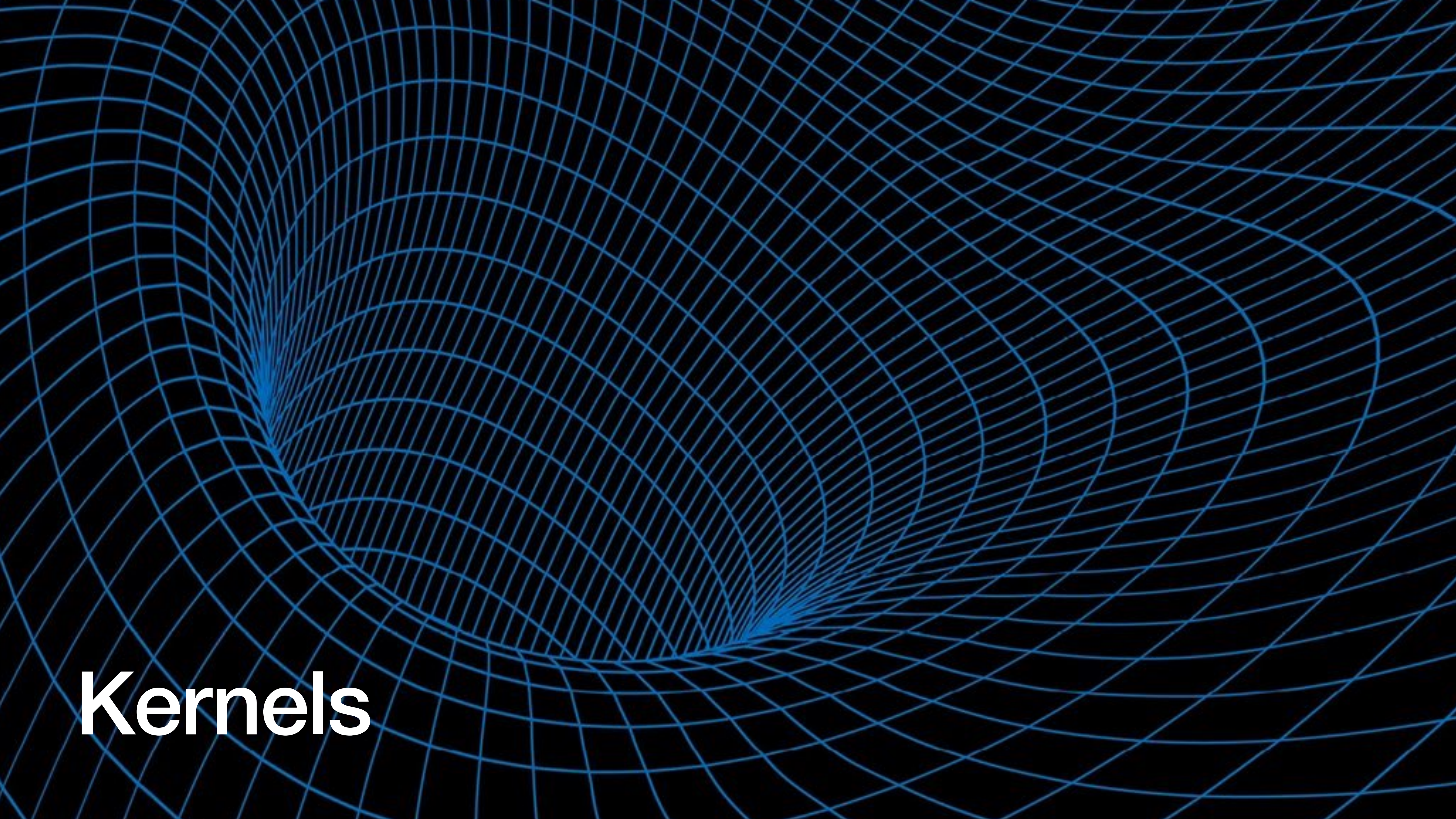
What are potential disadvantages of transforming the data?

→ could be computationally expensive

Stop and think...

What are potential disadvantages of transforming the data?

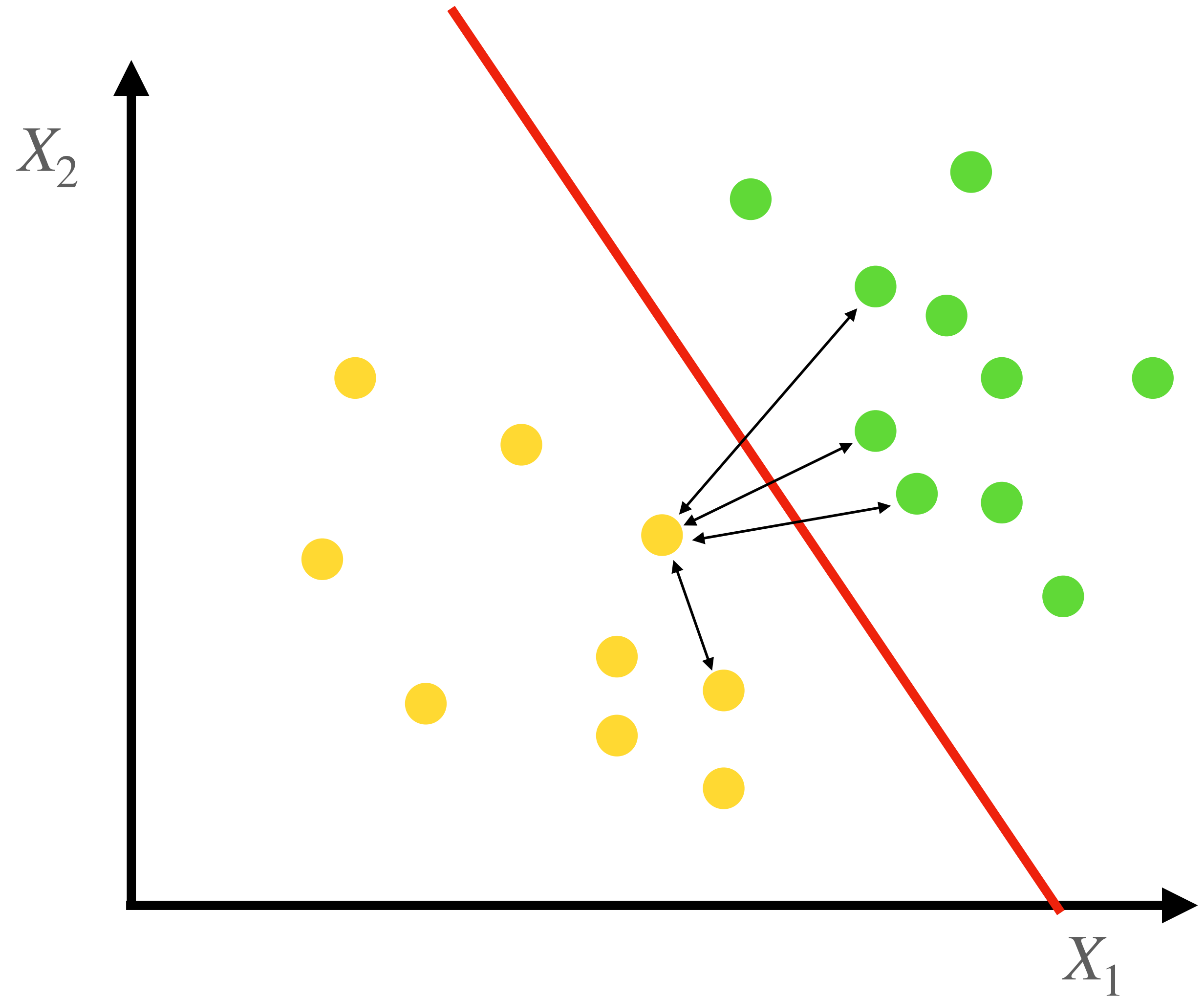
- could be computationally expensive
- not computationally expensive because of kernels!

The background of the slide is a black field filled with a complex pattern of thin, light blue lines. These lines form a series of concentric, overlapping circles and arcs that create a sense of depth and movement, resembling a stylized tunnel or a series of ripples in water. The lines are most densely packed in the center and become more sparse towards the edges.

Kernels

Computation of the Hyperplane

When computing hyperplane, we actually only need to compute relationships between pairs of observations

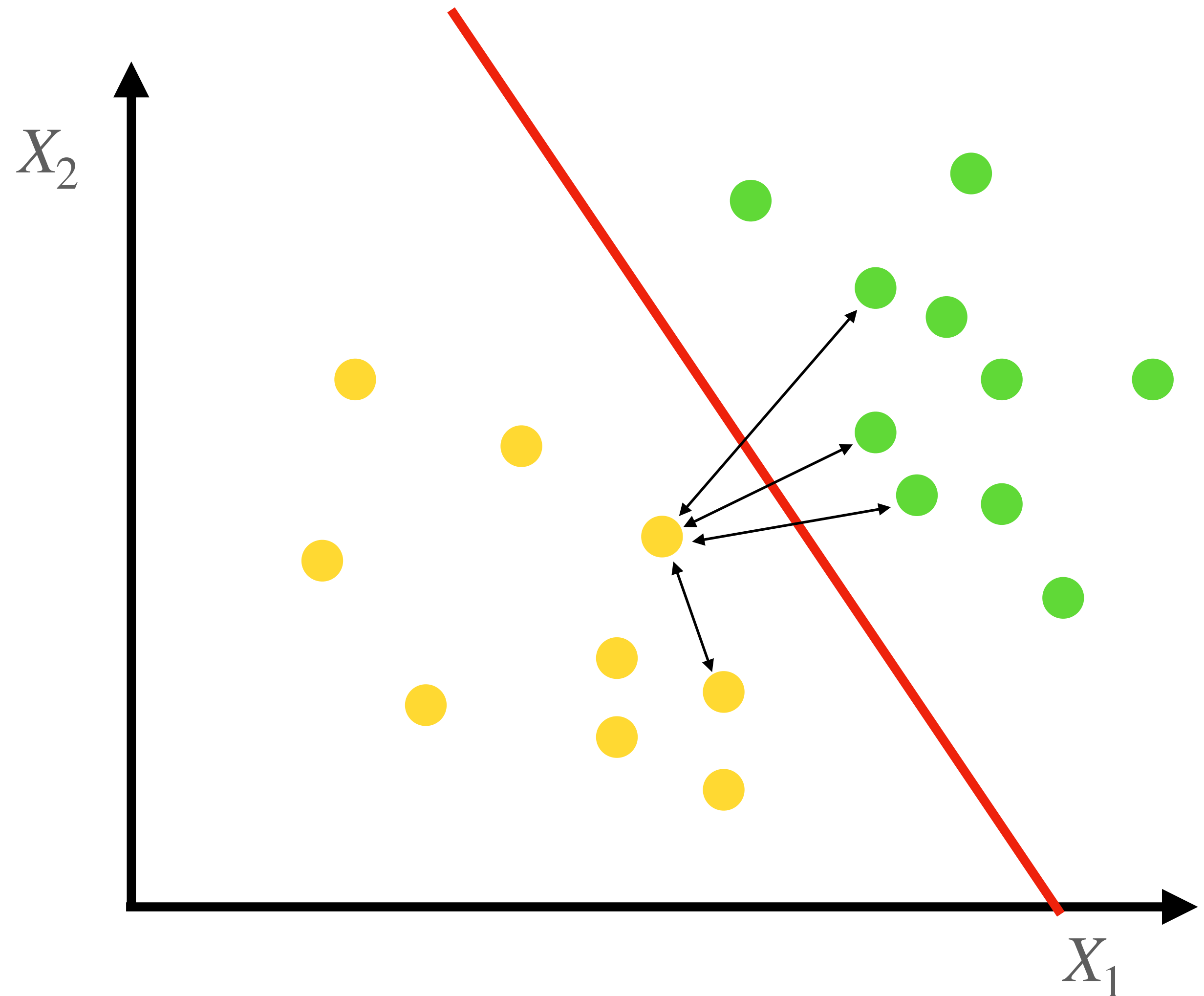


Computation of the Hyperplane

When computing hyperplane,
we actually only need to
compute relationships
between pairs of observations

Inner product

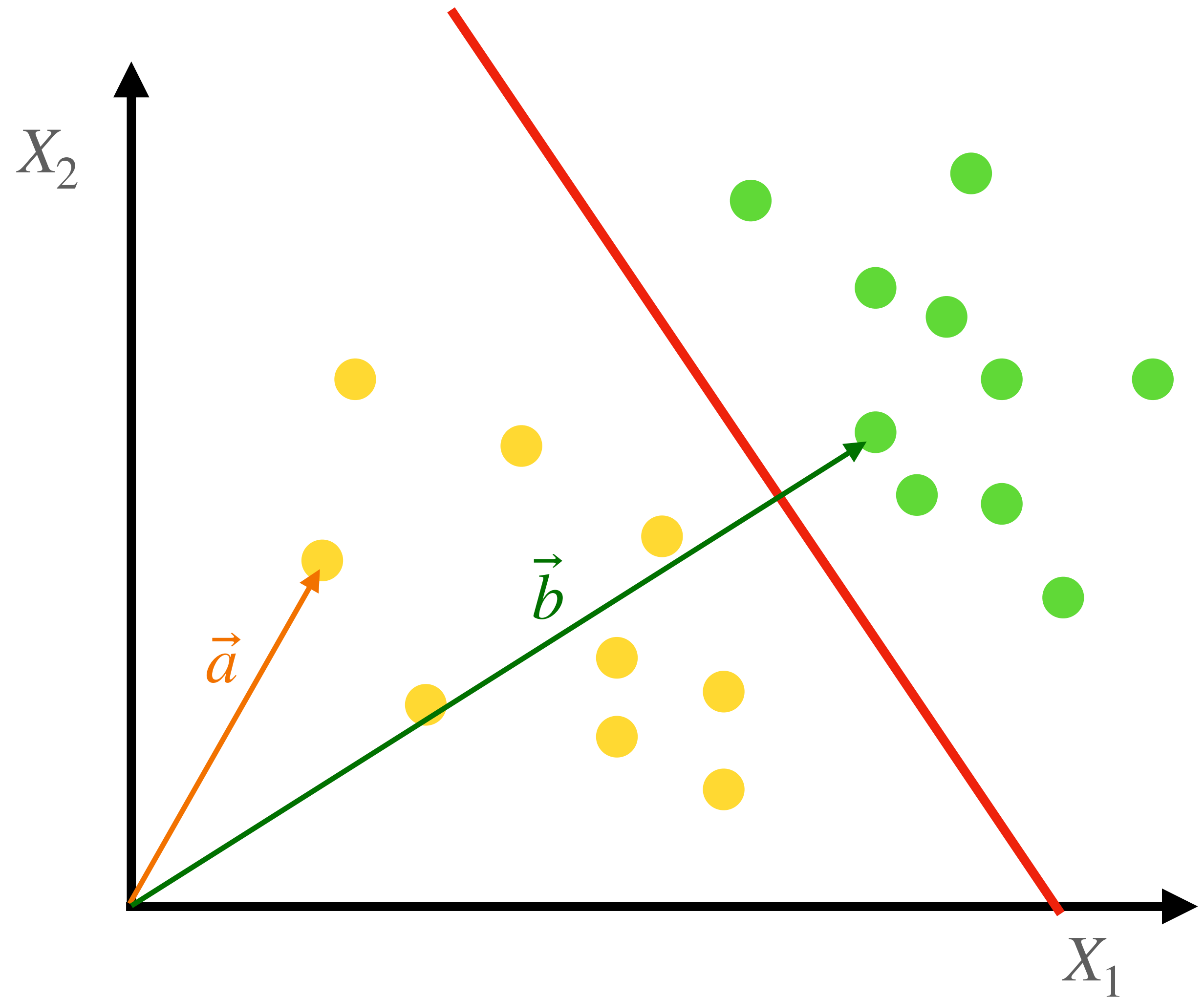
$$\langle a, b \rangle = a_1b_1 + a_2b_2 + \dots + a_rb_r$$



Computation of the Hyperplane

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

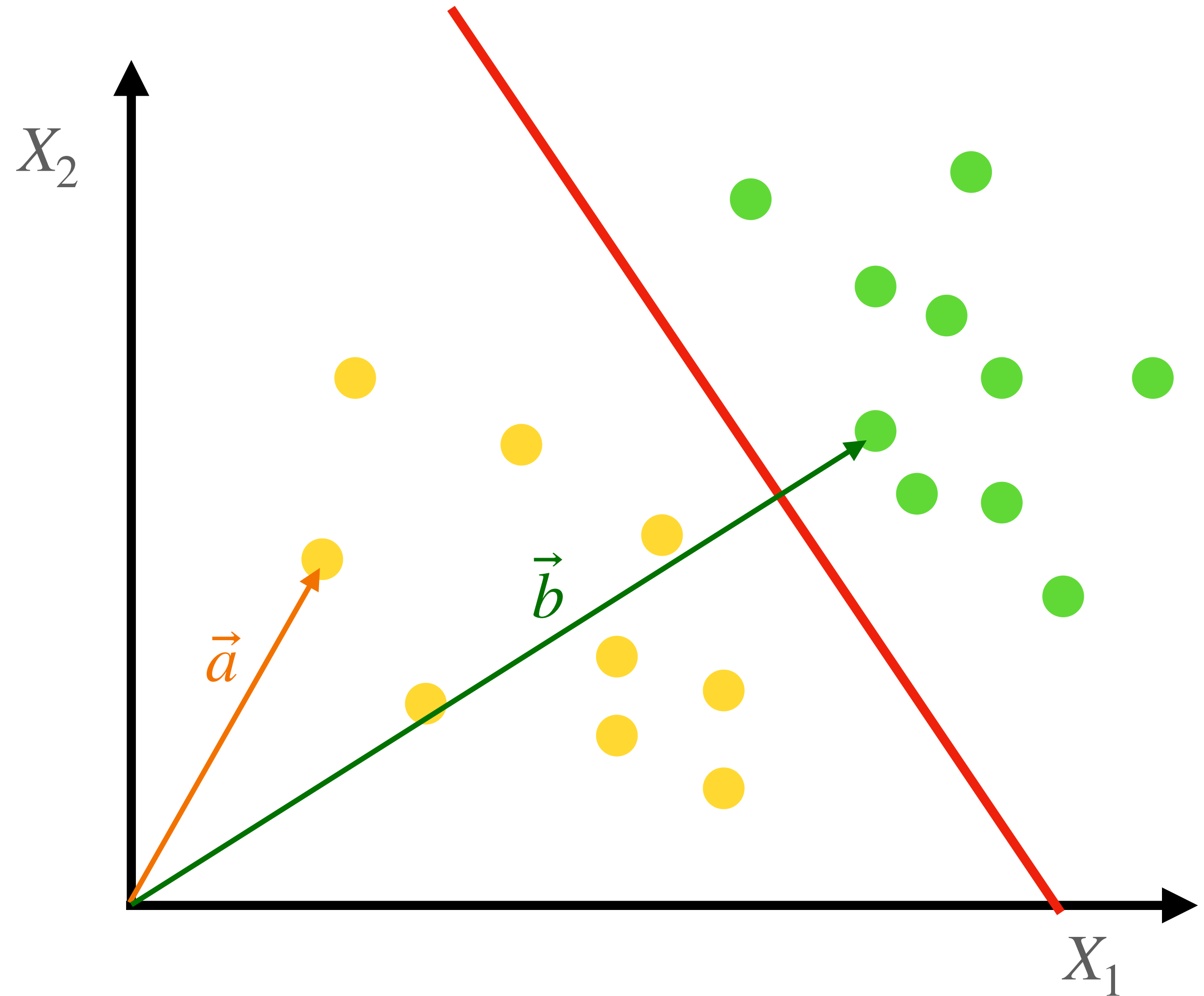
$$\vec{b} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$



Computation of the Hyperplane

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\langle a, b \rangle = 1(4) + 2(3) = 10$$



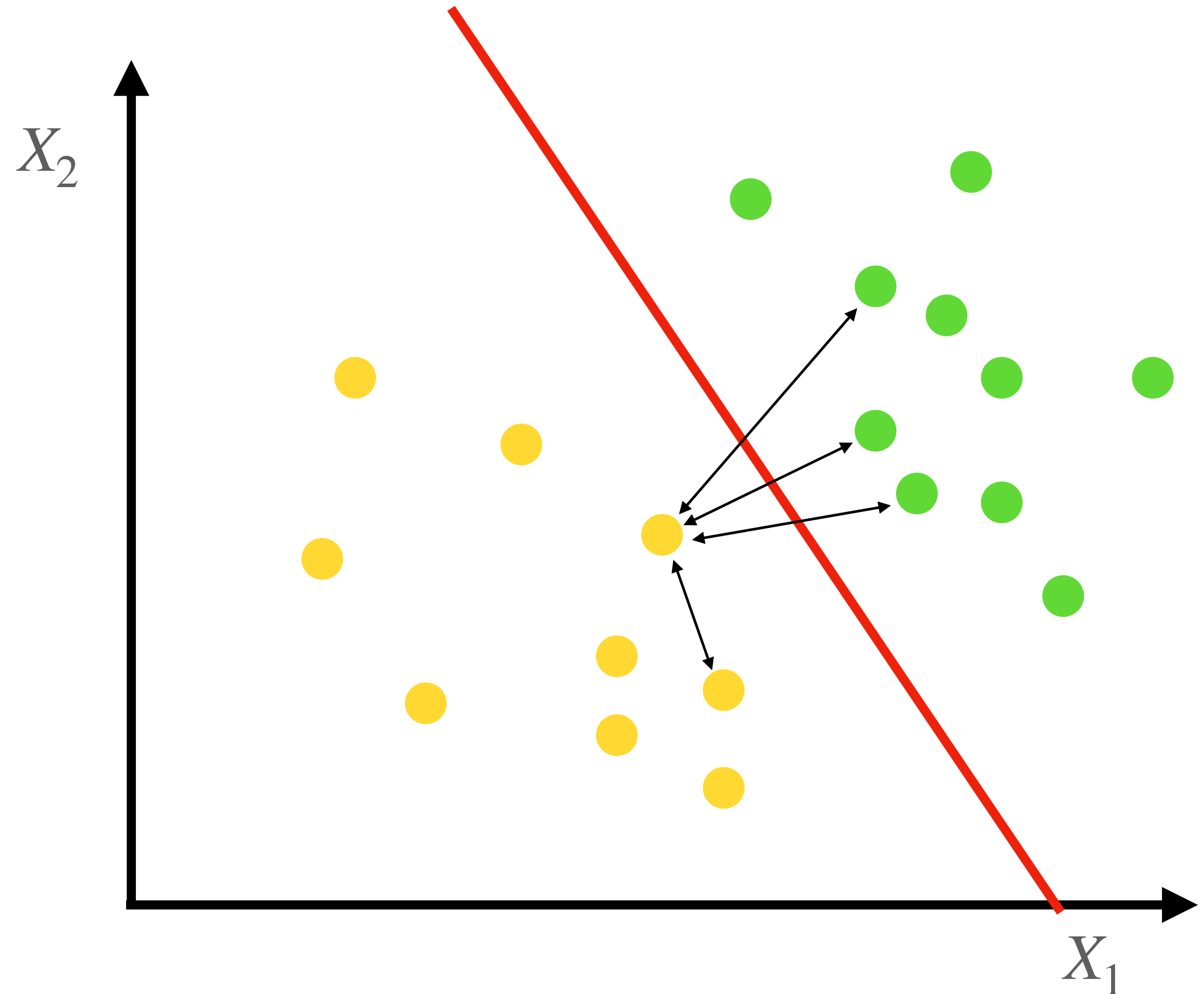
Computation of the Hyperplane

Hyperplane

$$f(x) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$$

can be written as

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$$



Computation of the Hyperplane

Hyperplane

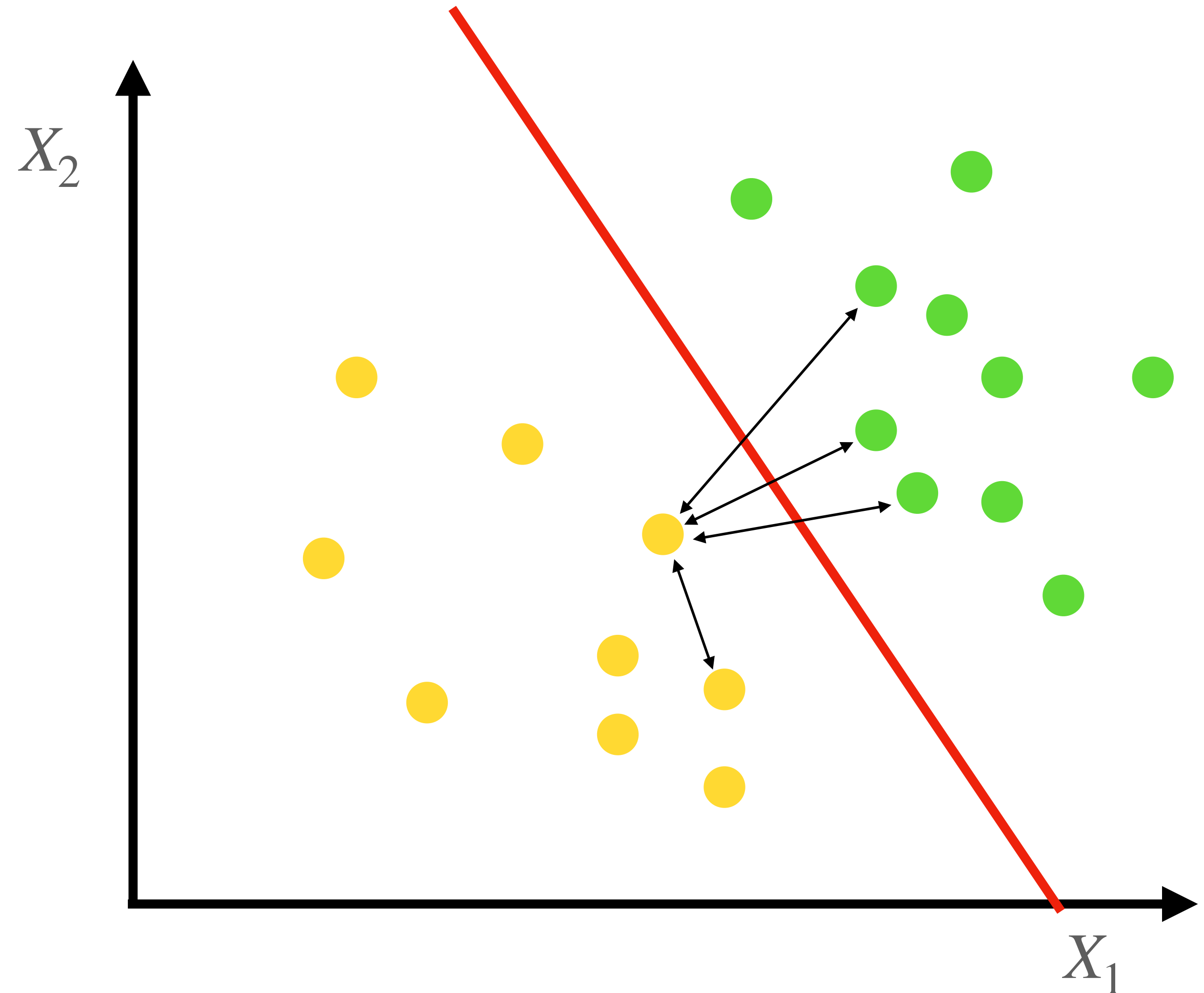
$$f(x) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$$

can be written as

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$$

Which becomes

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

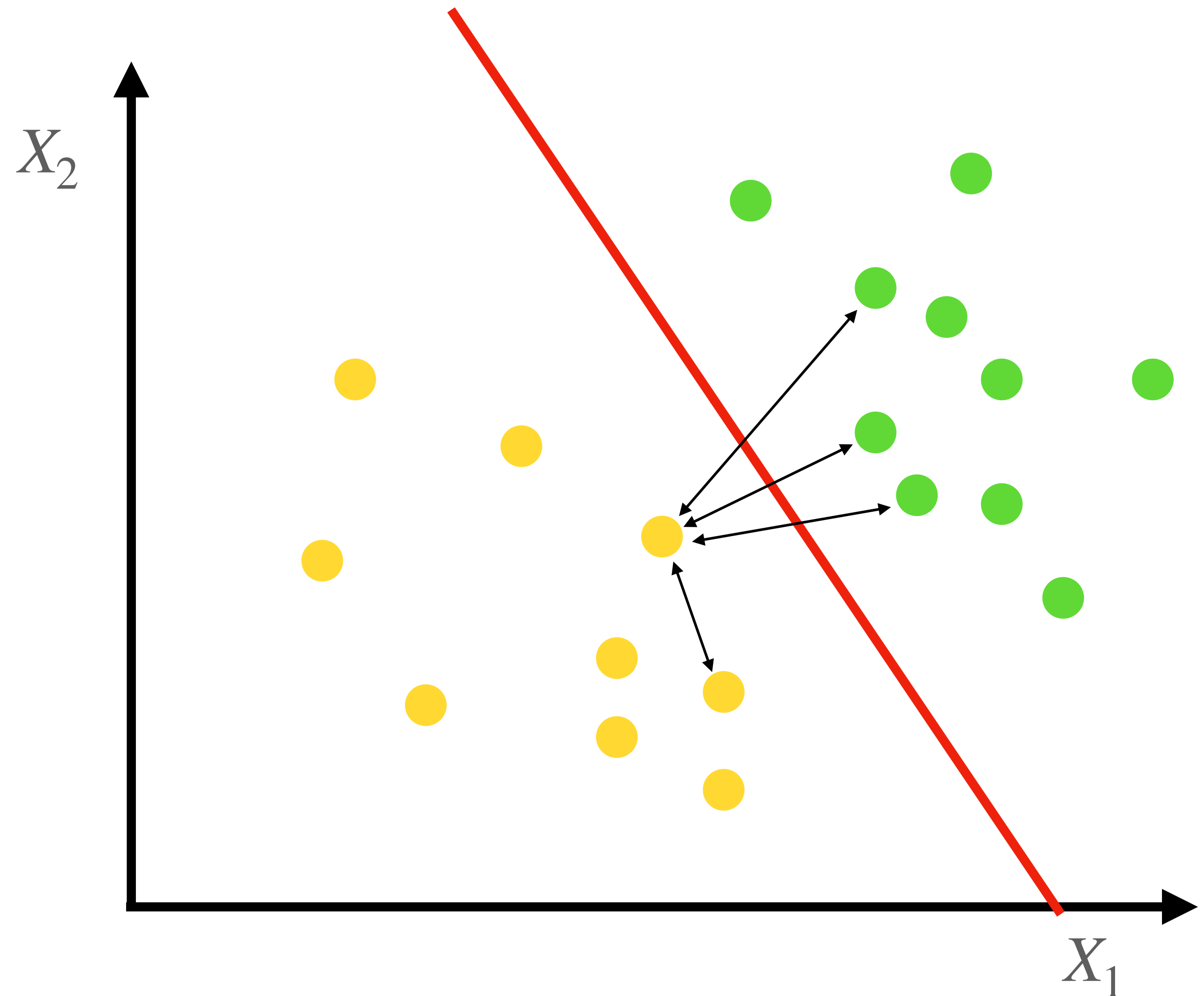


Kernels

Are used to compute relationships between observations

Compute higher-dimensional relationship without actually enlarging feature space (fast)!

Are very popular and used outside of SVMs

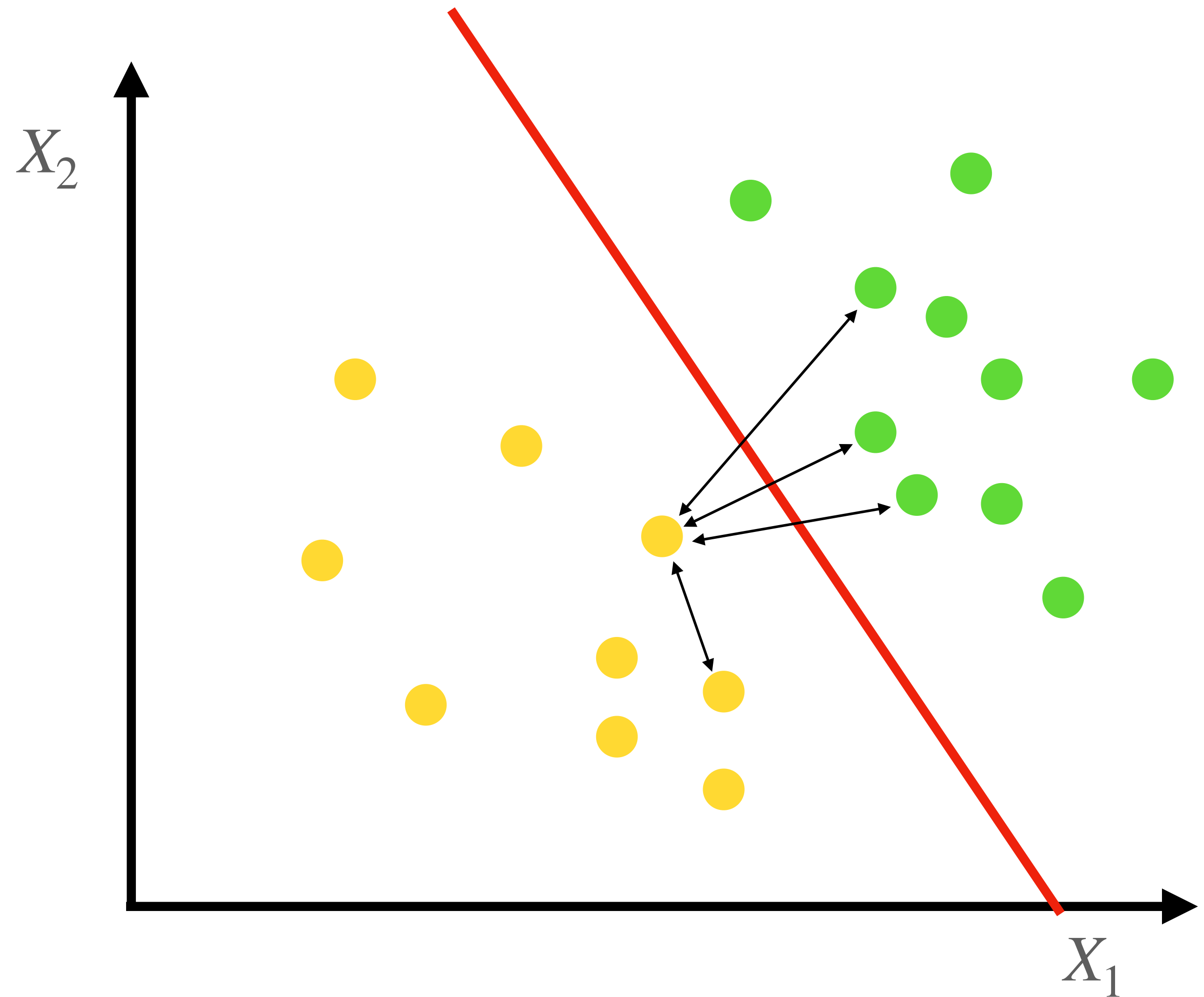


Kernels

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

Generalizes to

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \mathbf{K}(x, x_i)$$



Kernel Options

Linear Kernel

$$K(x_i, x_{i'}) = \langle x_i, x_{i'} \rangle$$

Linear relationships

Equivalent to support vector
classifier

Kernel Options

Polynomial Kernel

Polynomial relationships between points

Allows curved boundaries

$$K(x_i, x_{i'}) = \left(r + \langle x_i, x_{i'} \rangle \right)^d$$

Kernel Options

Polynomial Kernel

$$K(x_i, x_{i'}) = \left(r + \langle x_i, x_{i'} \rangle \right)^d$$

Polynomial relationships between points

Allows curved boundaries

Higher degree d = more inflections

$r = 1$ is a good choice

Cross-validate for r and d

Kernel Options

Polynomial Kernel

$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^{\textcolor{red}{d}}$$

Compute higher-dimensional relationship without actually enlarging feature space (fast)!

Kernel Options

Polynomial Kernel

$$r = 1/2, \quad d = 2$$

$$K(a, b) = \left(\frac{1}{2} + \langle a, b \rangle \right)^2$$

$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^{\textcolor{red}{d}}$$

Compute higher-dimensional relationship without actually enlarging feature space (fast)!

Kernel Options

Polynomial Kernel

$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^d$$

$$\begin{aligned} K(a, b) &= \left(\frac{1}{2} + \langle a, b \rangle \right)^2 \\ &= \left(\frac{1}{2} + ab \right) \left(\frac{1}{2} + ab \right) \end{aligned}$$

Kernel Options

Polynomial Kernel

$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^d$$

$$\begin{aligned} K(a, b) &= \left(\frac{1}{2} + \langle a, b \rangle \right)^2 \\ &= \left(\frac{1}{2} + ab \right) \left(\frac{1}{2} + ab \right) \\ &= ab + a^2b^2 + \frac{1}{4} \end{aligned}$$

Kernel Options

Polynomial Kernel

$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^d$$

$$\begin{aligned} K(a, b) &= \left(\frac{1}{2} + \langle a, b \rangle \right)^2 \\ &= \left(\frac{1}{2} + ab \right) \left(\frac{1}{2} + ab \right) \\ &= ab + a^2b^2 + \frac{1}{4} \\ &= \langle (a, a^2, 1/2), (b, b^2, 1/2) \rangle \end{aligned}$$

Kernel Options

Polynomial Kernel

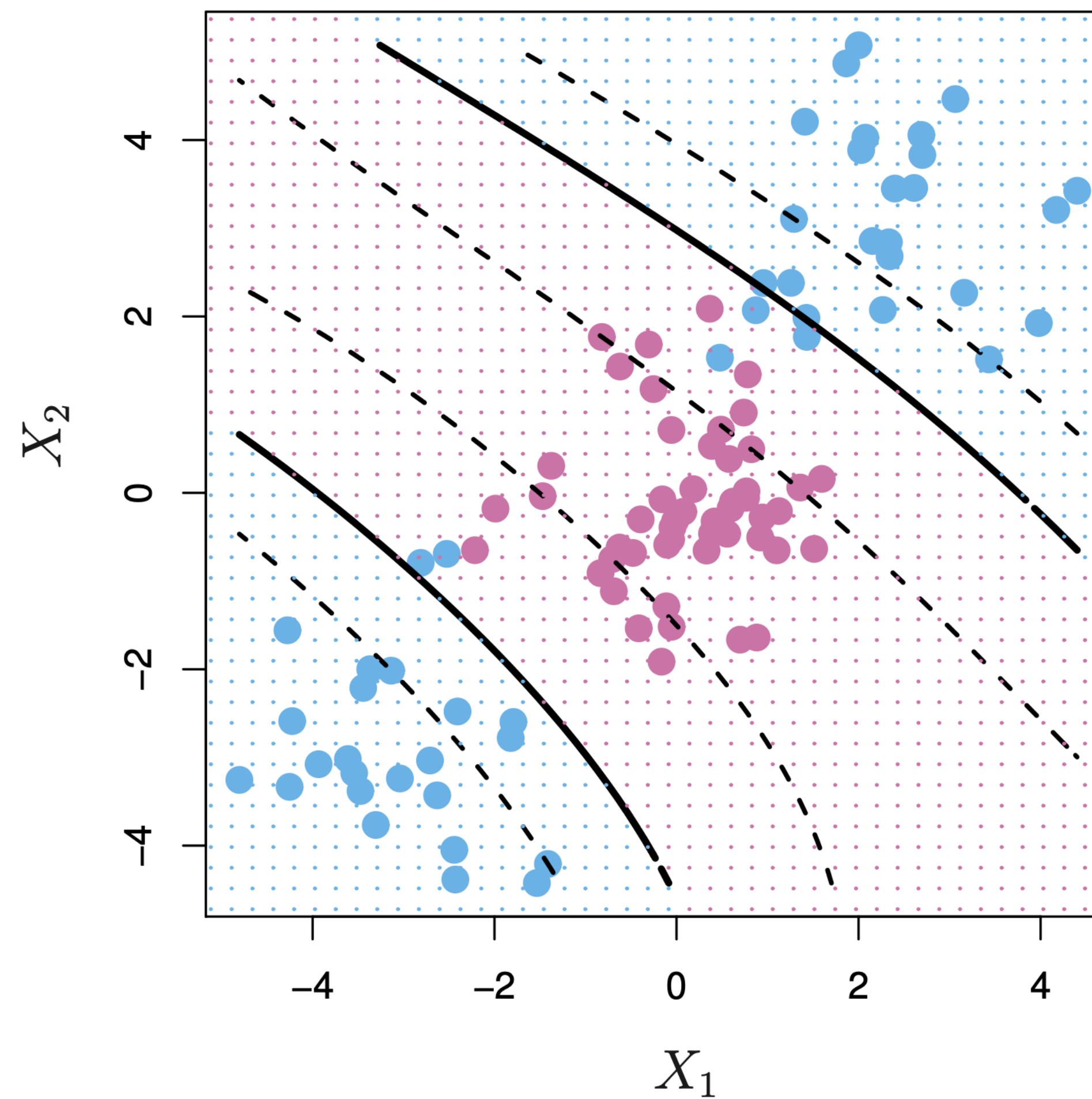
$$K(x_i, x_{i'}) = \left(\textcolor{red}{r} + \langle x_i, x_{i'} \rangle \right)^{\textcolor{red}{d}}$$

Compute higher-dimensional relationship without actually enlarging feature space (fast)!

$$\begin{aligned} K(a, b) &= \left(\frac{1}{2} + \langle a, b \rangle \right)^2 \\ &= \left(\frac{1}{2} + ab \right) \left(\frac{1}{2} + ab \right) \\ &= ab + a^2b^2 + \frac{1}{4} \\ &= \langle (a, a^2, 1/2), (b, b^2, 1/2) \rangle \end{aligned}$$

Kernel Options

Polynomial Kernel



Kernel Options

Radial Kernel

Weights the Euclidean distance between points

Far away points matter much less, close together points matter a lot

$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$

Allows curvier, even circular, boundaries

Kernel Options

Radial Kernel

Weights the Euclidean distance between points

Far away points matter much less, close together points matter a lot

$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$

Allows curvier, even circular, boundaries

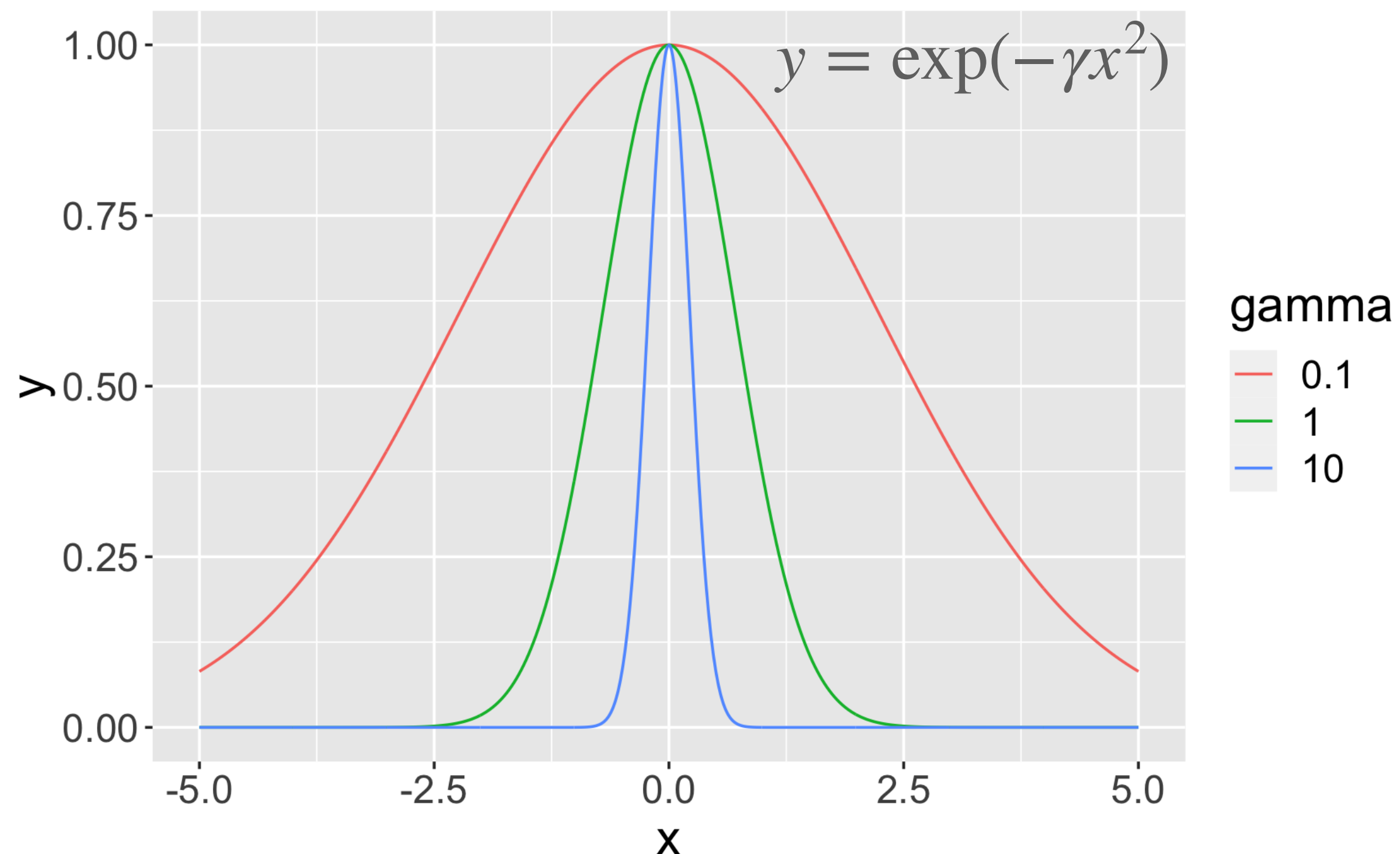
Fun fact: because of its Taylor series expansion, this kernel implicitly computes relationships in *infinite dimensions*!

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \frac{x^\infty}{\infty!}$$

Kernel Options

Radial Kernel

$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$



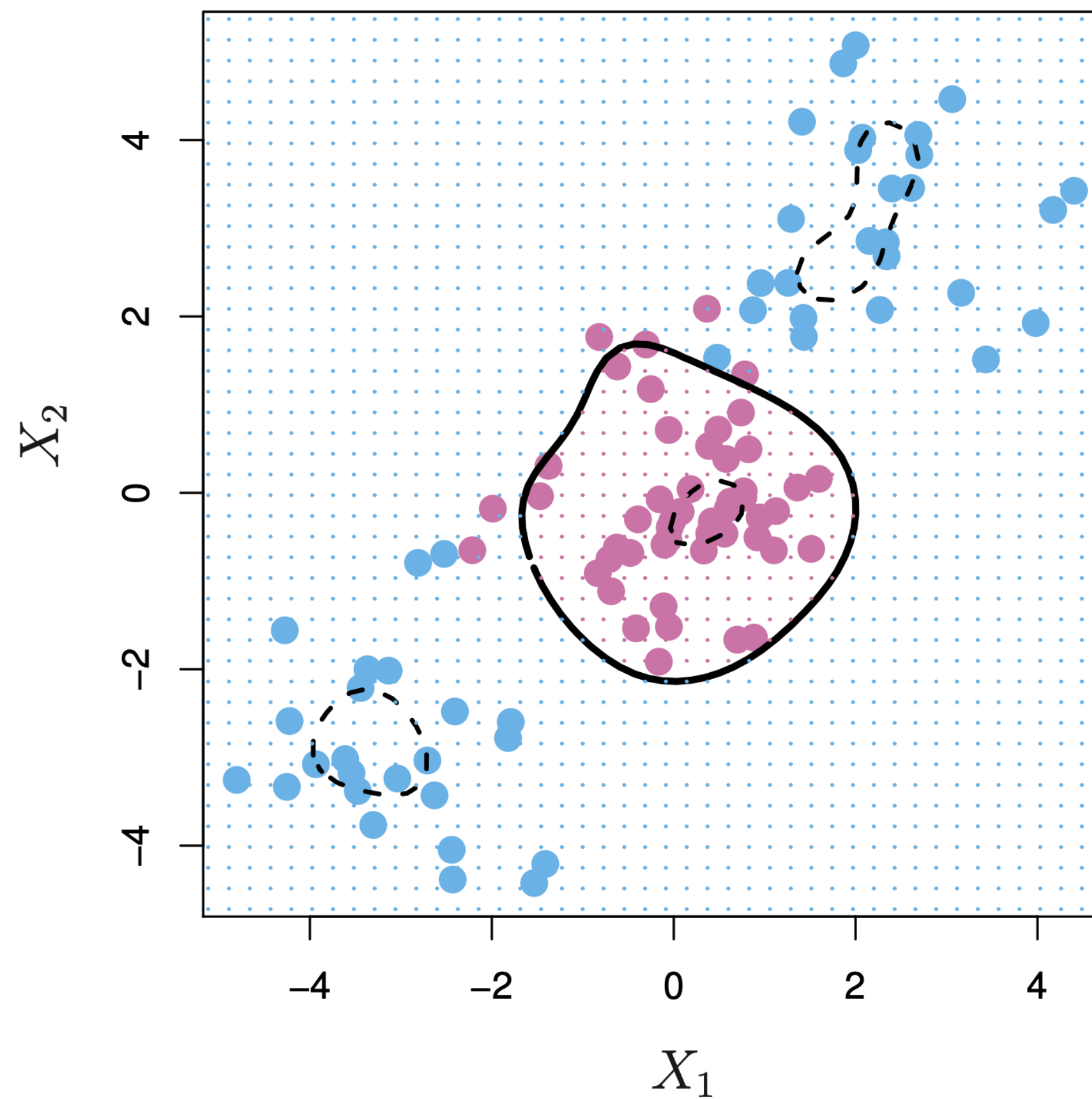
Gamma tuning parameter: positive constant

Large gamma: smaller sphere of influence

Small gamma: larger sphere of influence

Kernel Options

Radial Kernel

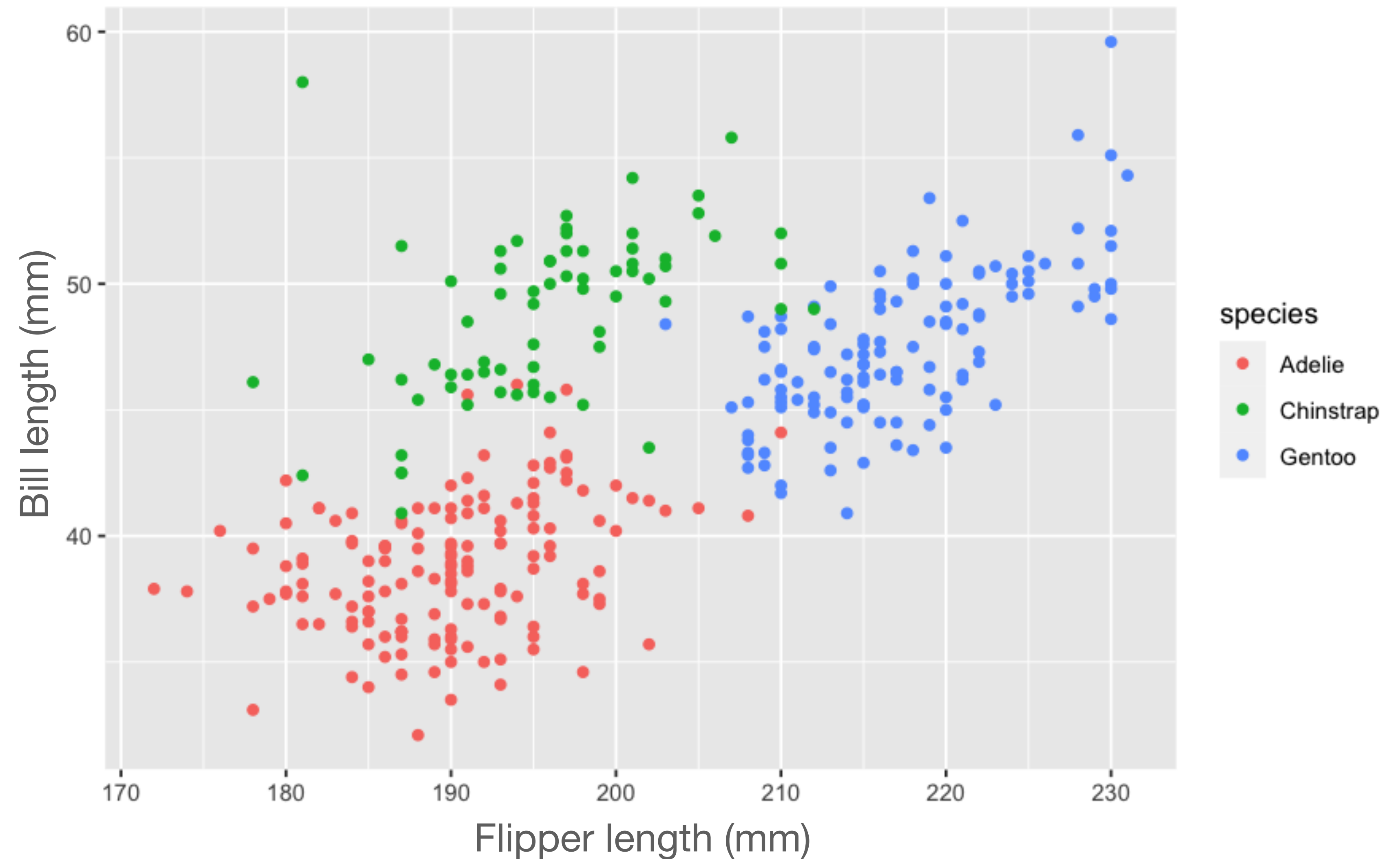


Lab 9.2



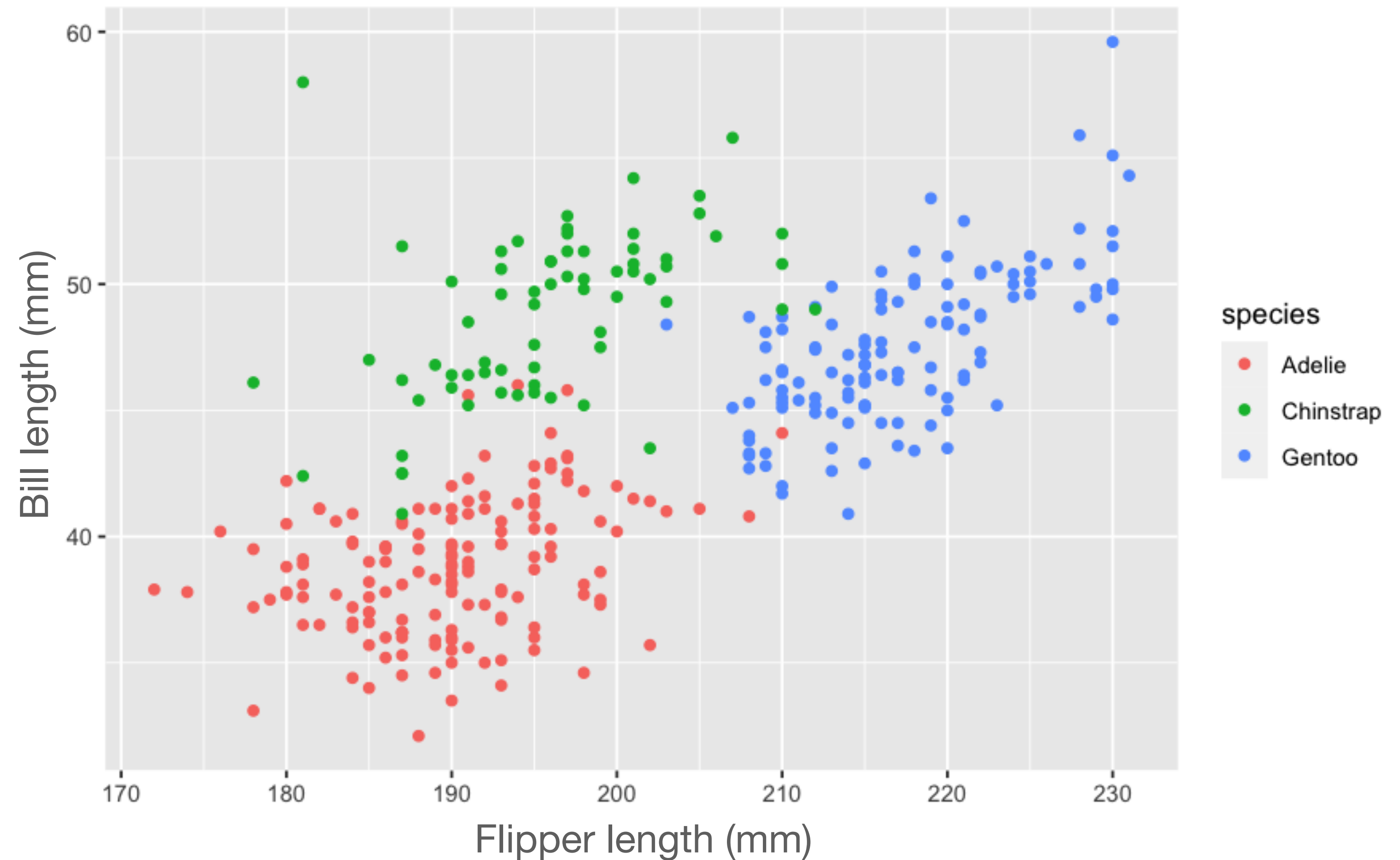
Multiclass SVMs

SVMs with More than Two Classes



Stop and think...

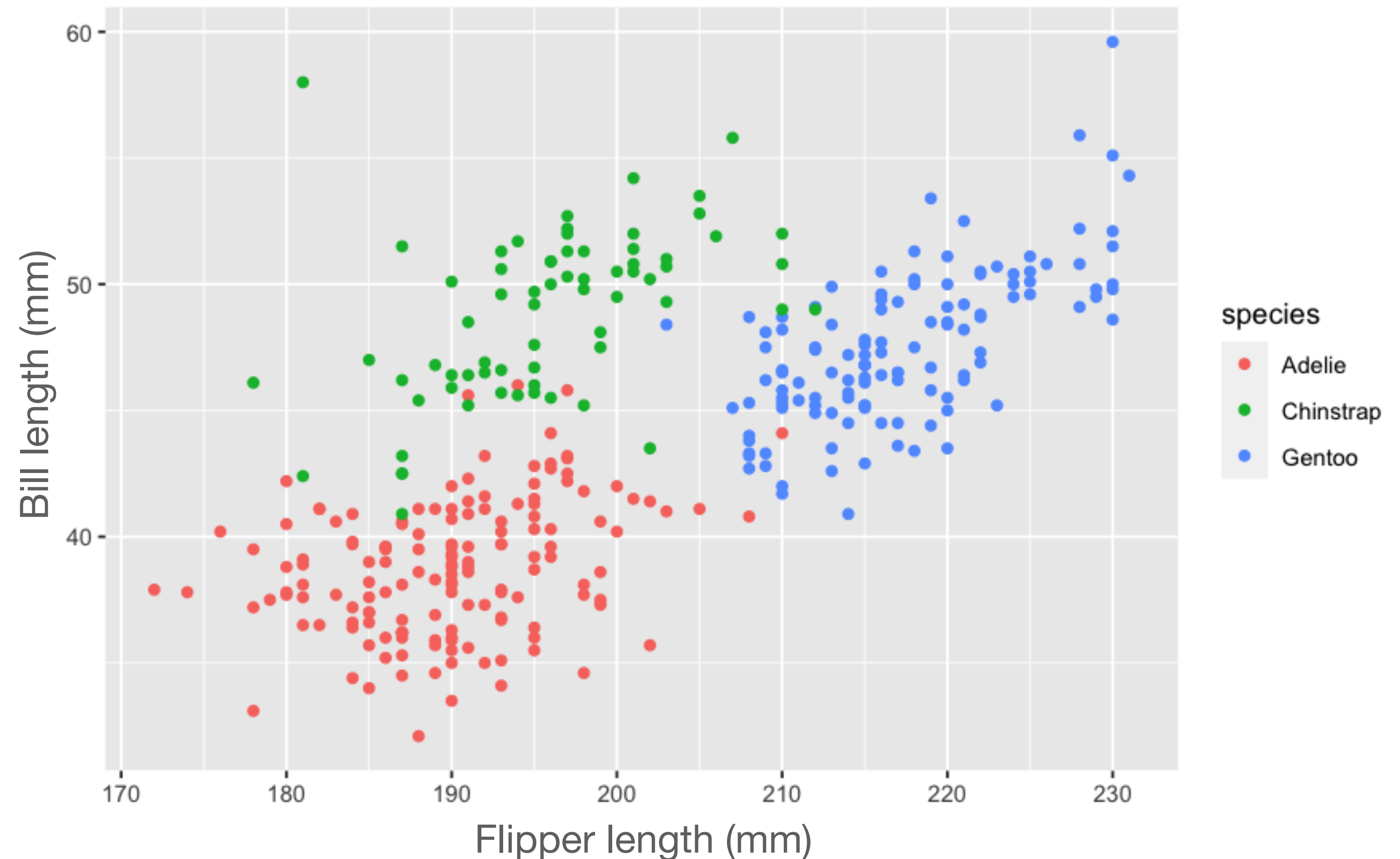
How could you fit a single SVM model to separate these three species?



Stop and think...

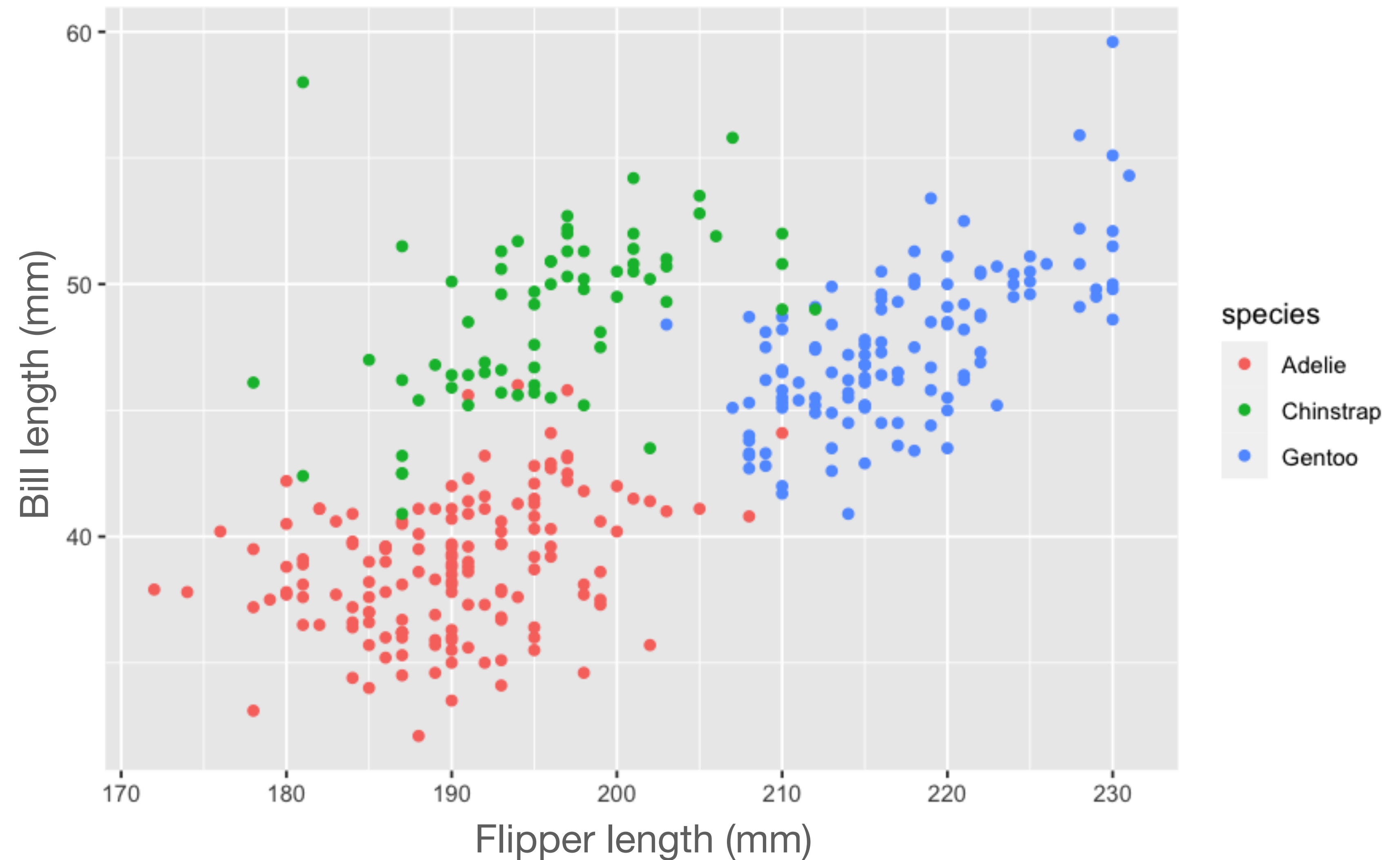
How could you fit a single SVM model to separate these three species?

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$



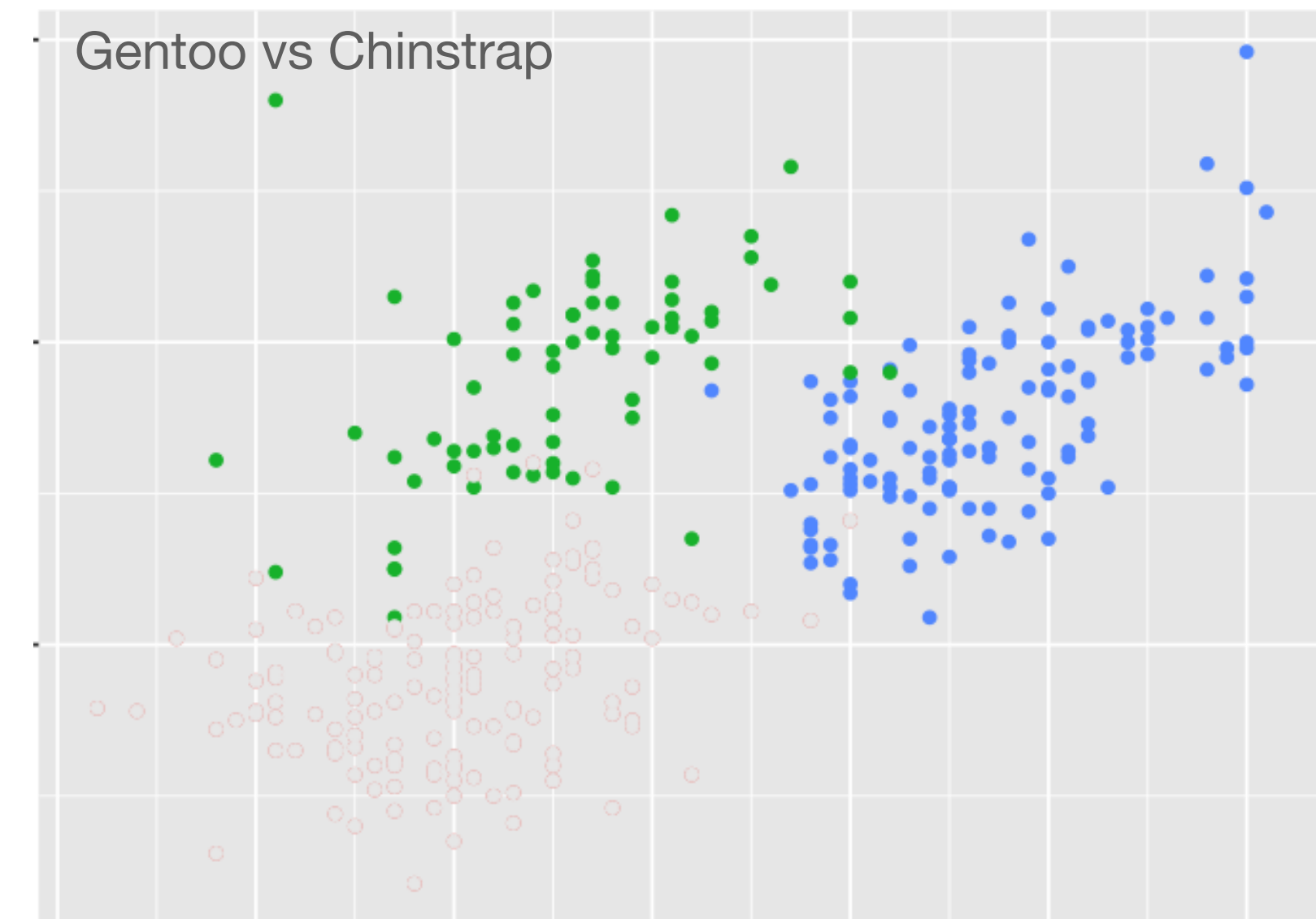
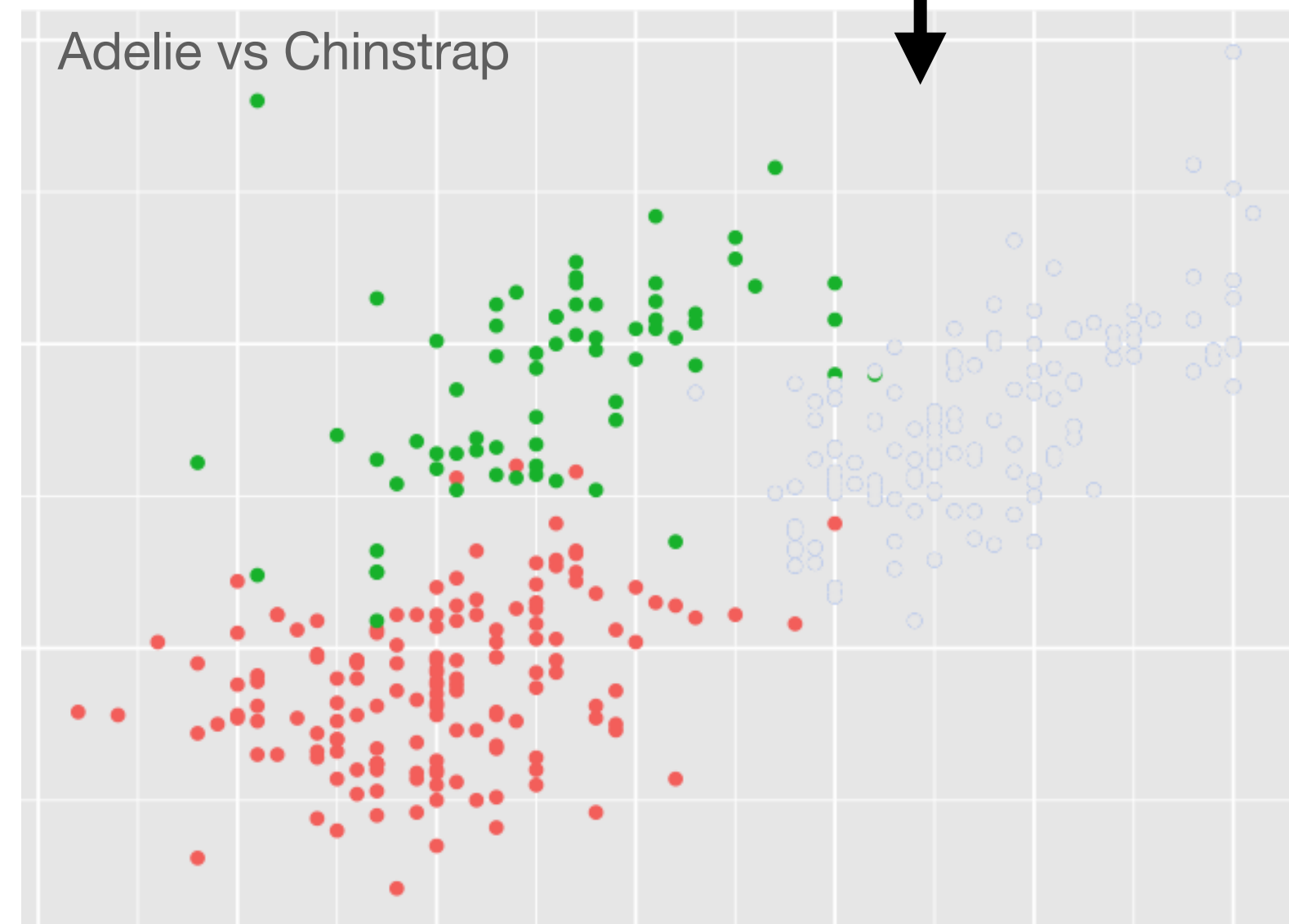
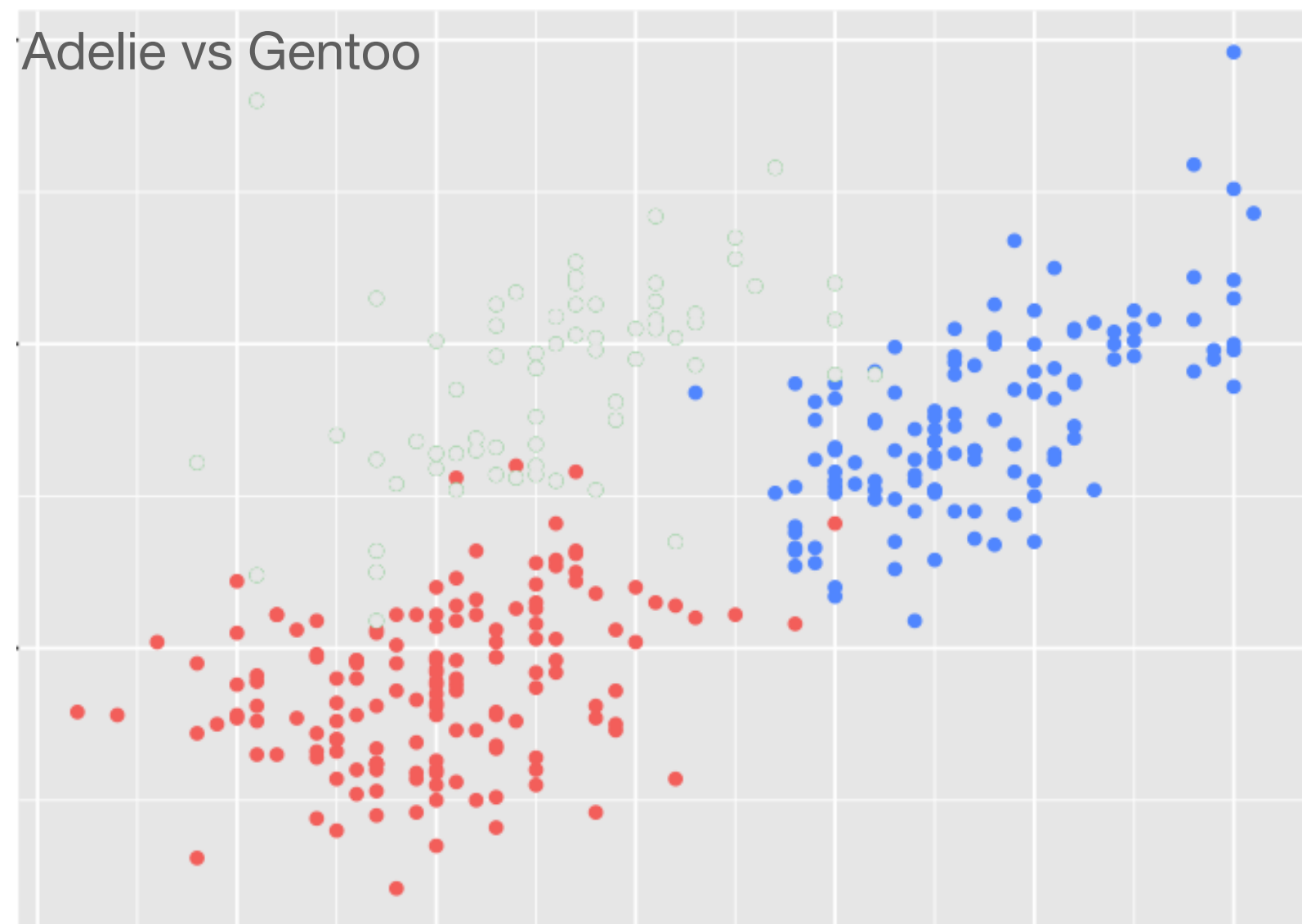
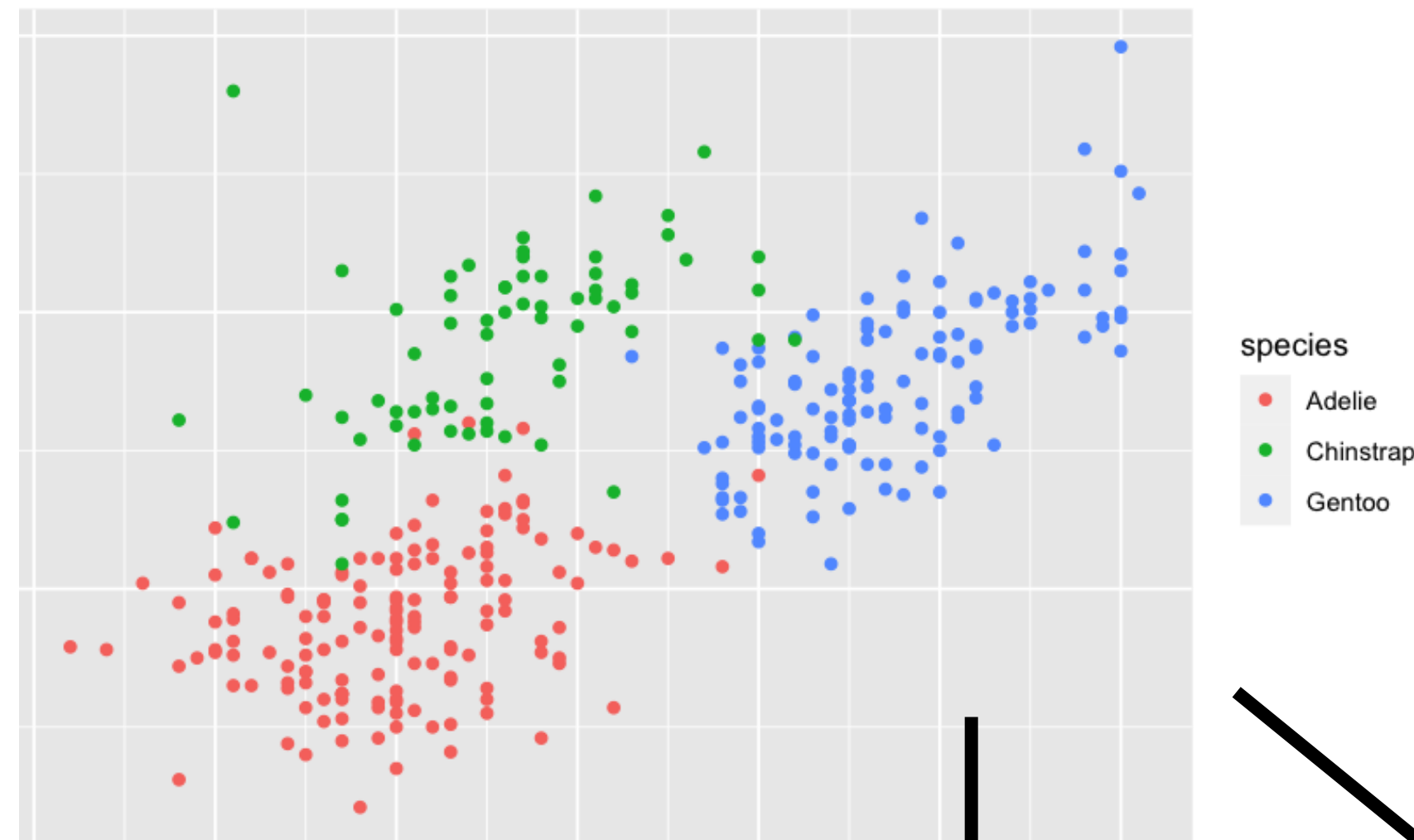
Stop and think...

How could you fit an ensemble of SVM models to separate these three species?



SVMs with More than Two Classes

one-vs-one
or
all-pairs



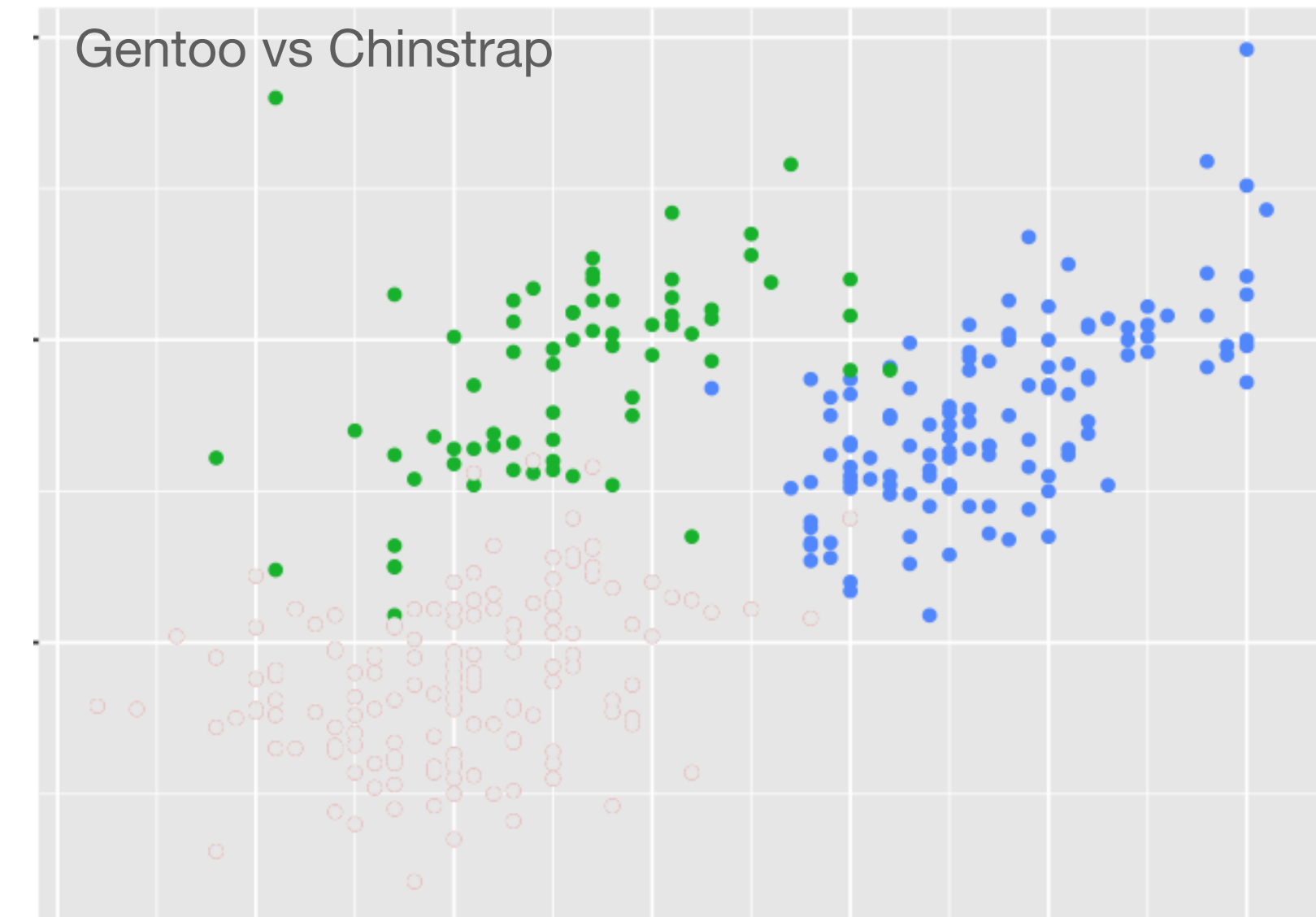
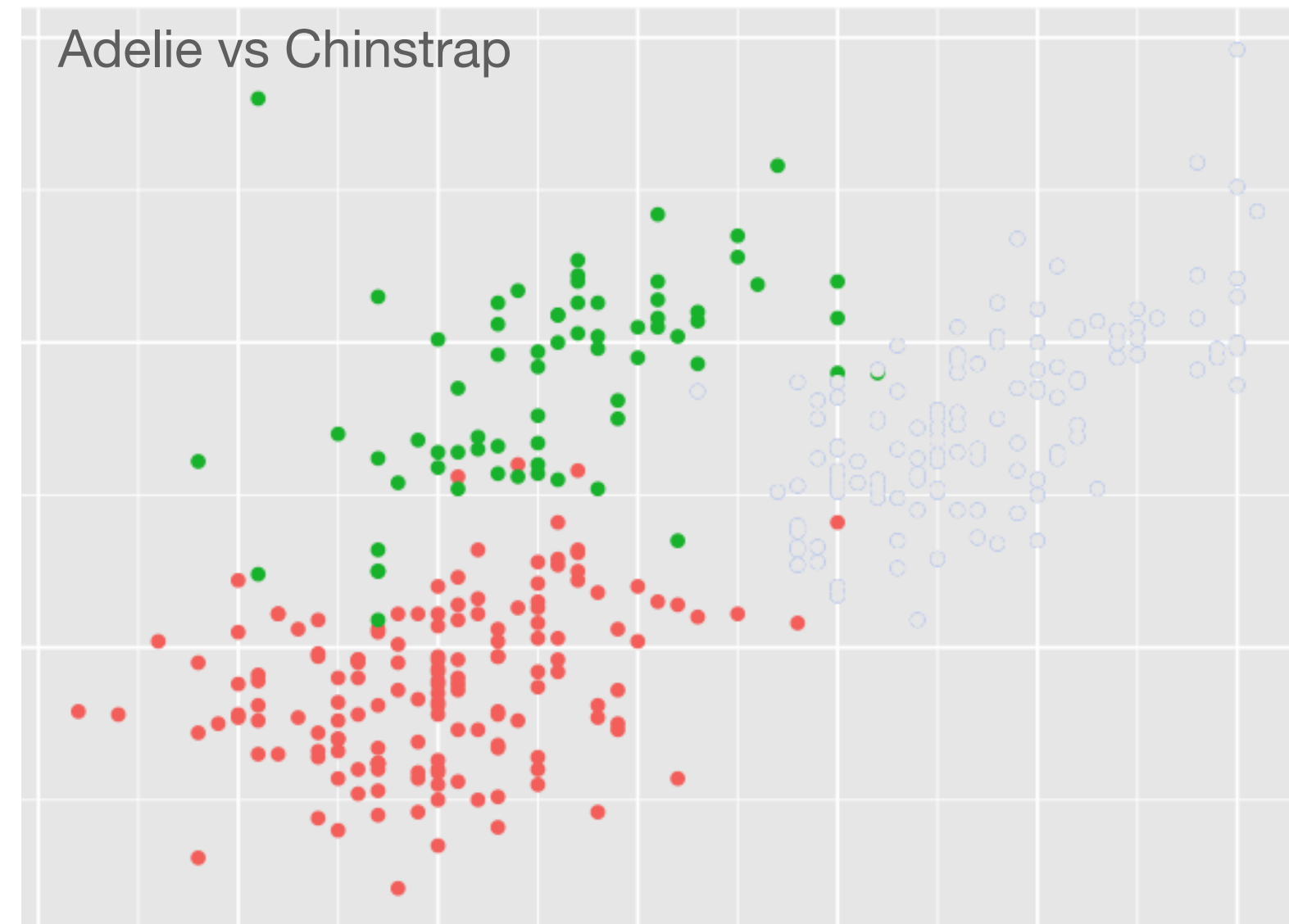
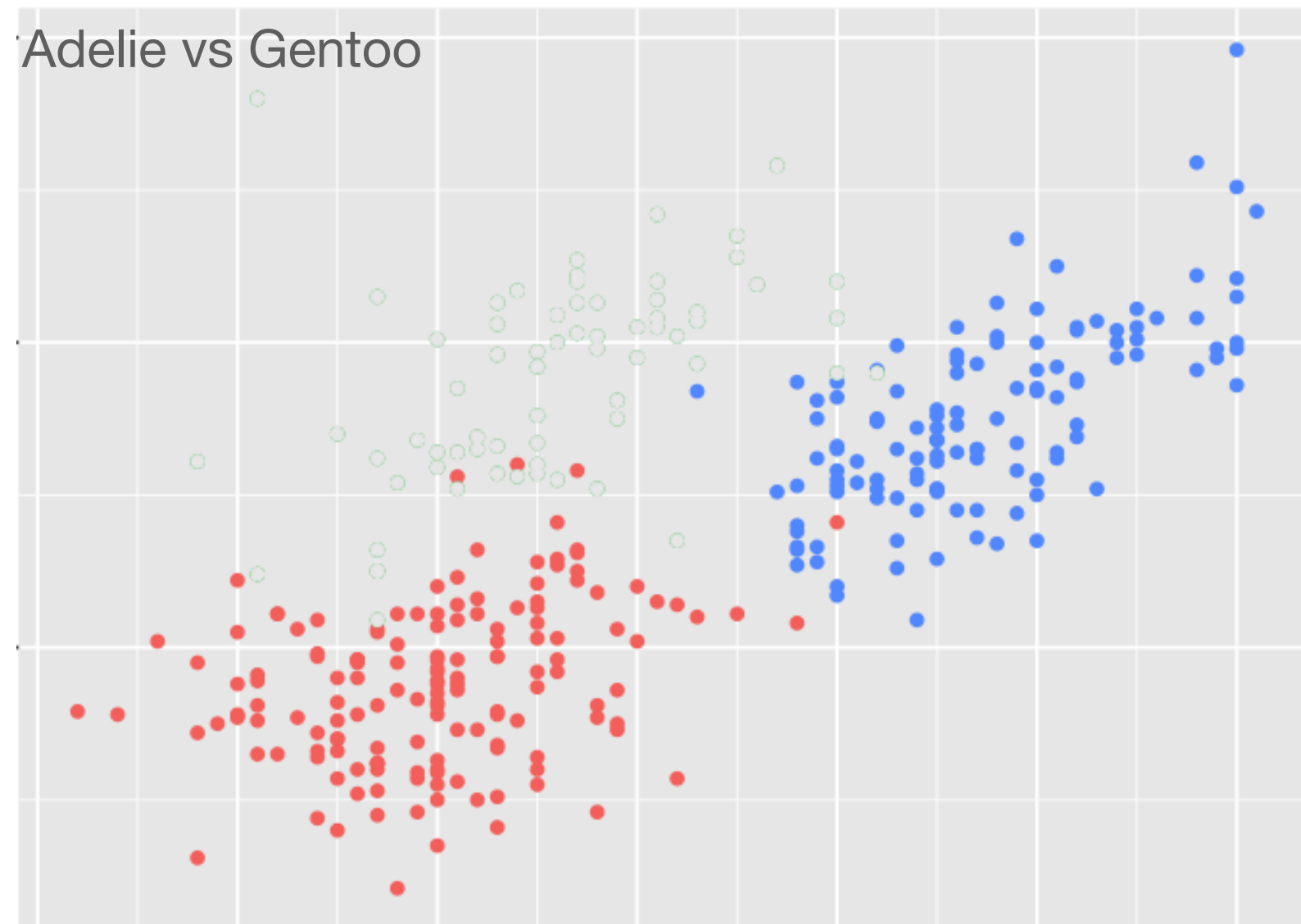
Stop and think...

one-vs-one

or

all-pairs

Given a test observation, how would you classify it based on the three models?



Stop and think...

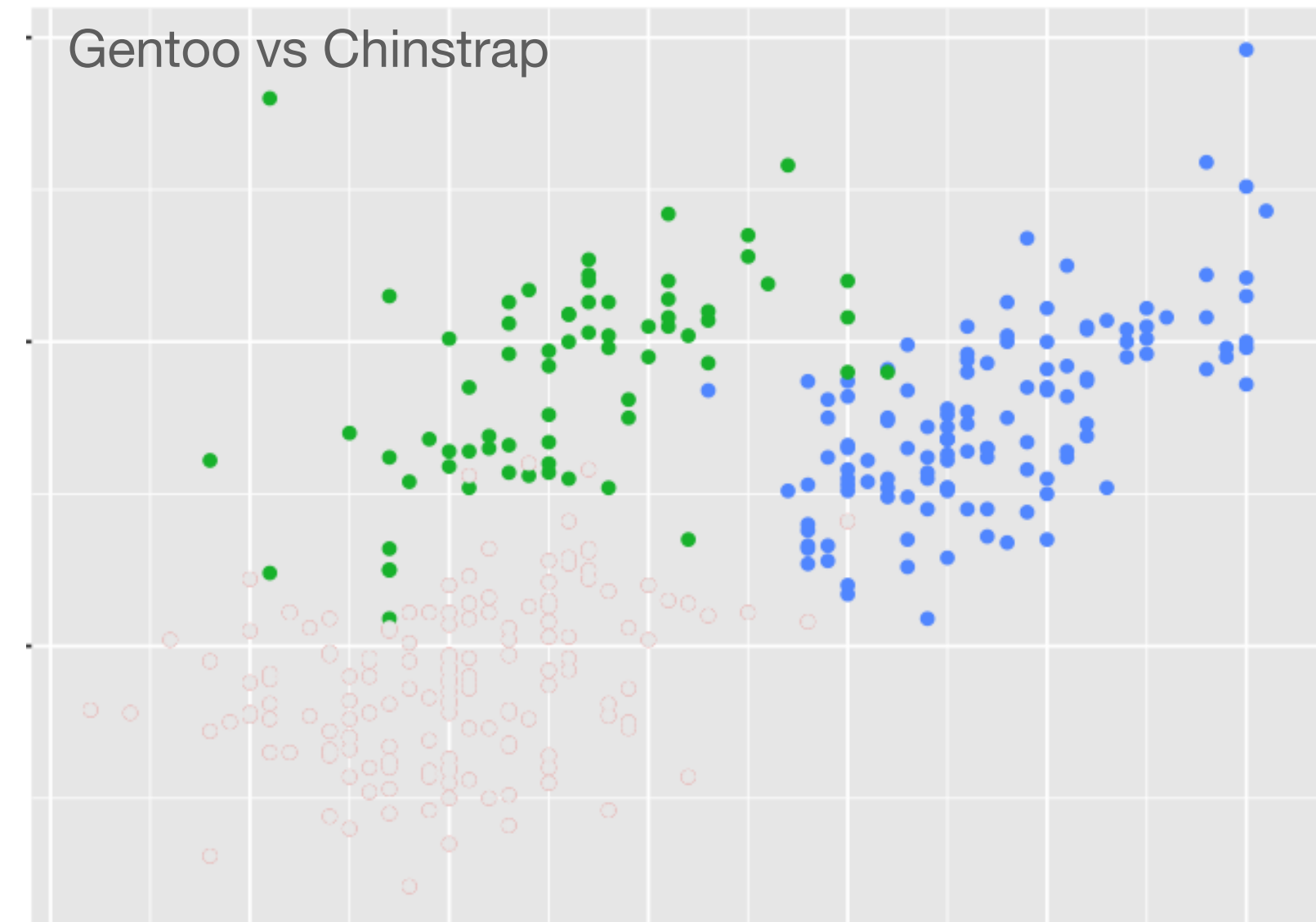
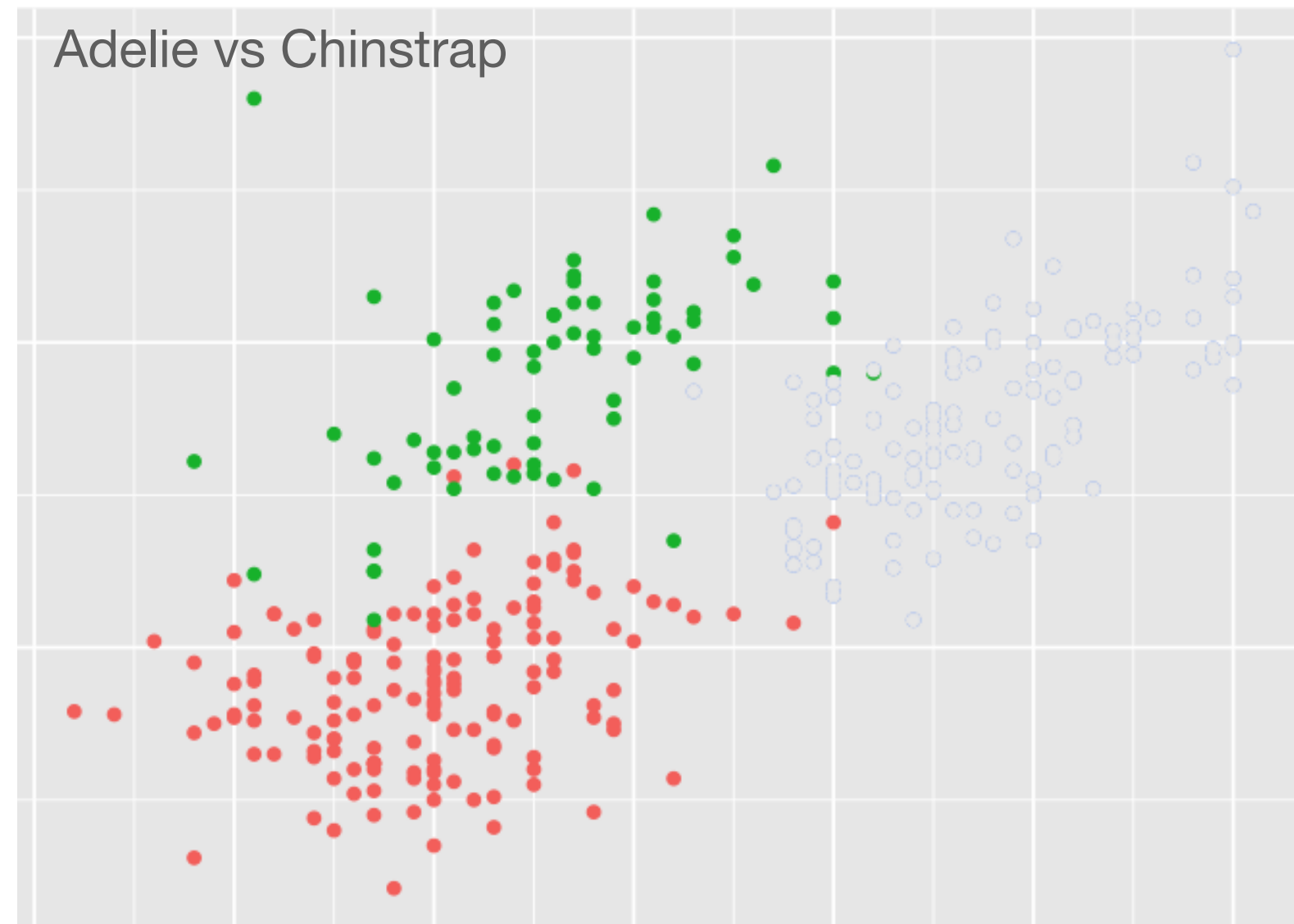
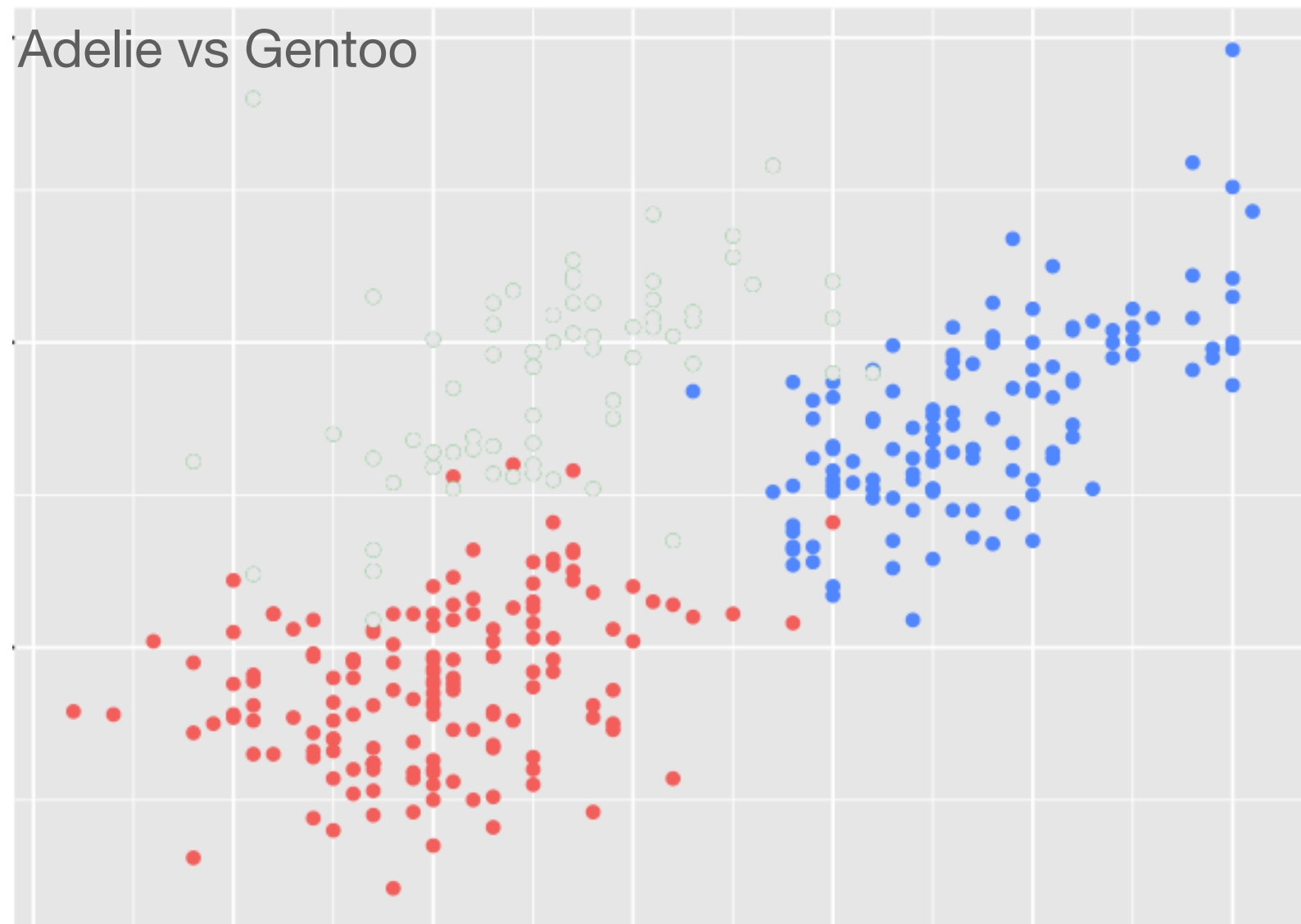
Given a test observation, how would you classify it based on the three models?

one-vs-one

or

all-pairs

→ classify with each model, most frequent classification wins



Stop and think...

one-vs-one

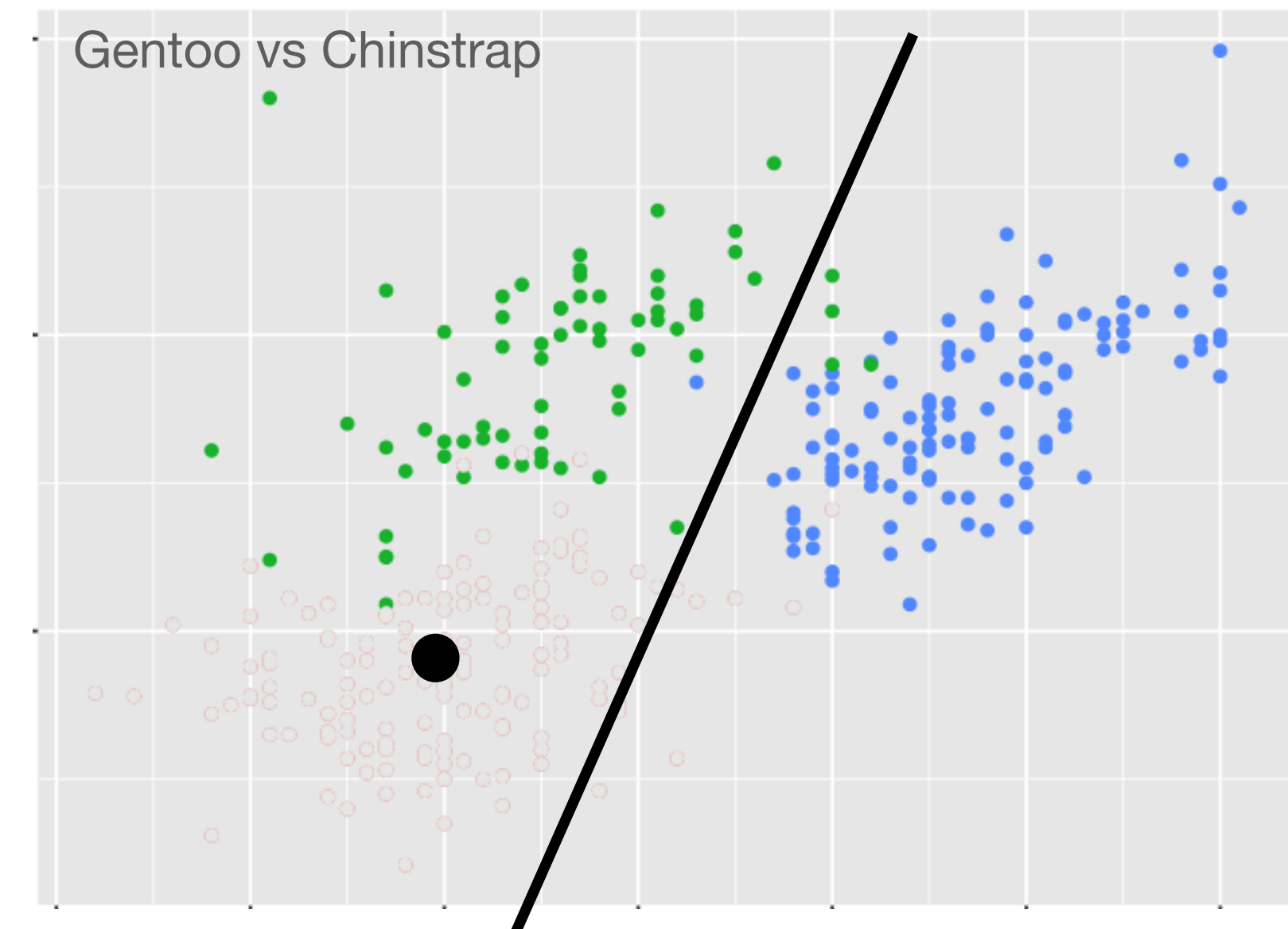
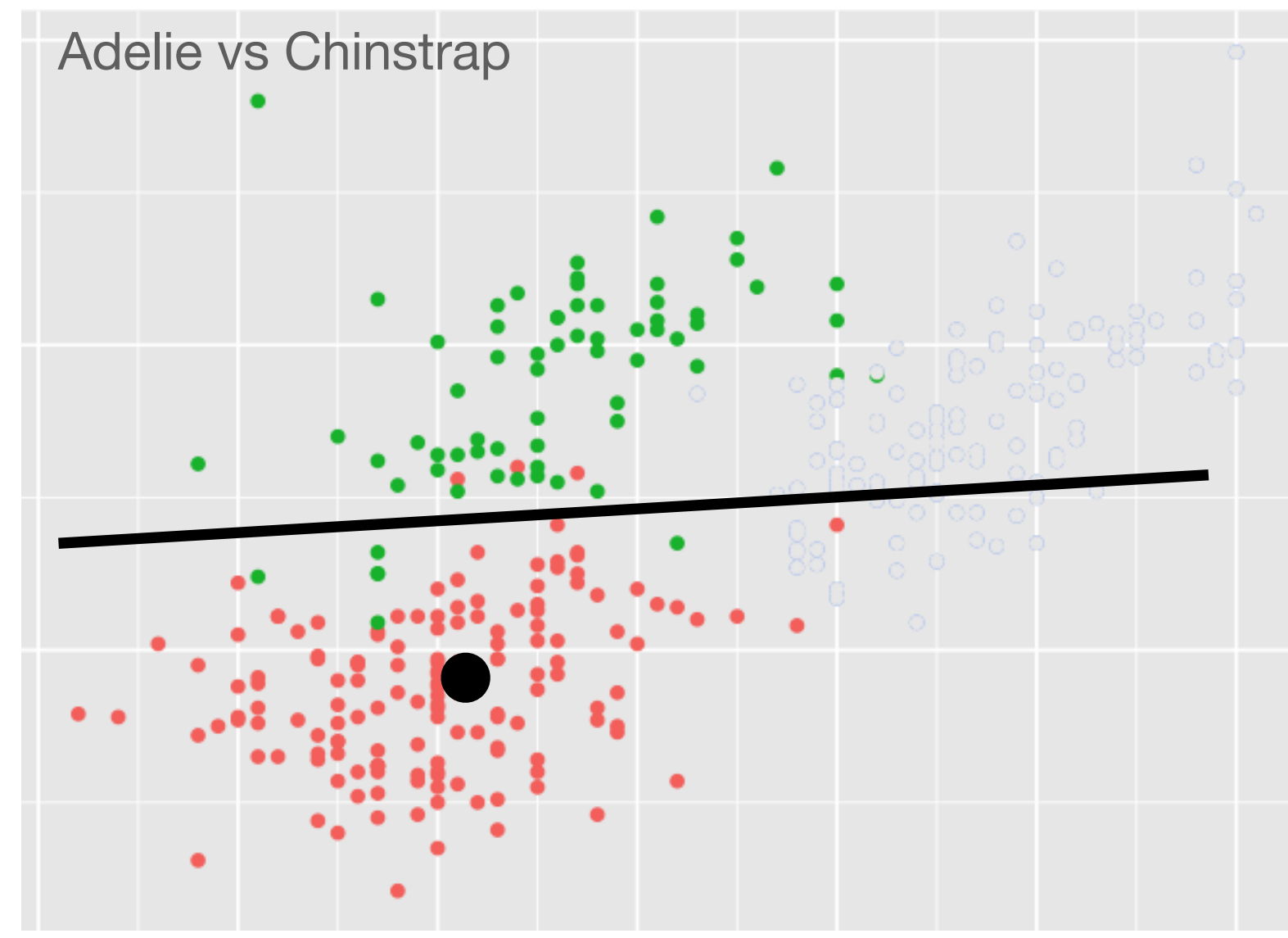
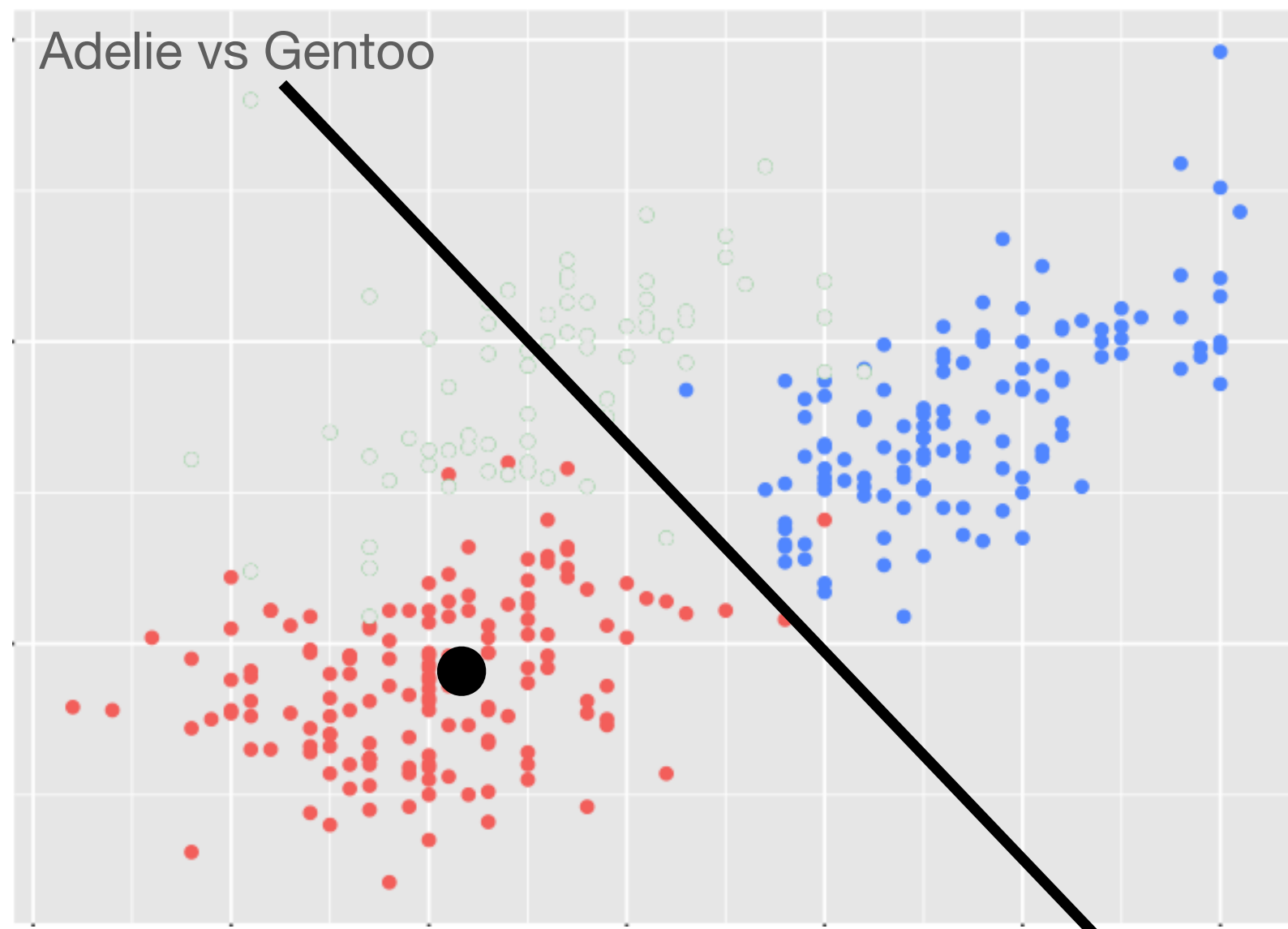
or

all-pairs

Given a test observation, how would you classify it based on the three models?

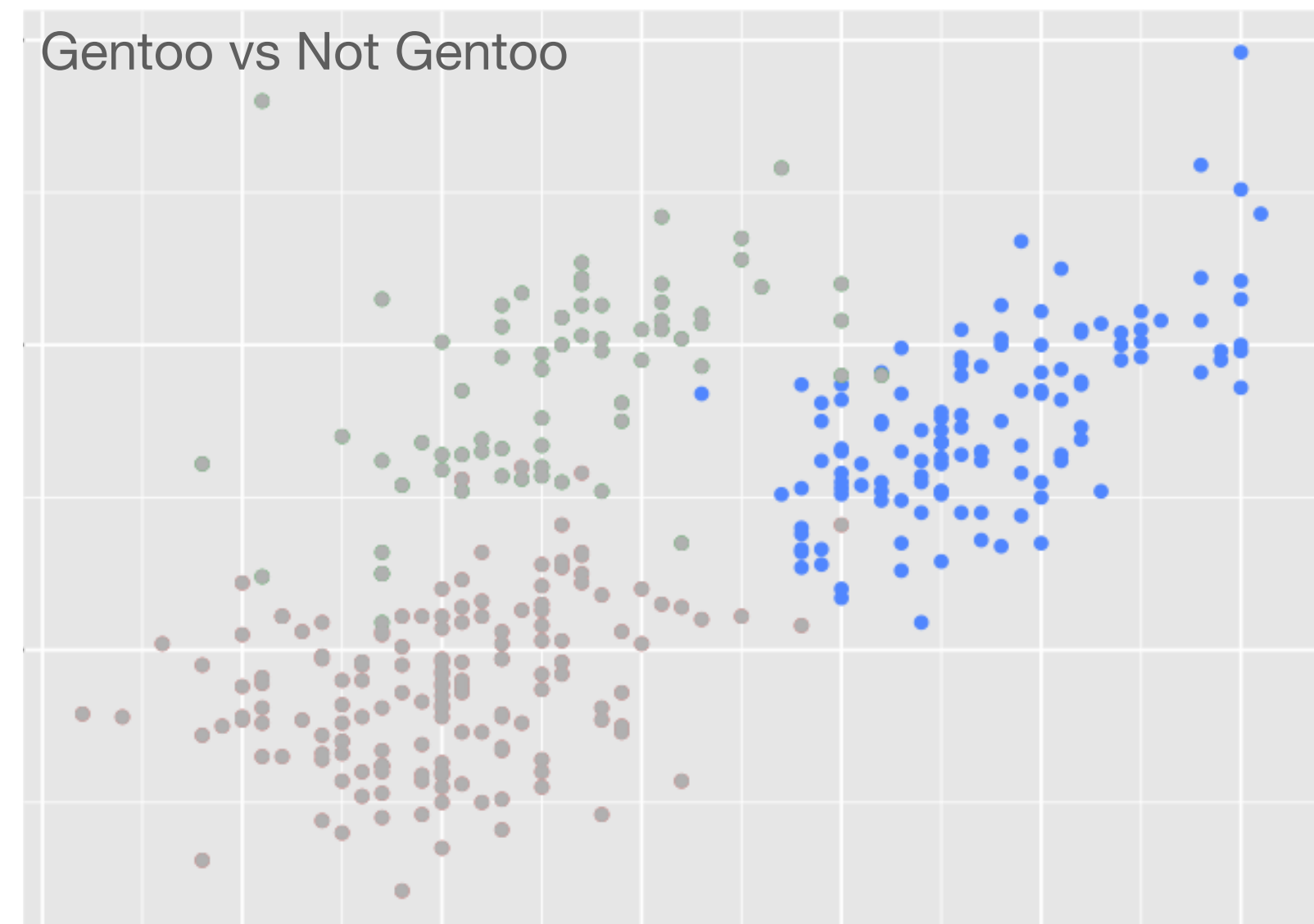
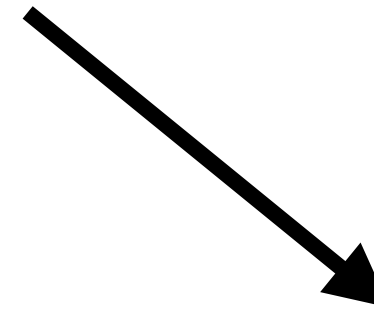
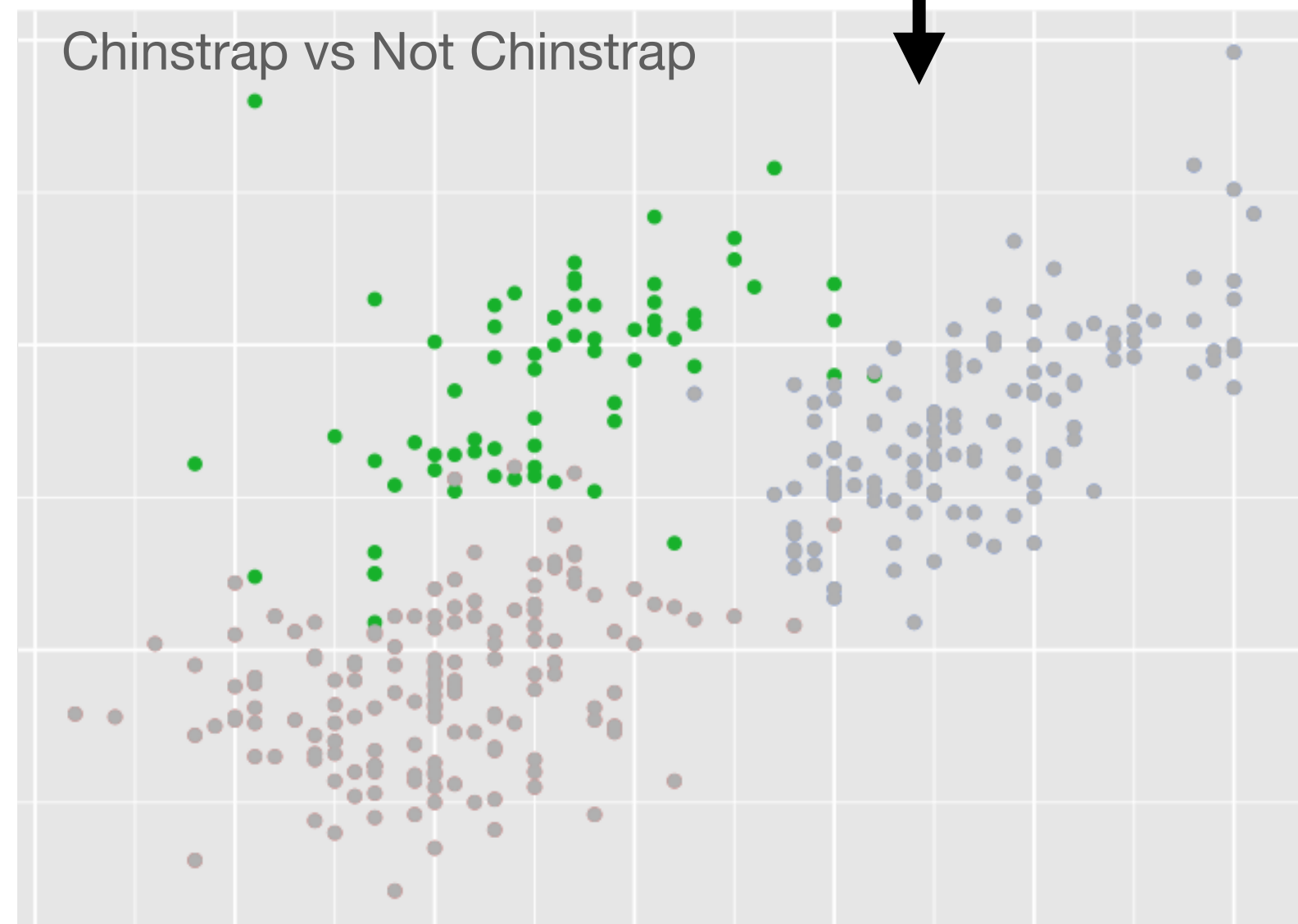
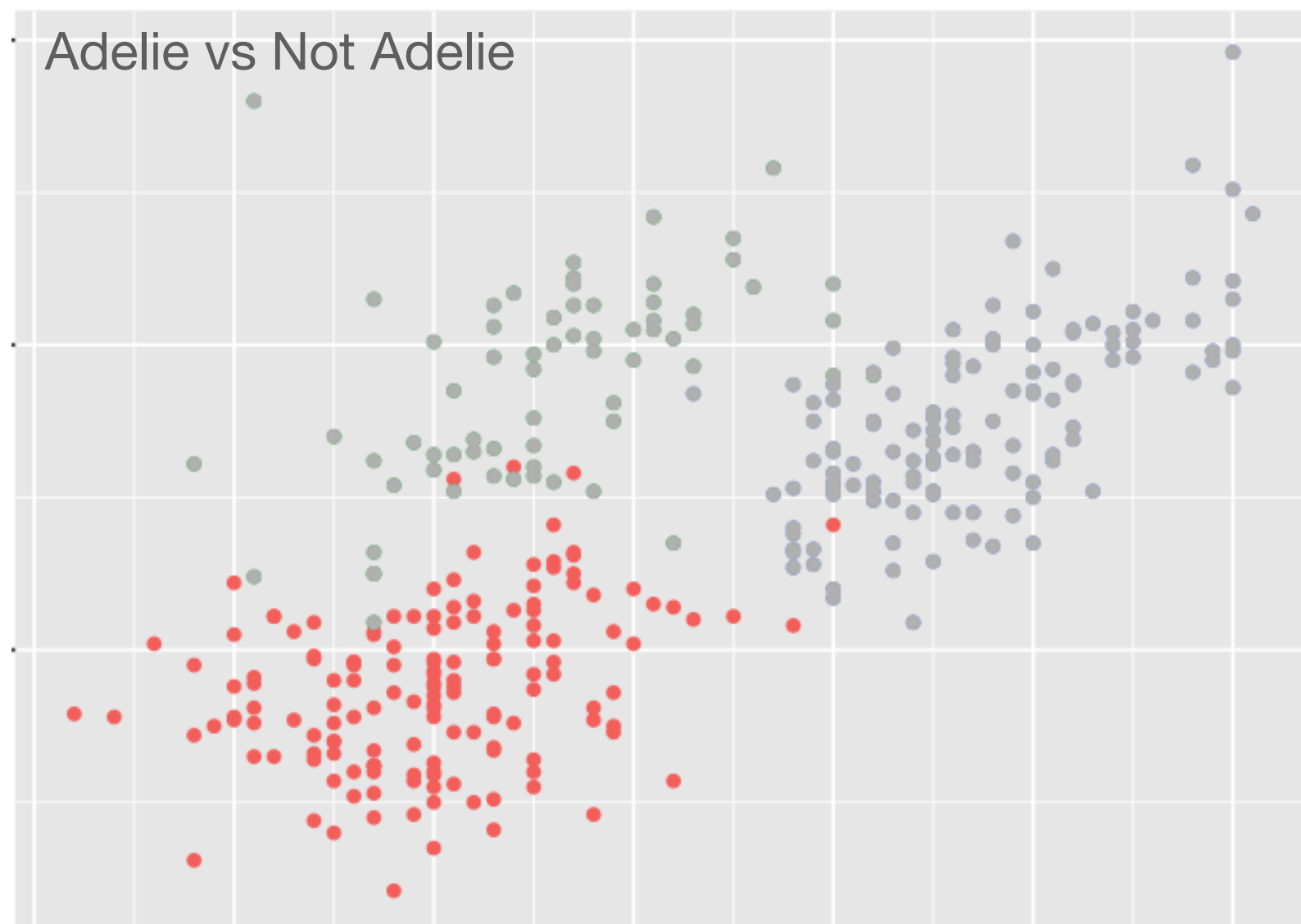
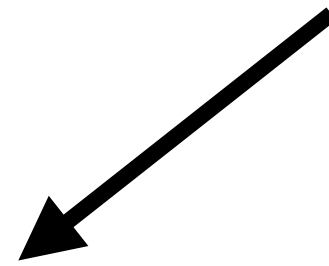
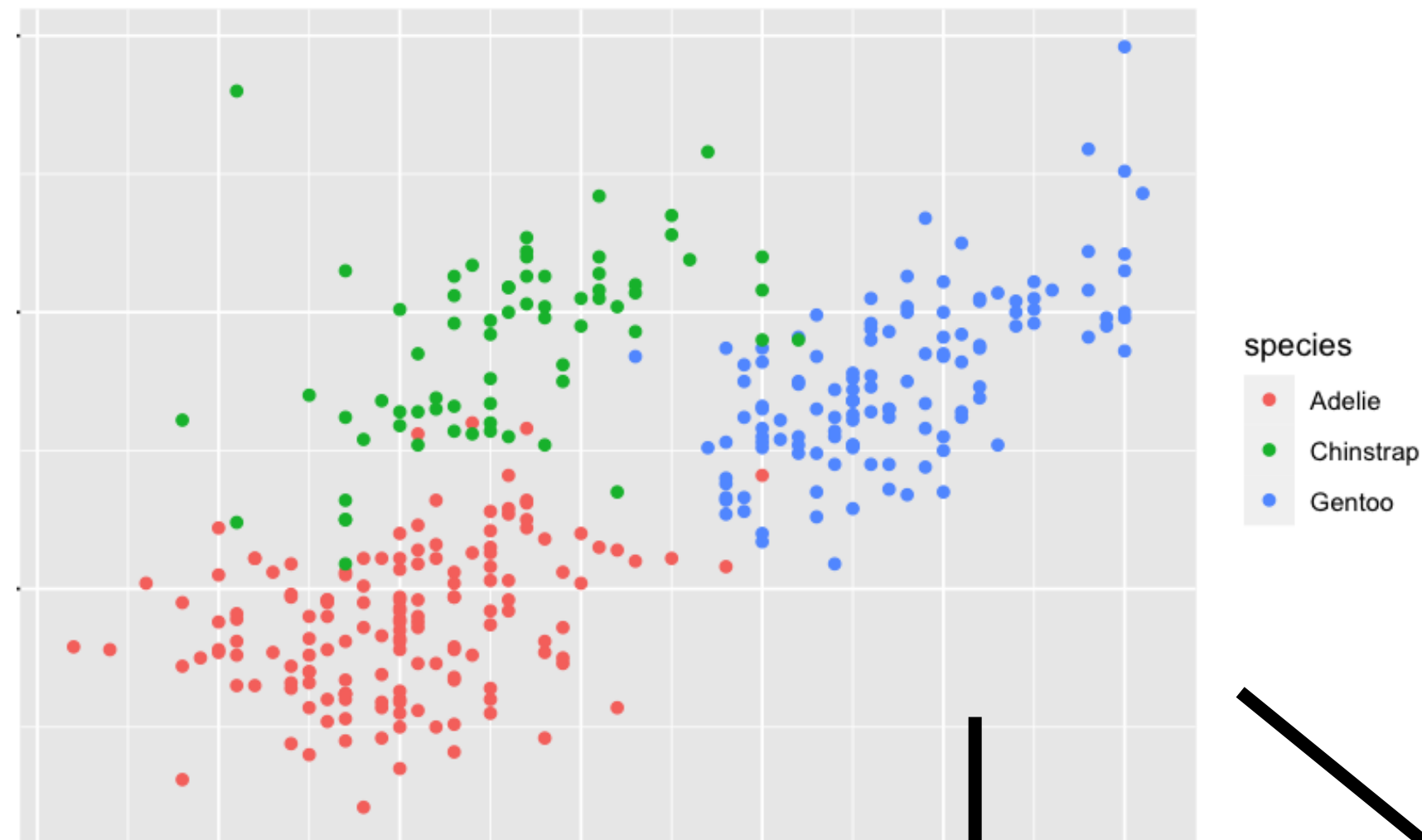
→ classify with each model, most frequent classification wins

e.g. test point gets 2 Adelie and 1 Chinstrap, classified as Adelie



SVMs with More than Two Classes

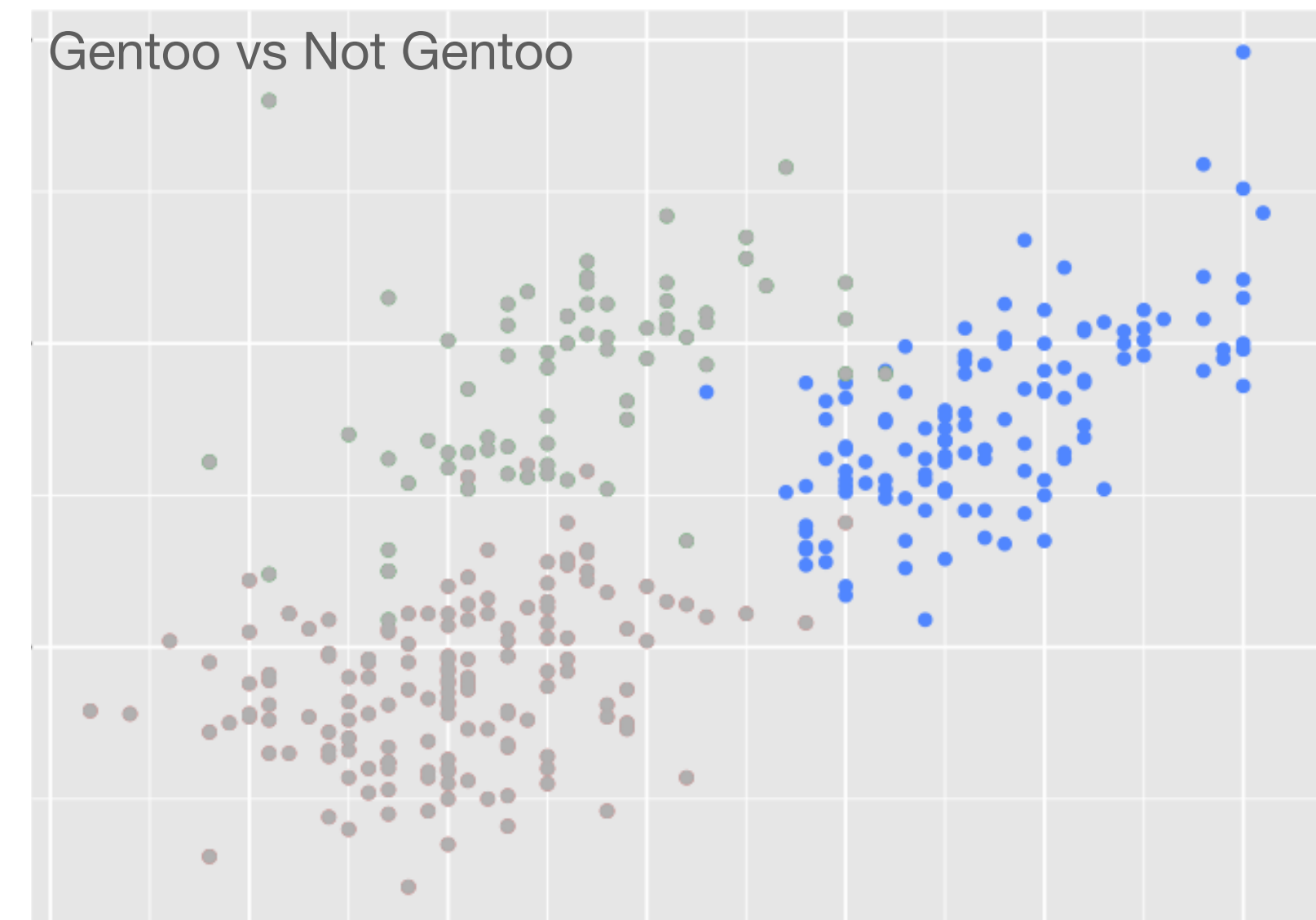
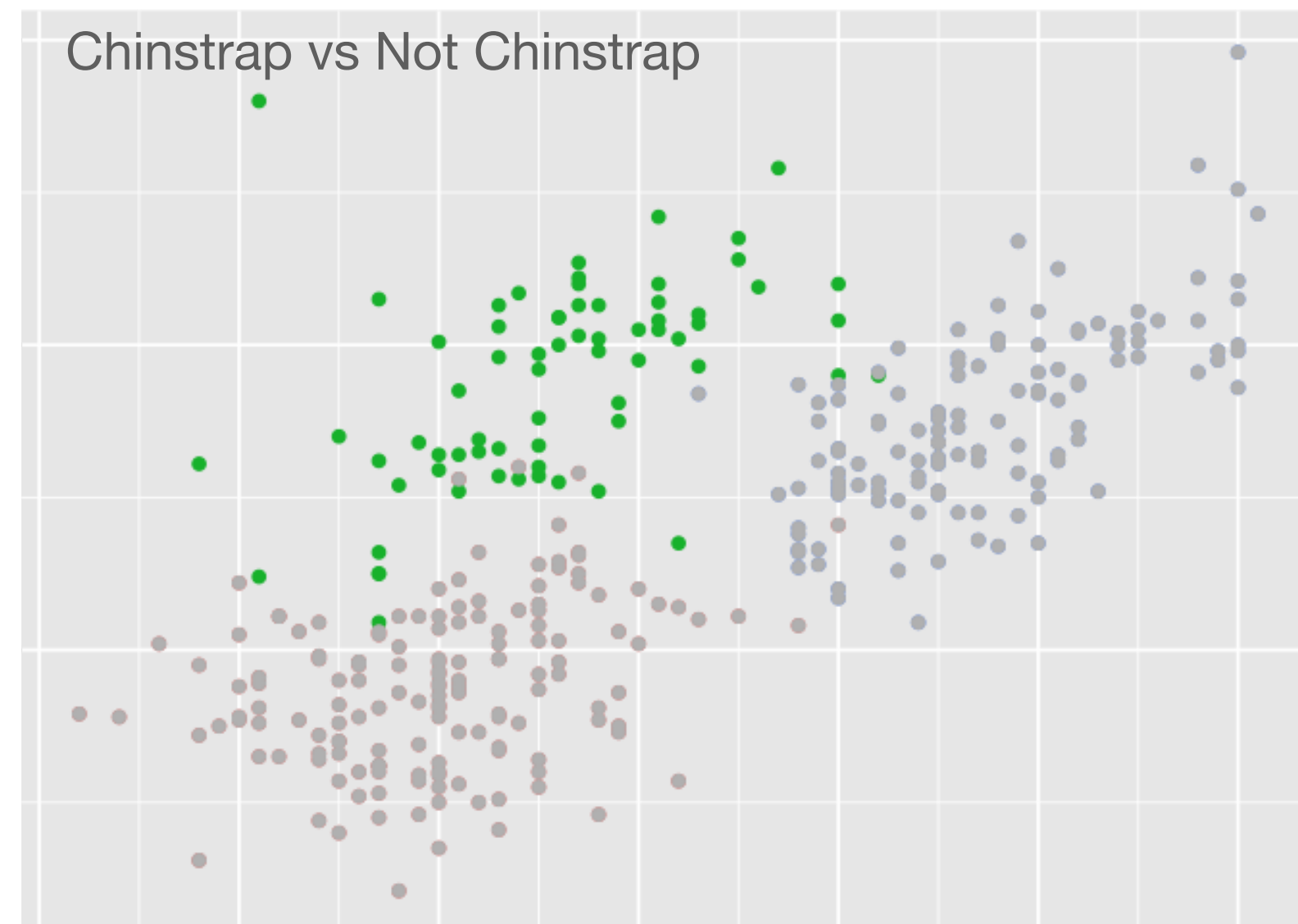
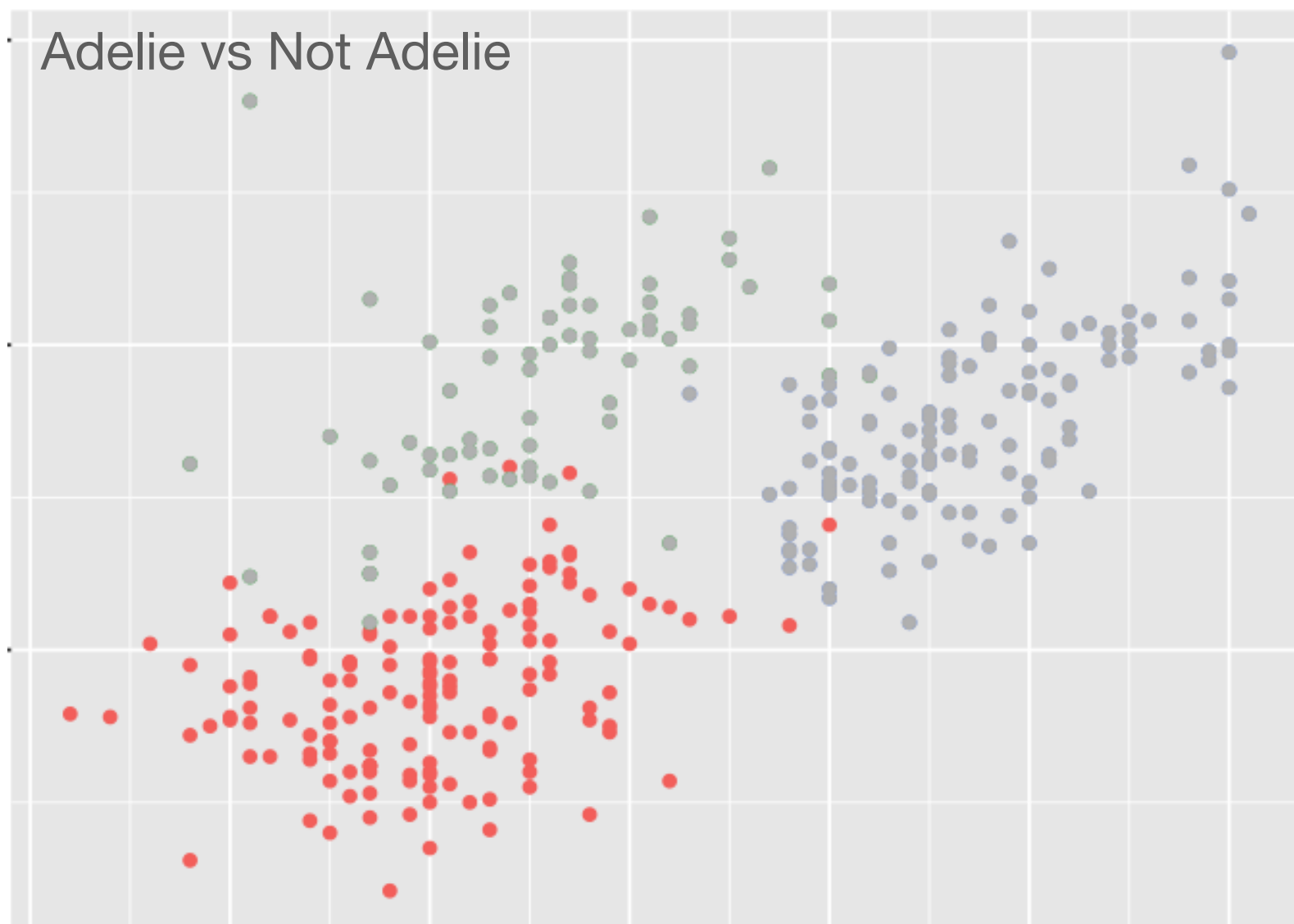
one-vs-all



Stop and think ...

Given a test observation, how would you classify it based on the three models?

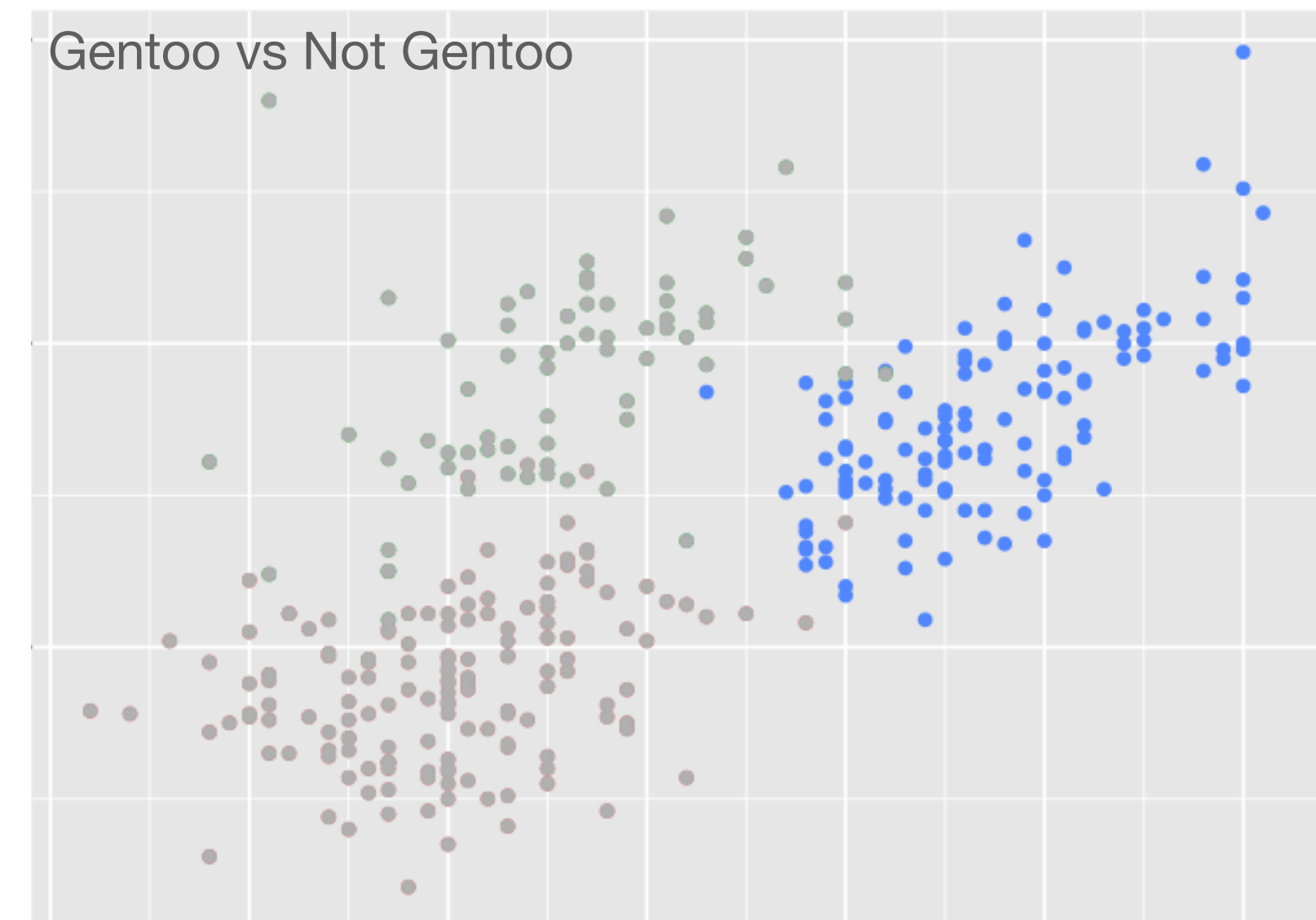
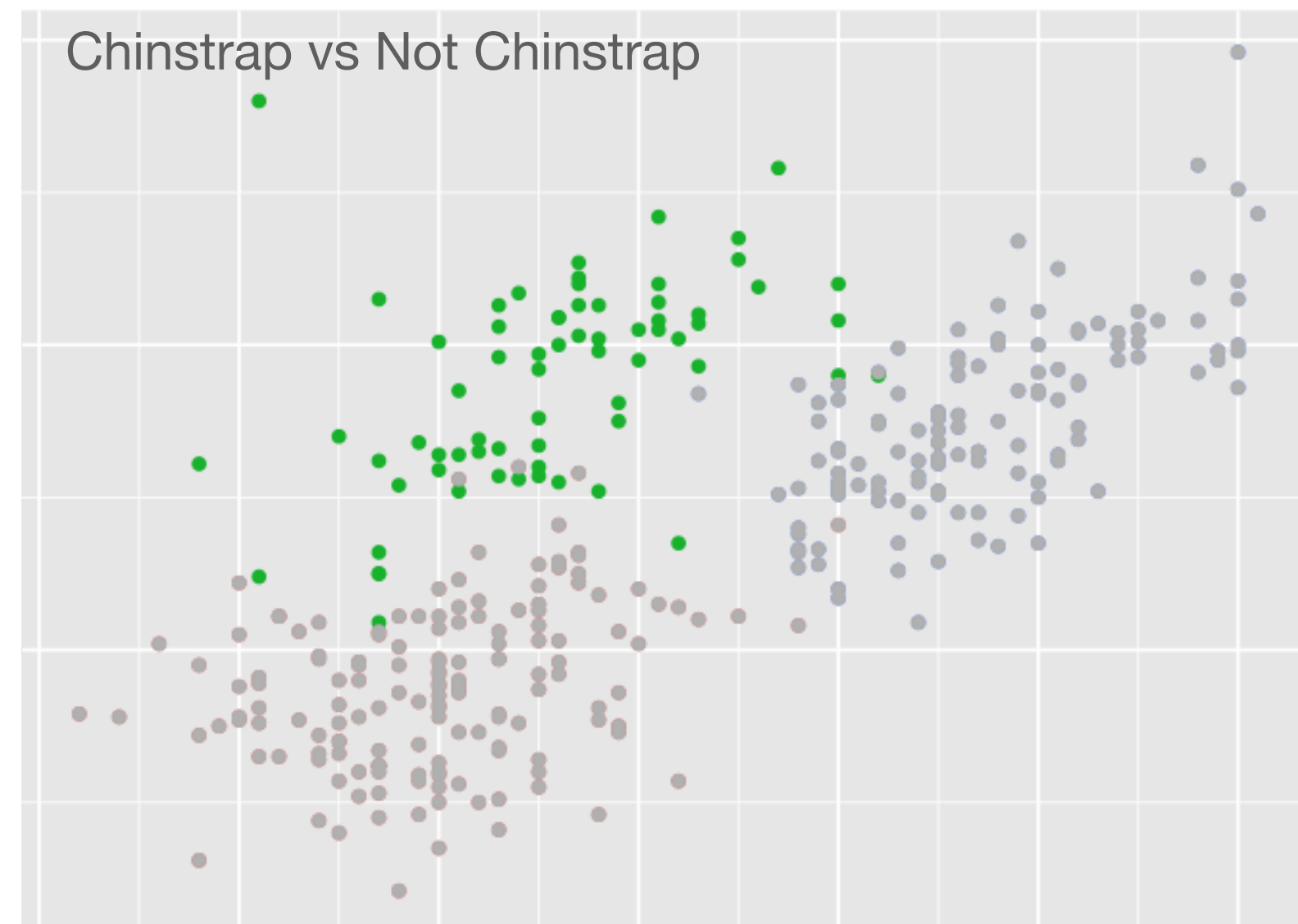
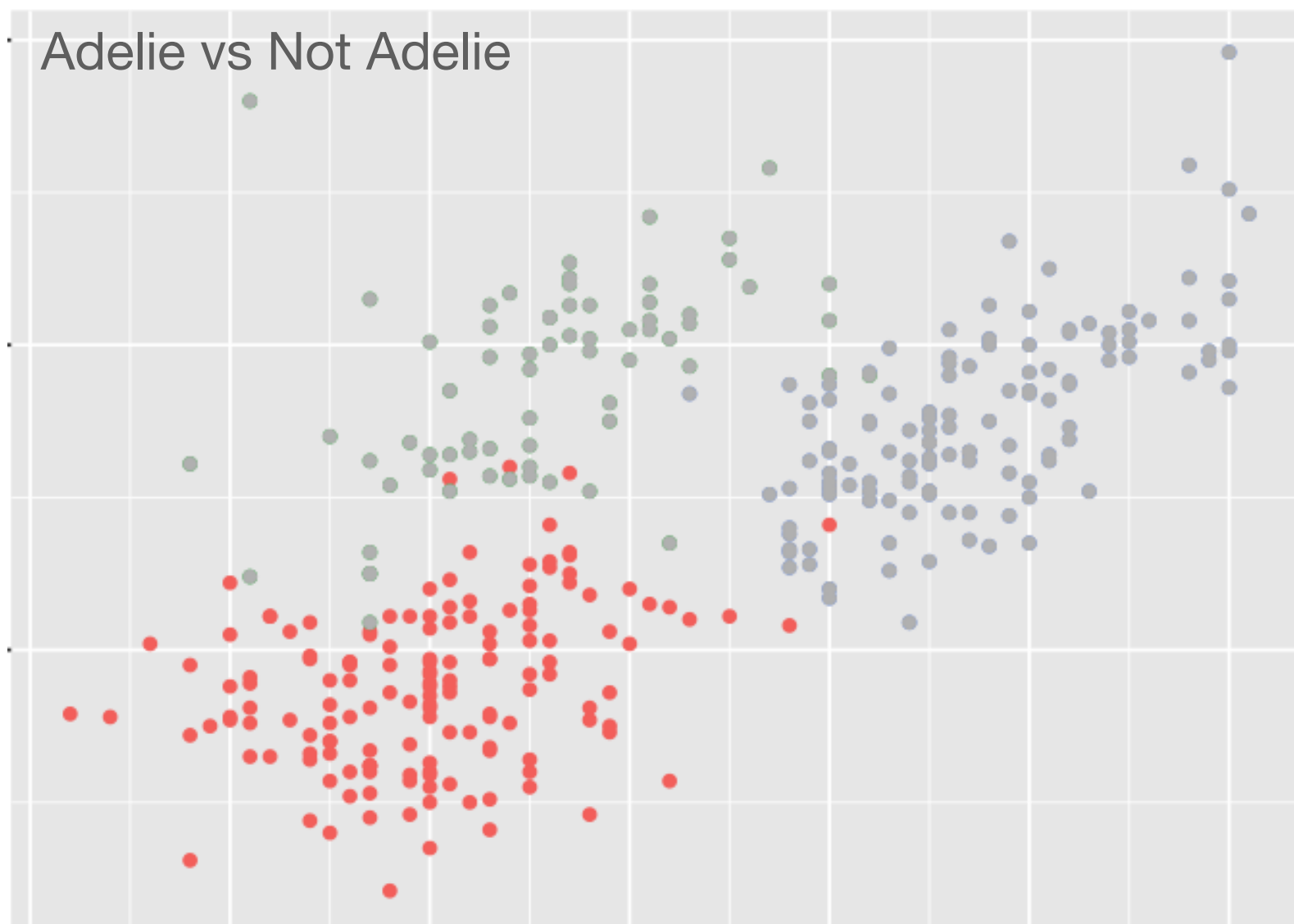
one-vs-all



Stop and think ...

Given a test observation, how would you classify it based on the three models?

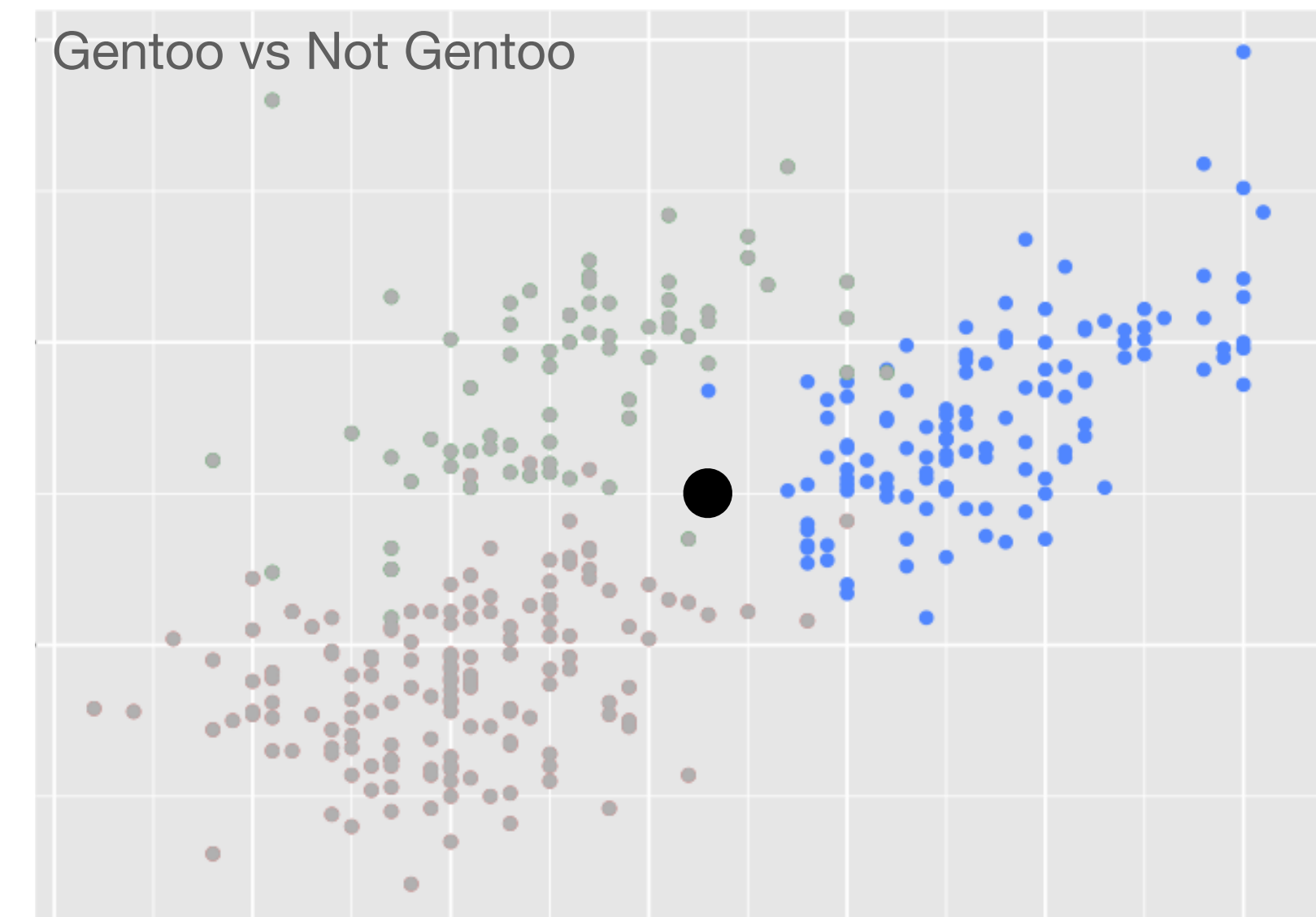
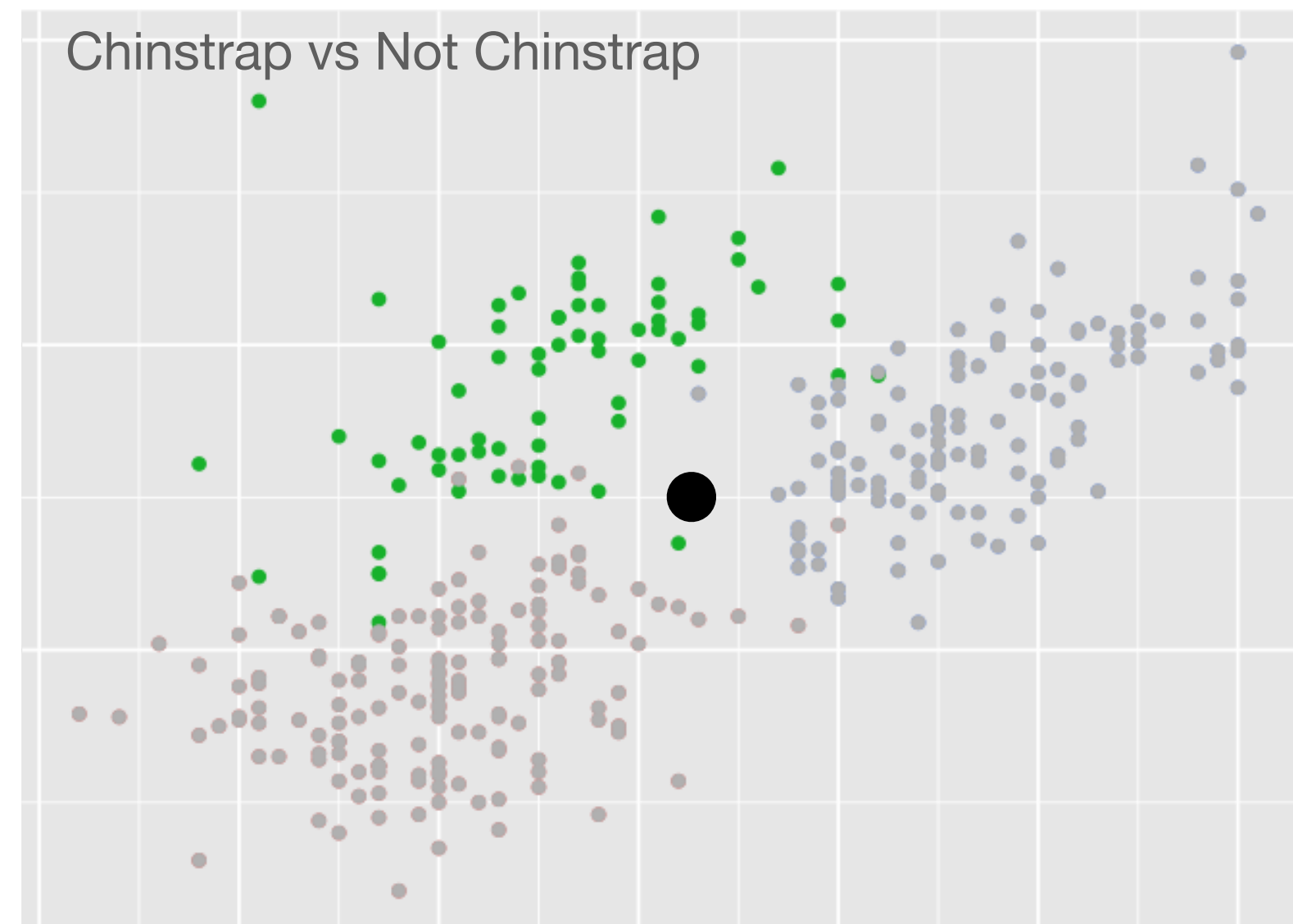
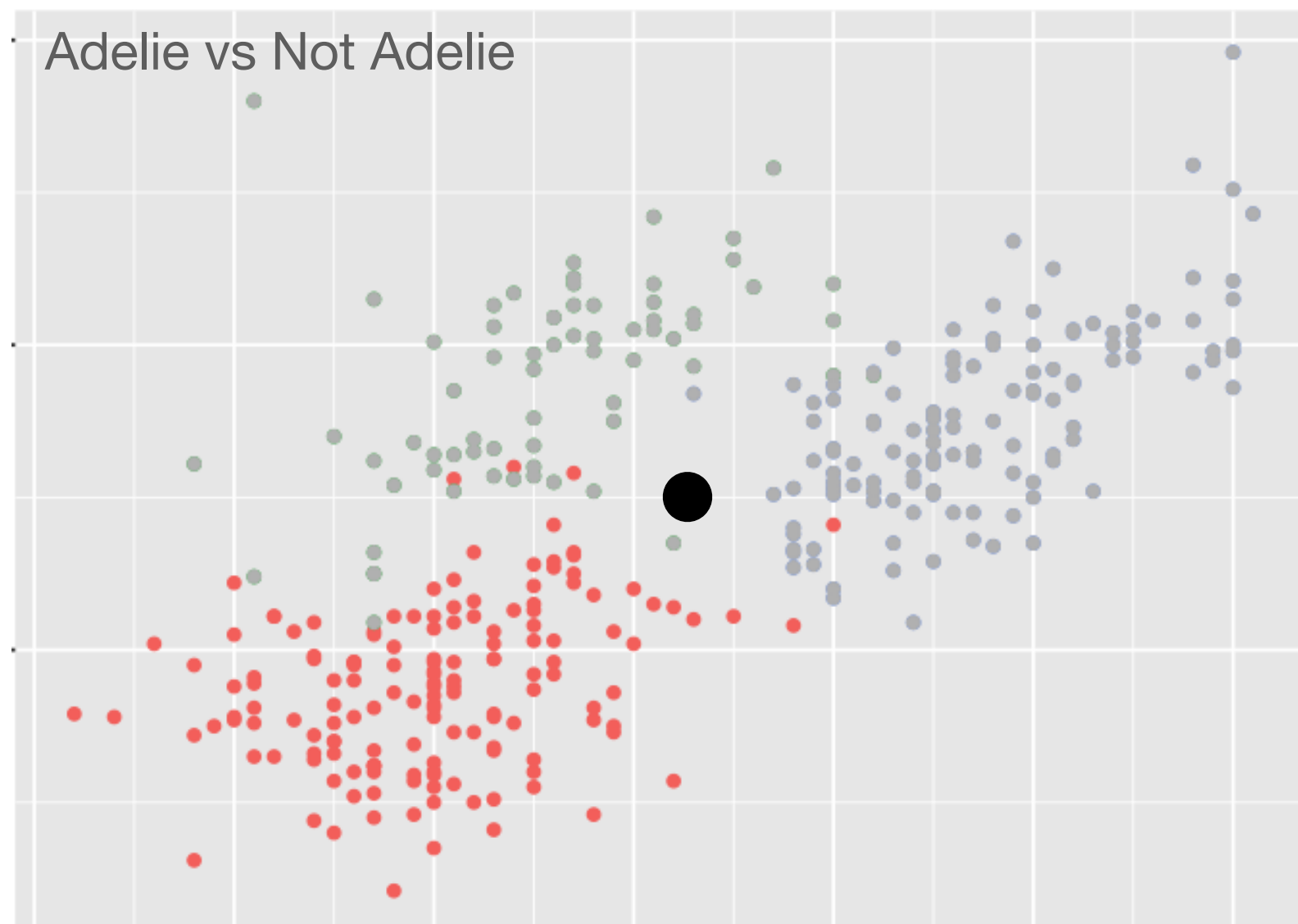
one-vs-all



Stop and think ...

Given a test observation, how would you classify it based on the three models?

one-vs-all

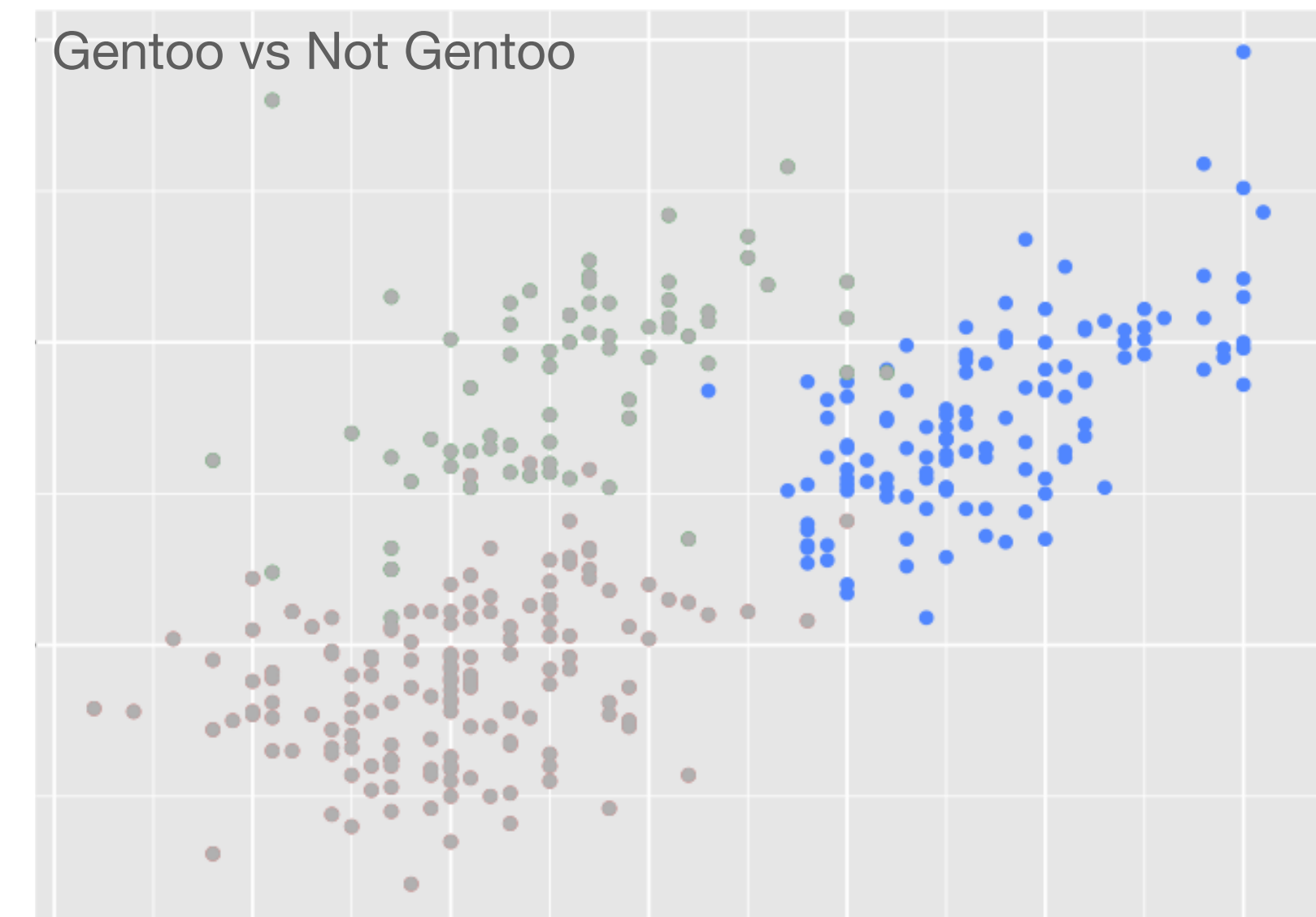
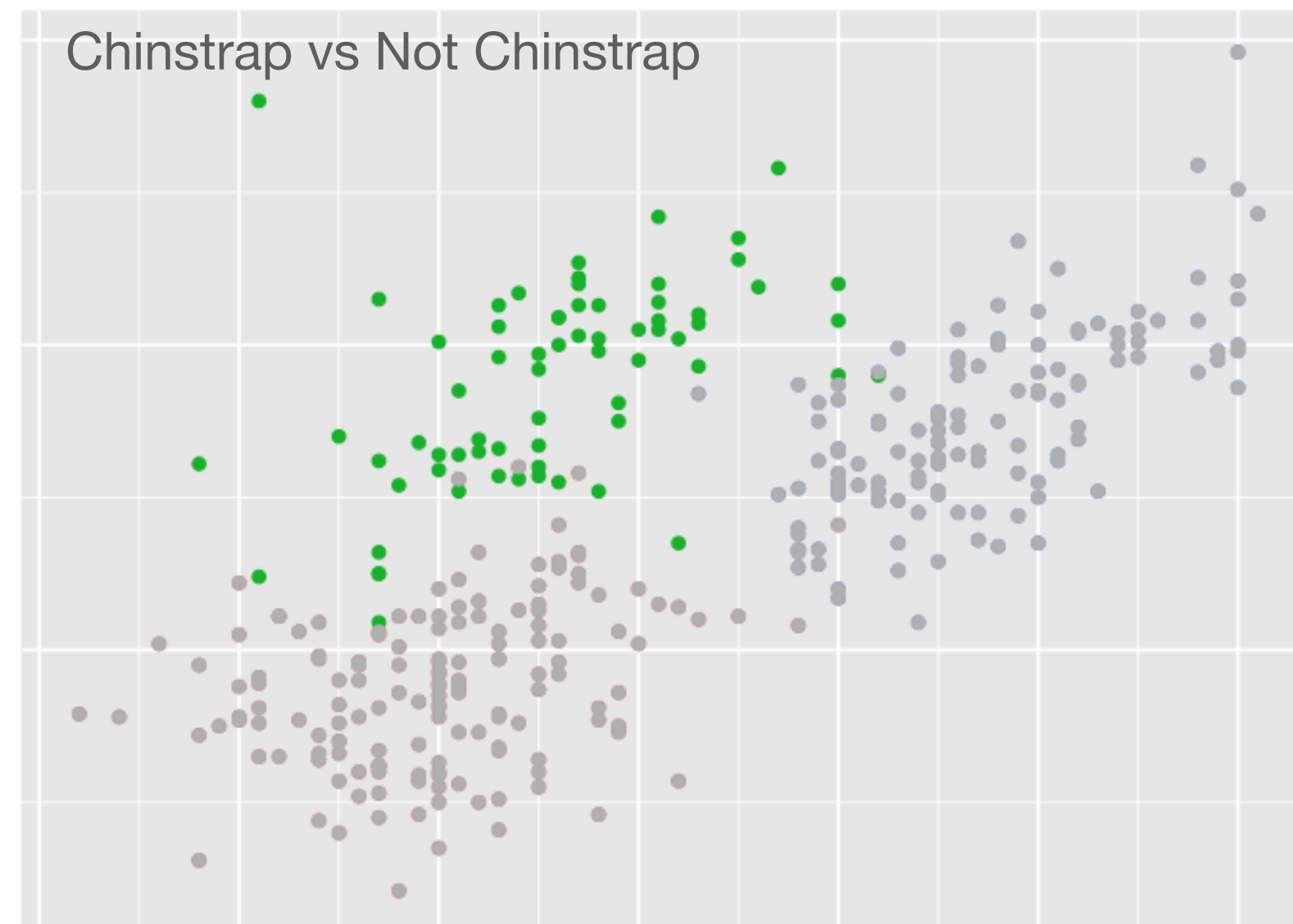
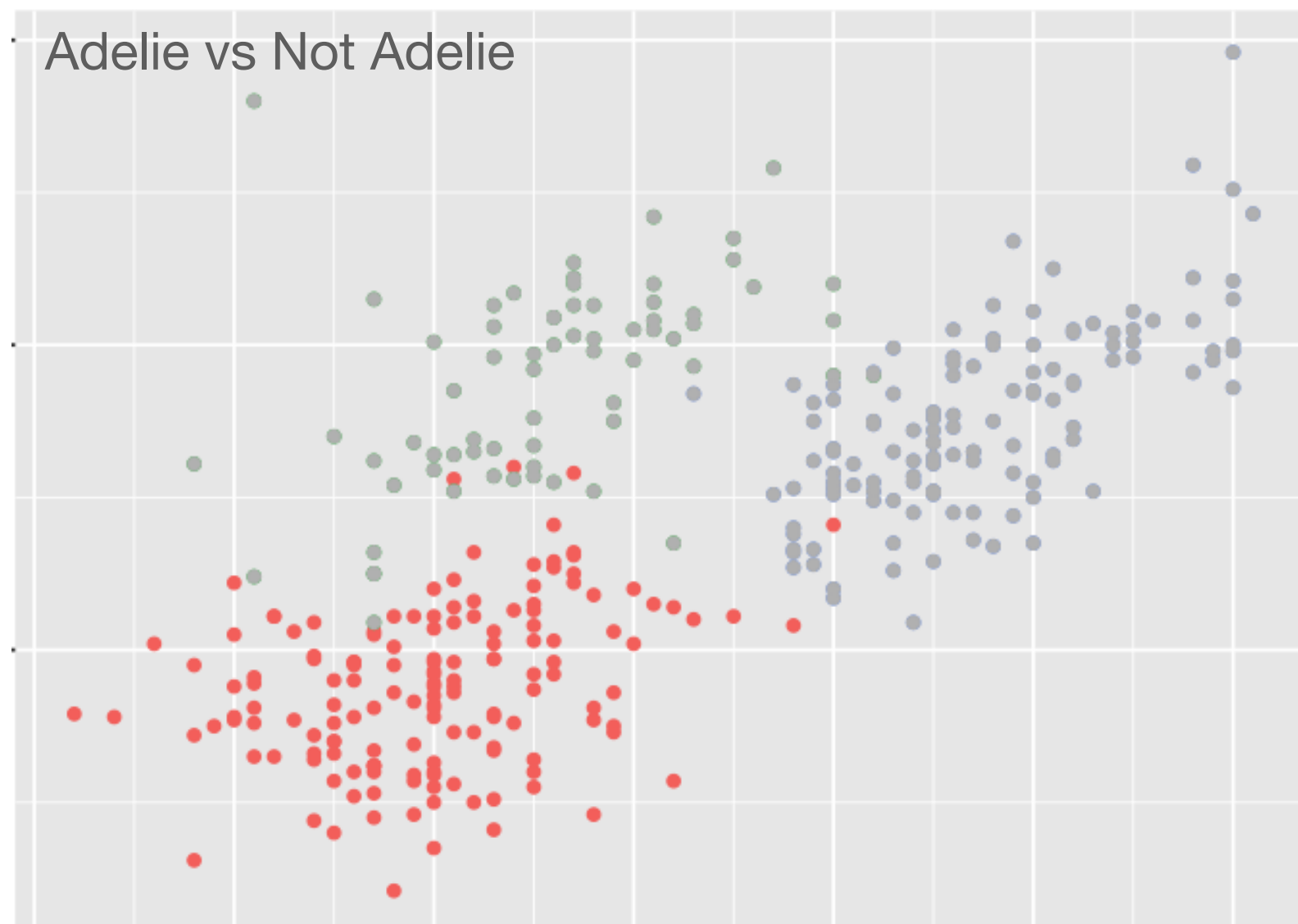


Stop and think ...

Given a test observation, how would you classify it based on the three models?

one-vs-all

→ classify with each model, furthest distance from any hyperplane wins



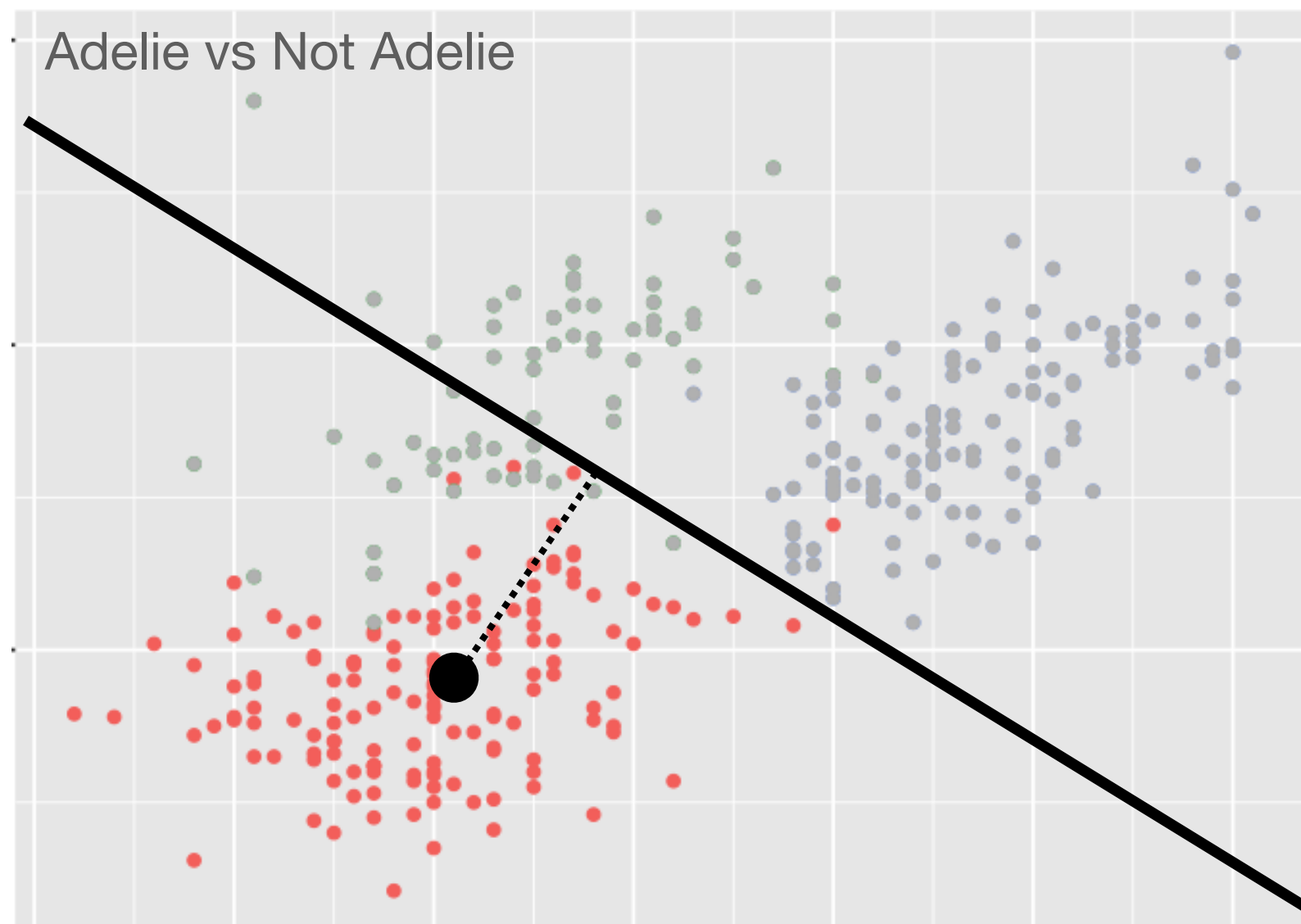
Stop and think ...

Given a test observation, how would you classify it based on the three models?

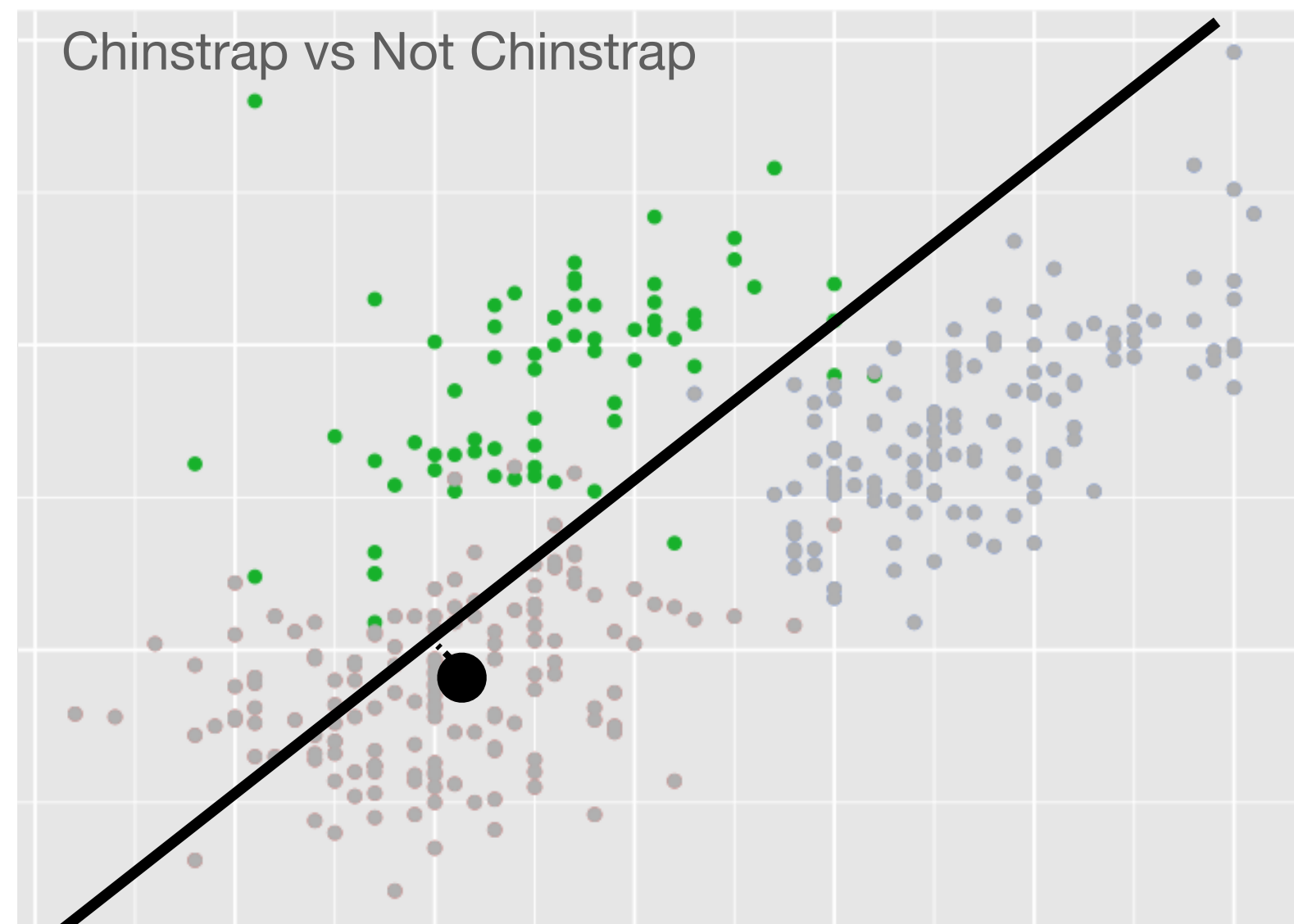
one-vs-all

→ classify with each model, furthest distance from any hyperplane wins

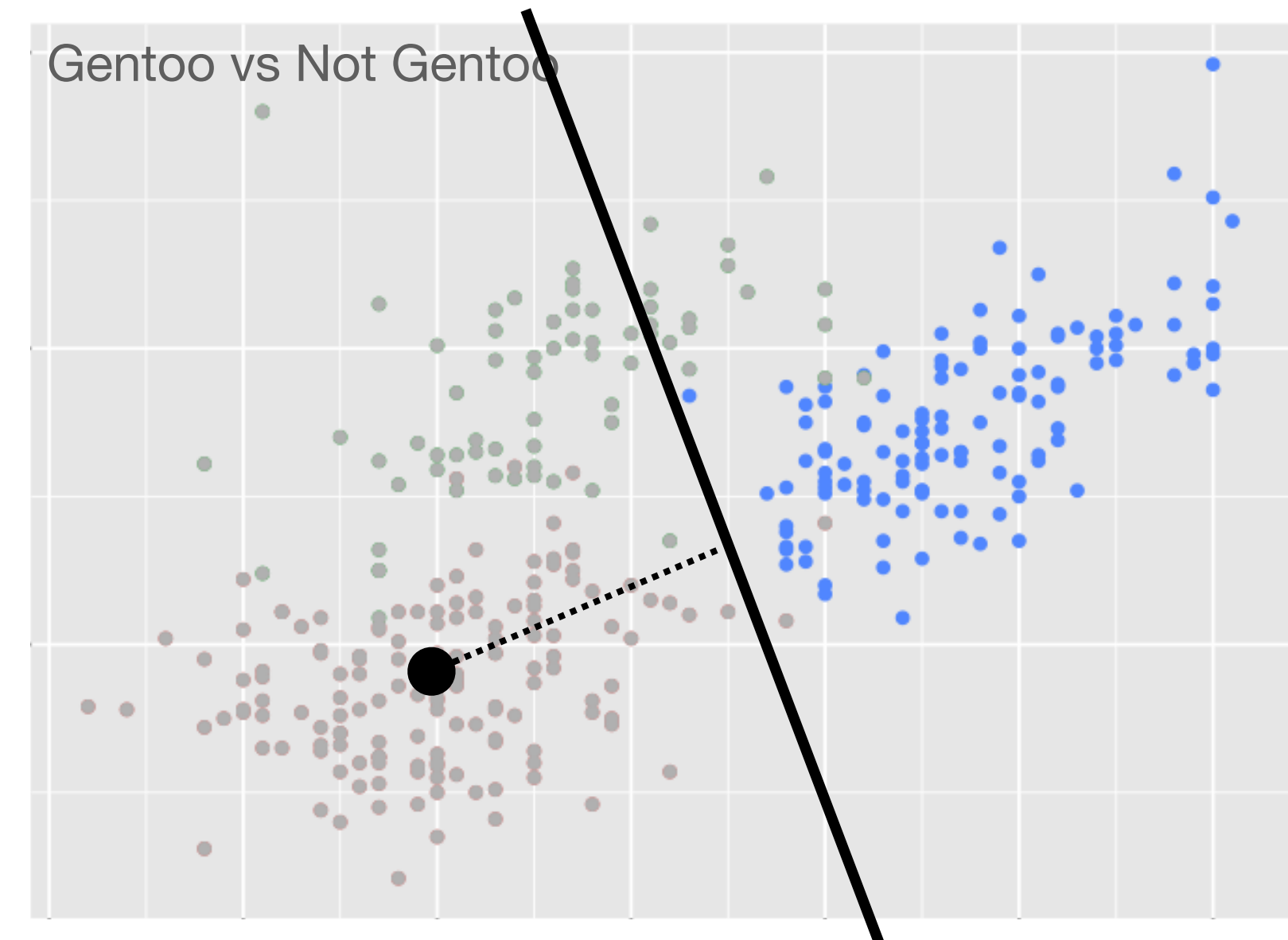
Moderate distance



Close distance



Moderate distance, wrong side of hyperplane for Gentoo



Stop and think...

For each multi-class classification scheme, one-vs-all and all-pairs, how many models do you need for:

# of classes (k)	one-vs-all	all-pairs
3 classes		
4 classes		
5 classes		
6 classes		

Stop and think...

For each multi-class classification scheme, one-vs-all and all-pairs, how many models do you need for:

	k	$k(k-1)/2$
# of classes (k)	one-vs-all	all-pairs
3 classes	3	3
4 classes	4	6
5 classes	5	10
6 classes	6	15

The background of the slide is a black field filled with a complex, abstract pattern of thin, light blue lines. These lines form a series of concentric, overlapping circles and arcs that create a sense of depth and movement, resembling a stylized tunnel or a series of ripples in water. The lines are most dense in the center and become more sparse towards the edges.

SVM Considerations

Comparison to Other Models

We will only use SVMs for classification in this class, but support vector regression exists

Comparison to Other Models

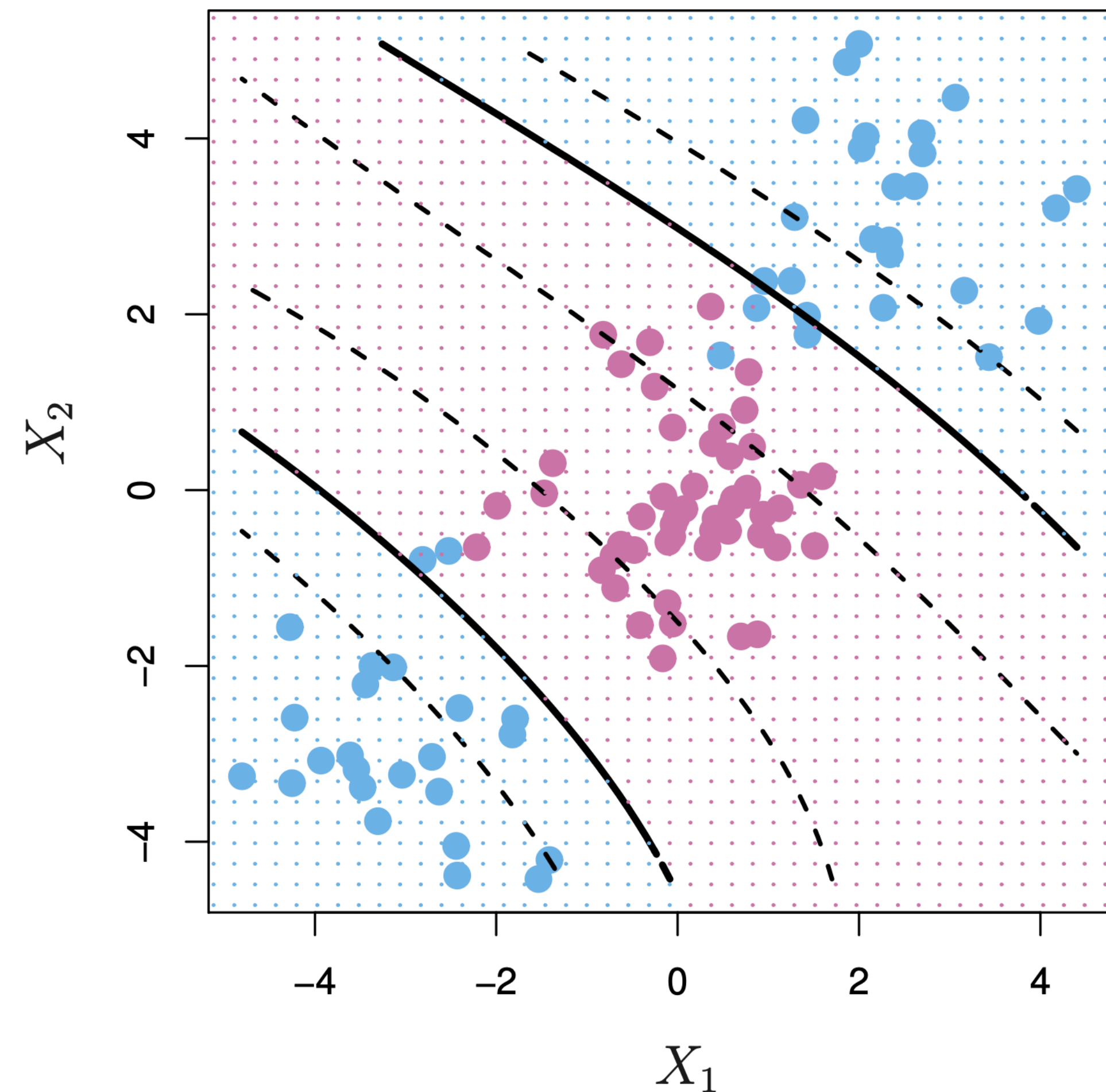
We will only use SVMs for classification in this class, but support vector regression exists

Very similar to logistic regression, just makes higher-dimensional computation easy



Cautions and Considerations

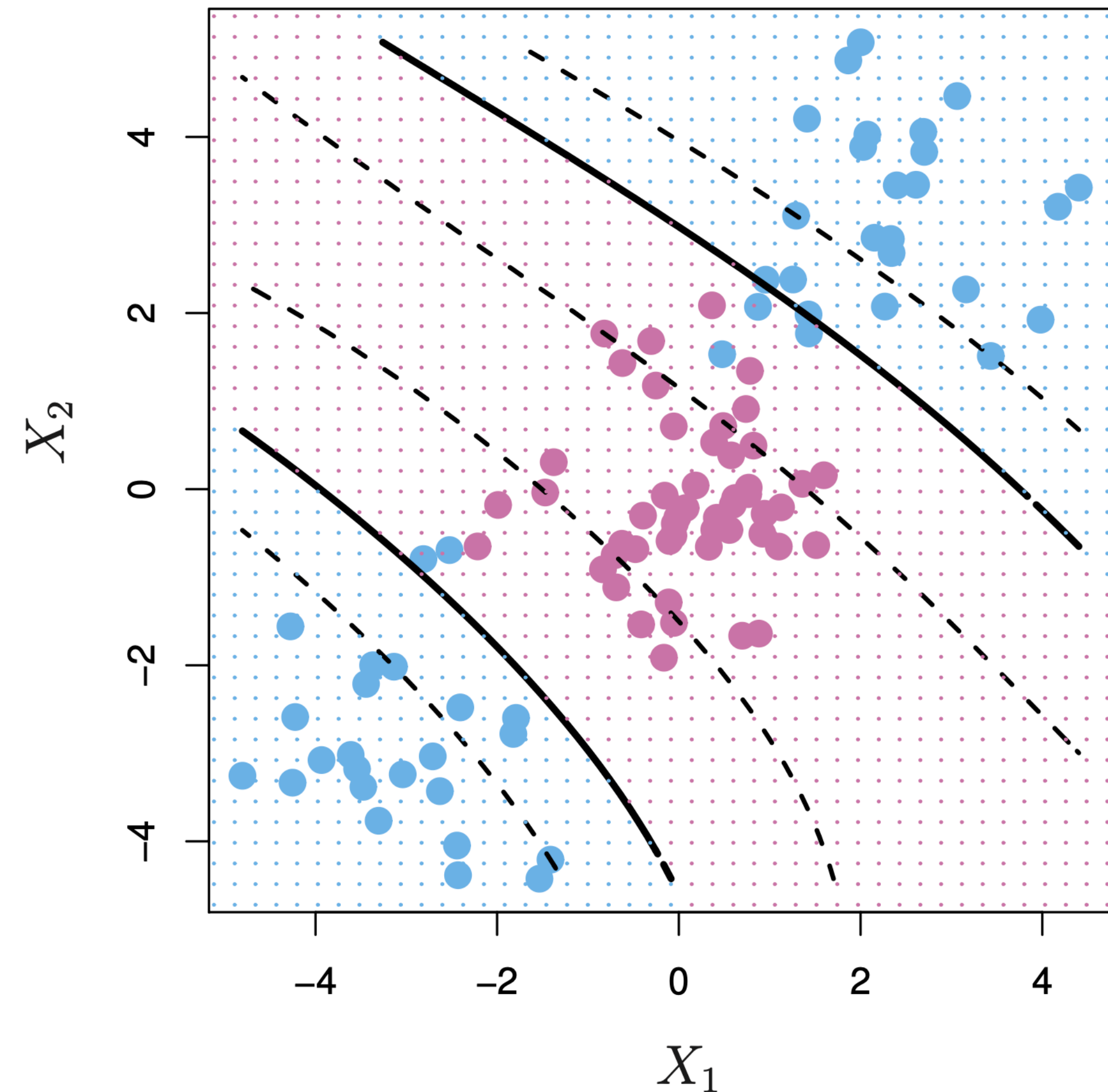
Need to transform and/or scale predictors



Cautions and Considerations

Need to transform and/or scale predictors

Interpretation can be tricky

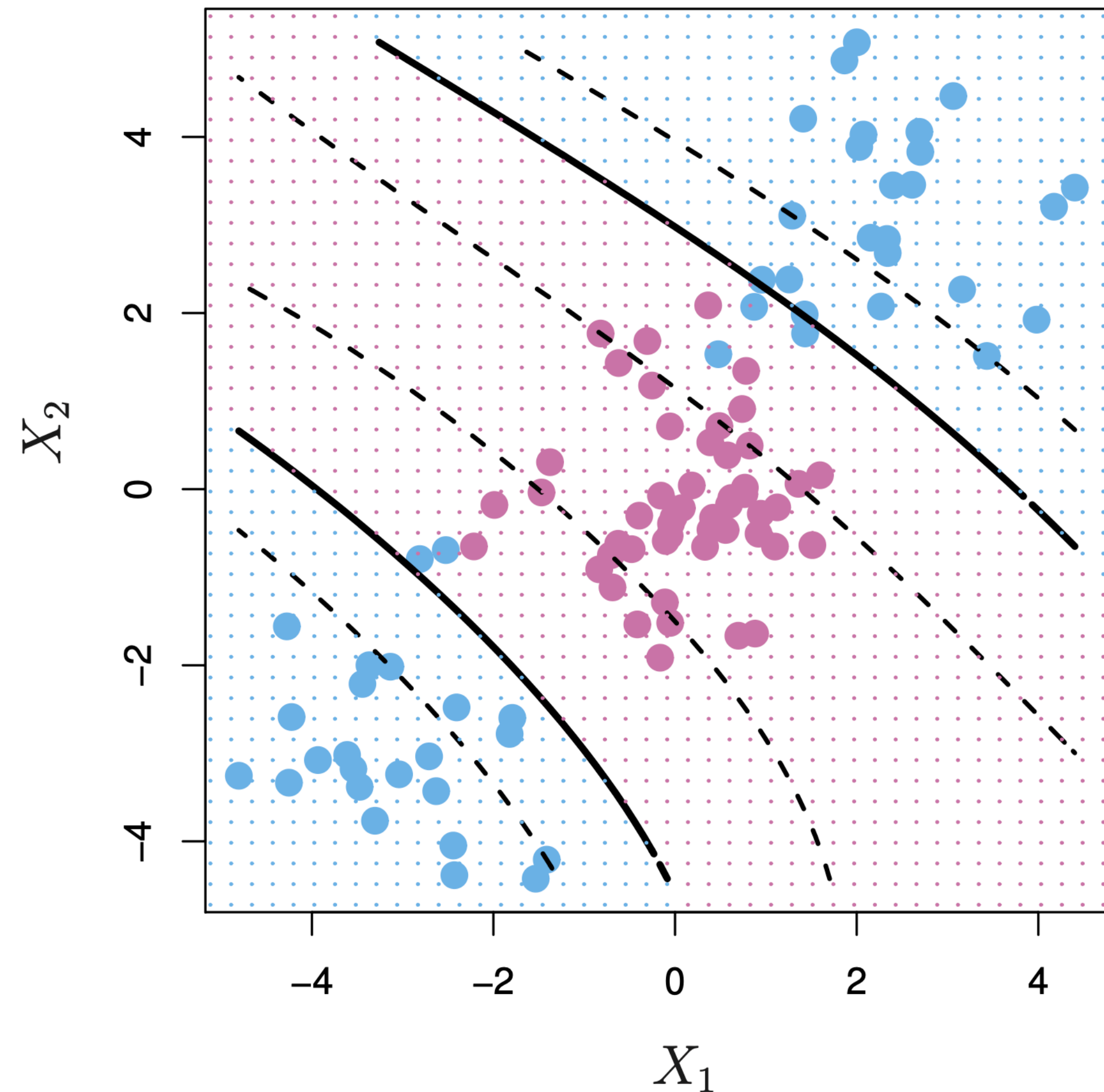


Cautions and Considerations

Need to transform and/or scale predictors

Interpretation can be tricky

Visualization can only be done in two dimensions at a time



Lab 9.2